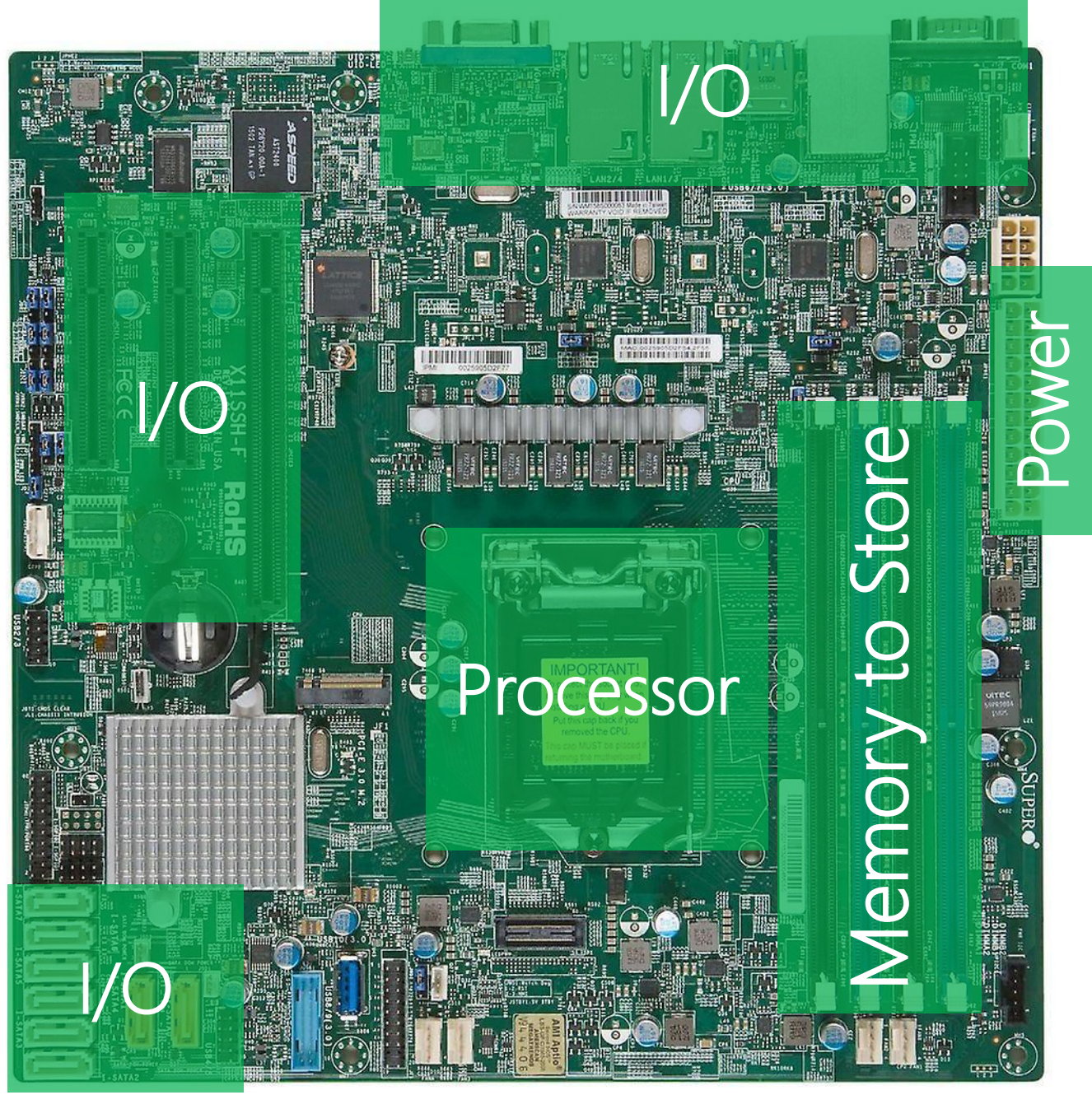
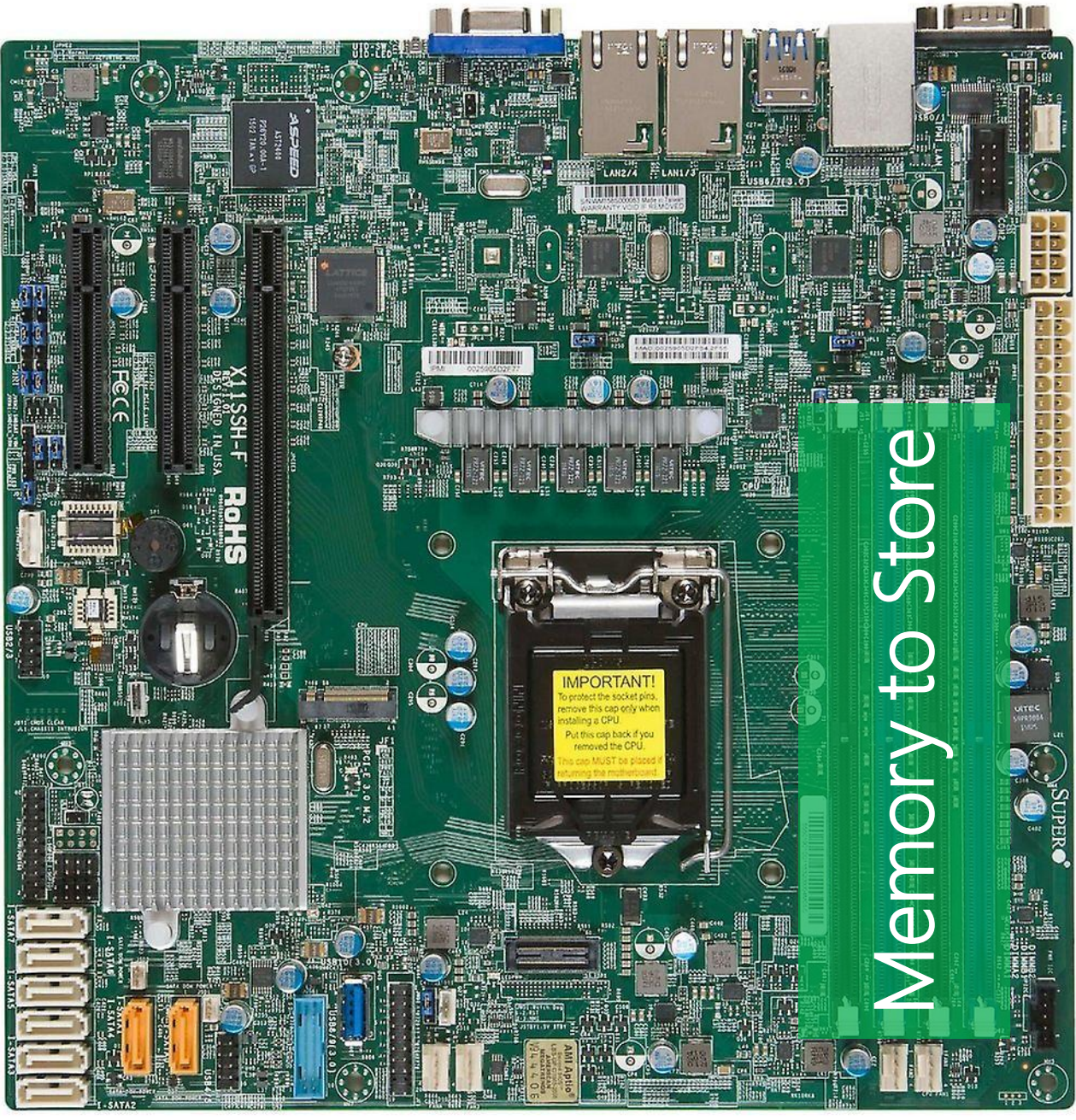

No More Lec/Lab >> But we have Lab Sections!
Bonus Feedback as Lab12





Memory to Store

Design Memory

To Store Data + Code

Design Memory

Data → Numbers

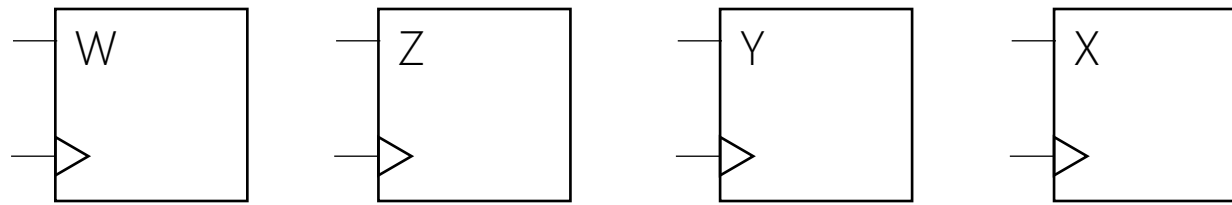
Design Memory

Each Number $\in [\text{min}, \text{max}] \rightarrow \text{Binary System}$

Design Memory

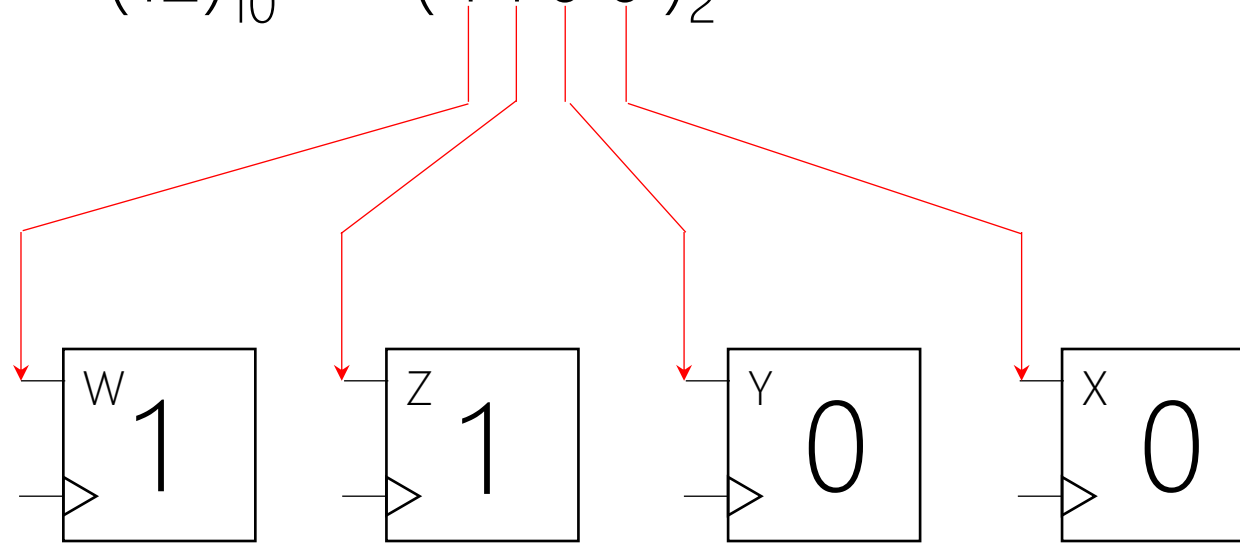
Each Number $\in [0, 2^n) \rightarrow n$ Cells

Number System's Position $\leftrightarrow$ Cells



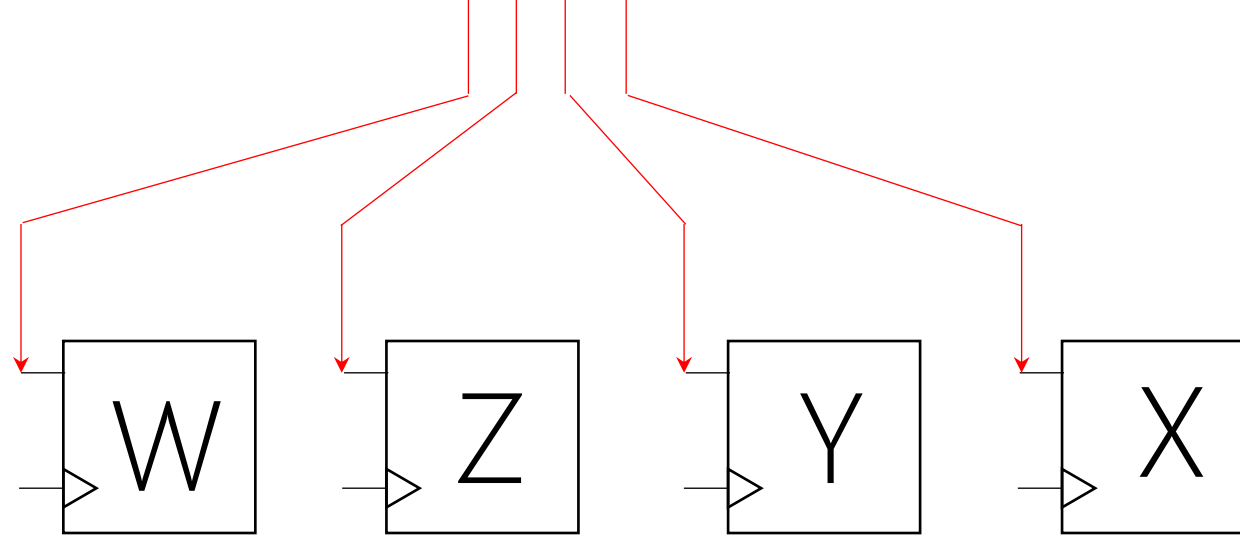
$n=4 \rightarrow$ Each Number $\in [0, 2^4=16) \rightarrow$ 4 Cells (1 word)

$$(12)_{10} = (1100)_2$$



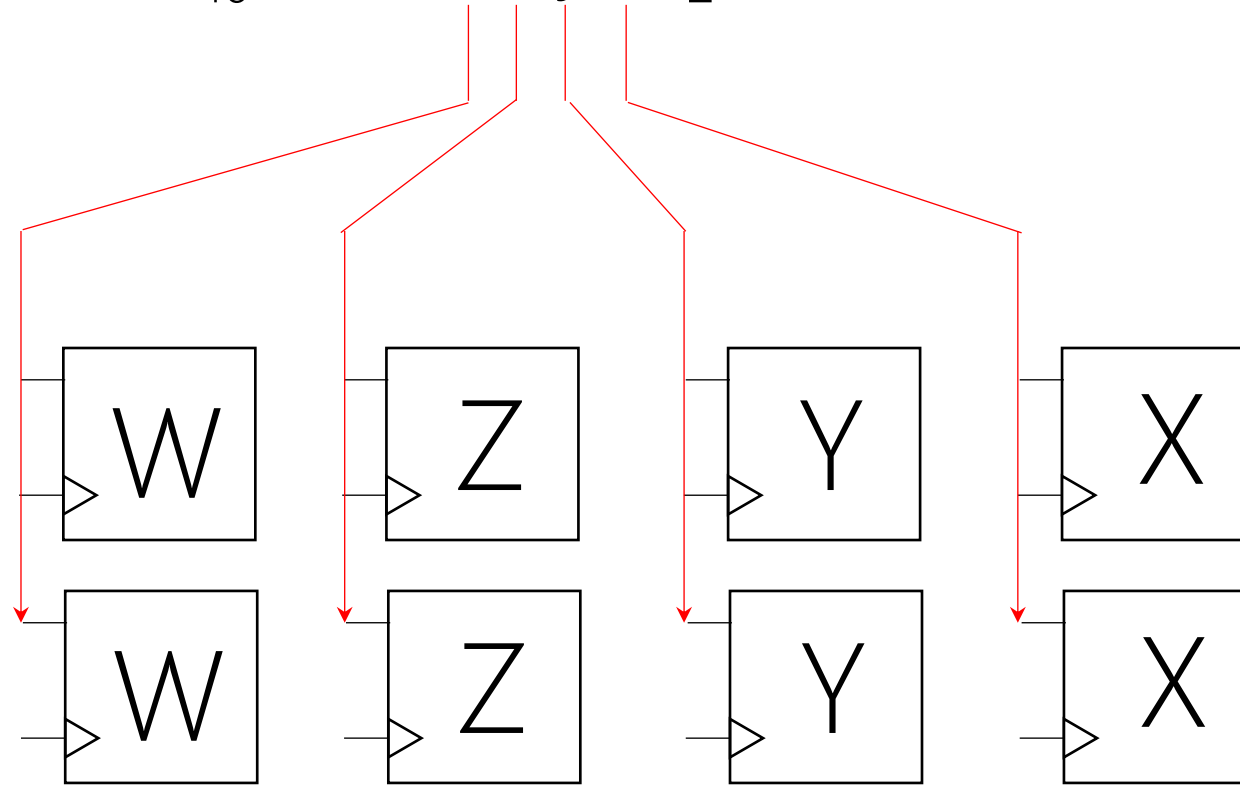
$n=4 \rightarrow$ Each Number $\in [0, 2^4=16) \rightarrow 4$ Cells

$$(N)_{10} = (wzyx)_2$$



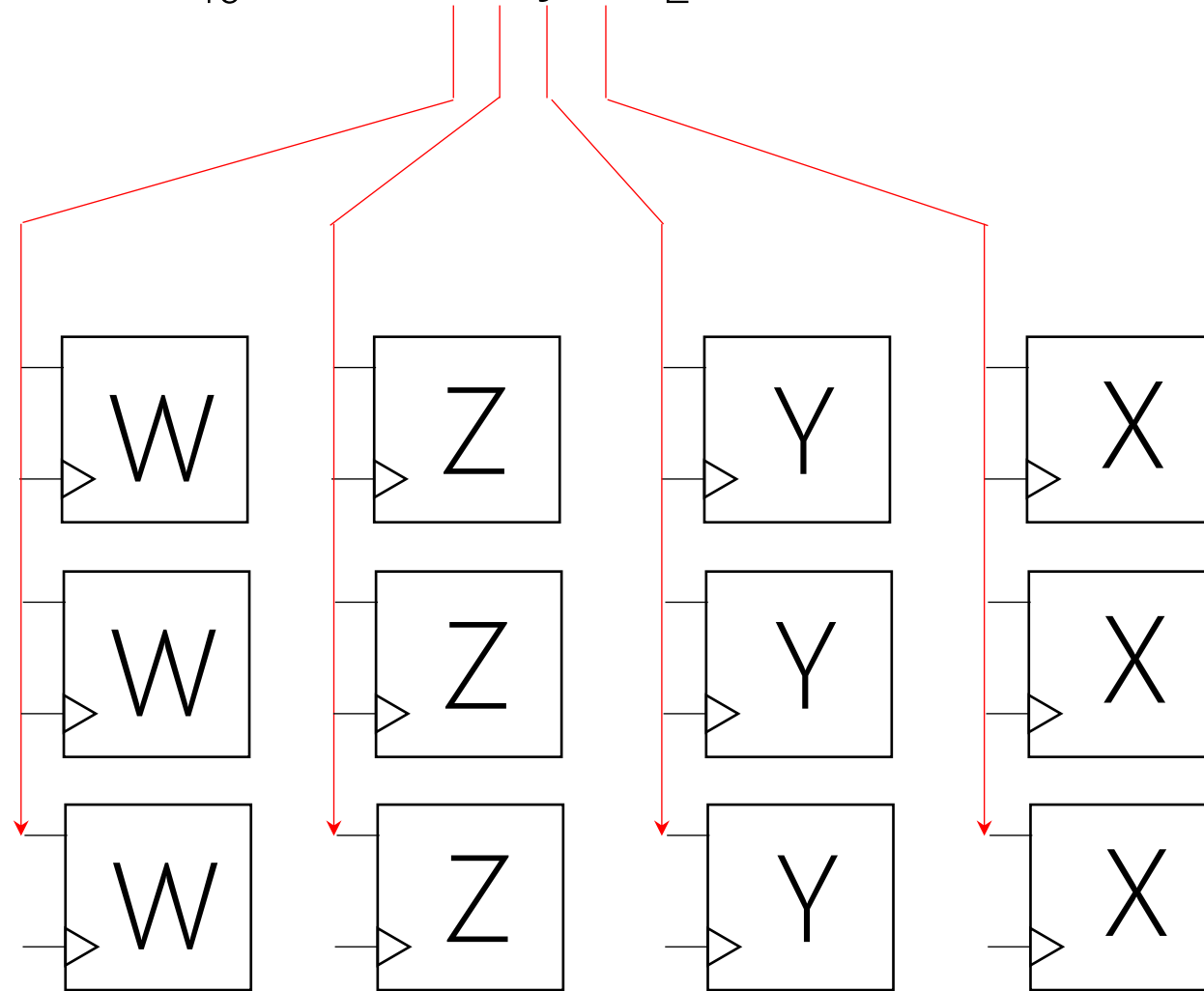
$1 \times \text{Number} \rightarrow 1 \times n \text{ Cells} \rightarrow 1 \times n\text{-bit Register}$
 $\text{Register} \rightarrow \text{a vector of Cells}$

$$(N)_{10} = (wzyx)_2$$



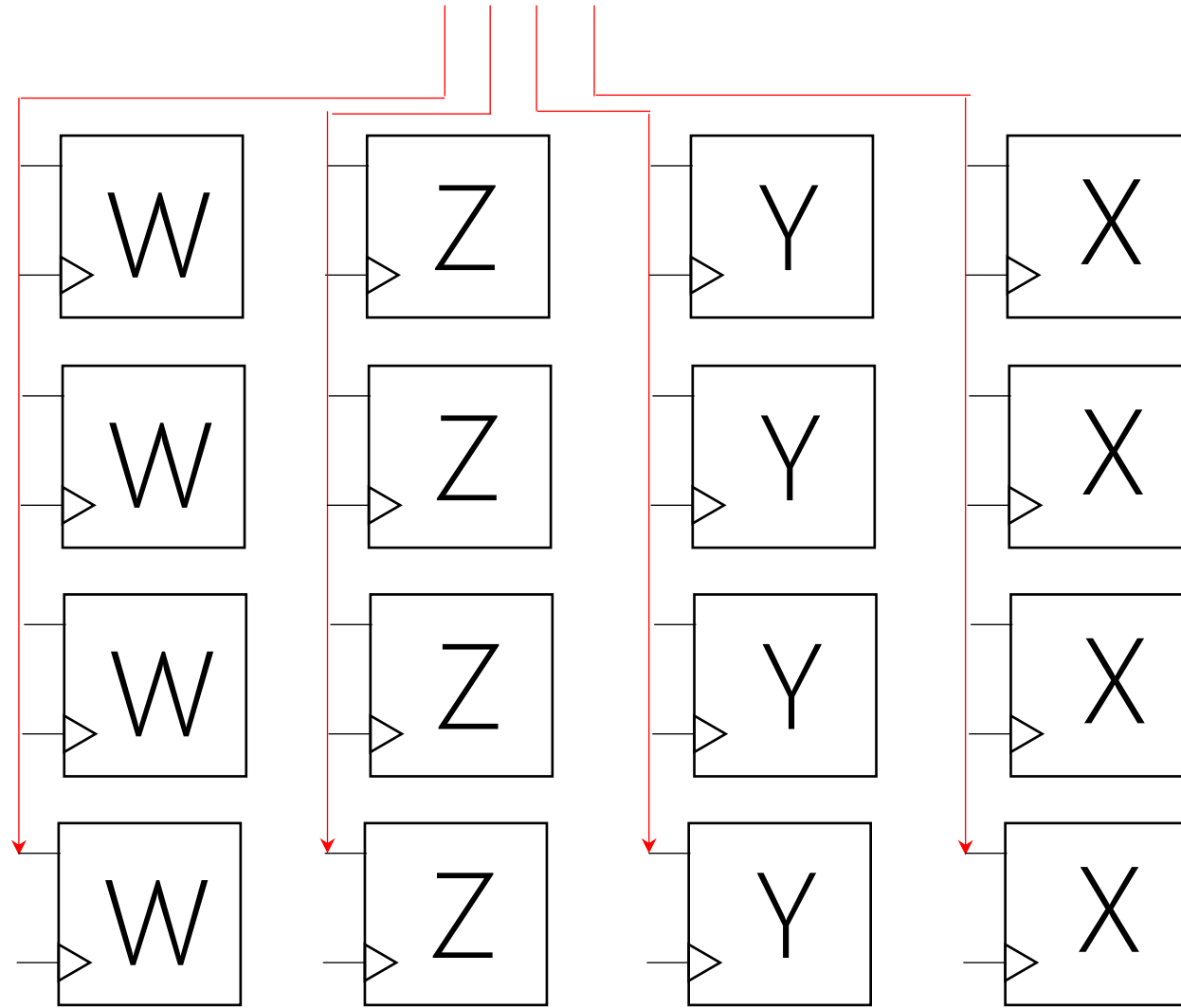
2 × Numbers → 2 × n Cells → 2 × n-bit Registers

$$(N)_{10} = (wzyx)_2$$



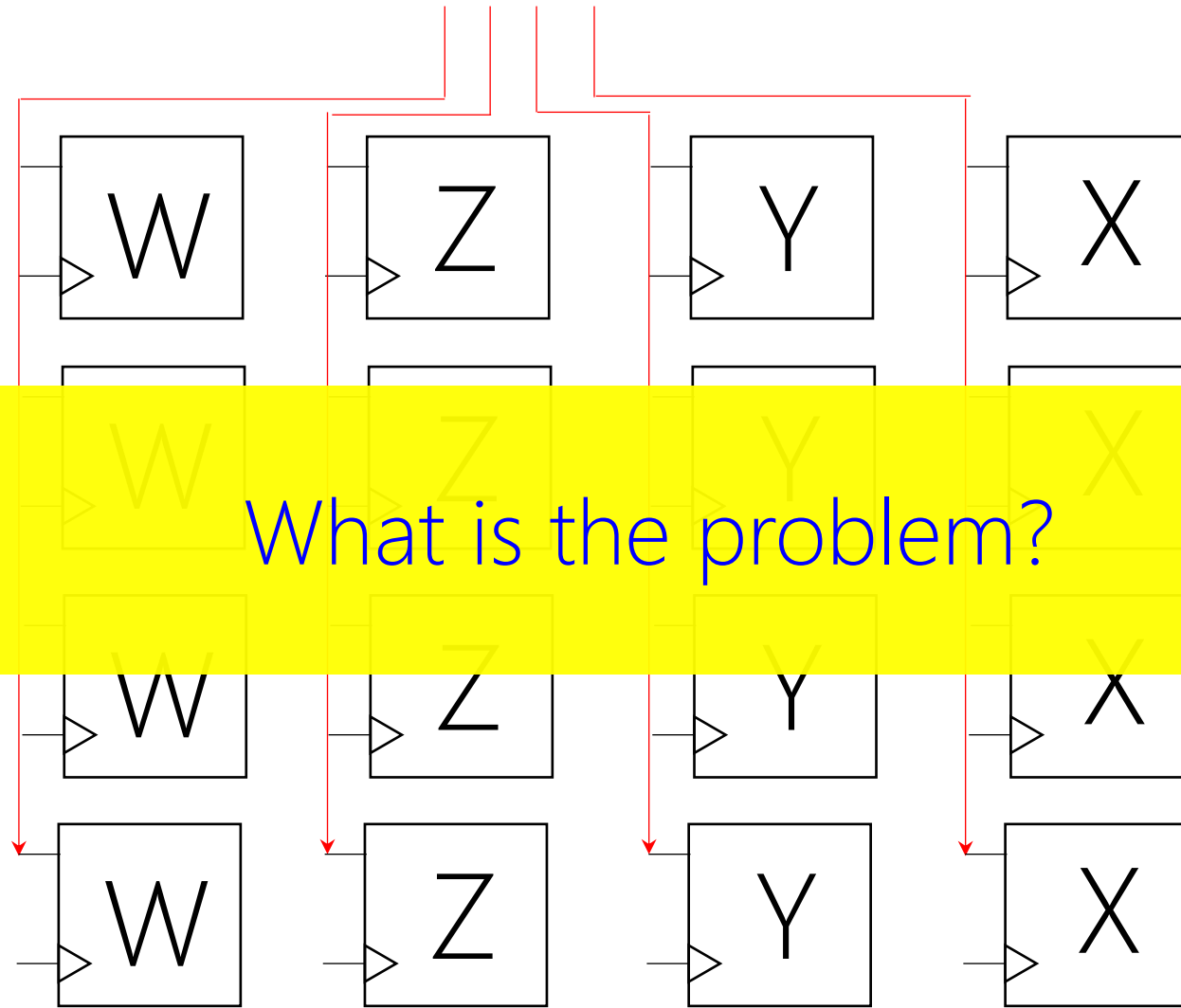
3 × Numbers → 3 × n Cells → 3 × n-bit Registers

4-bit Data Bus



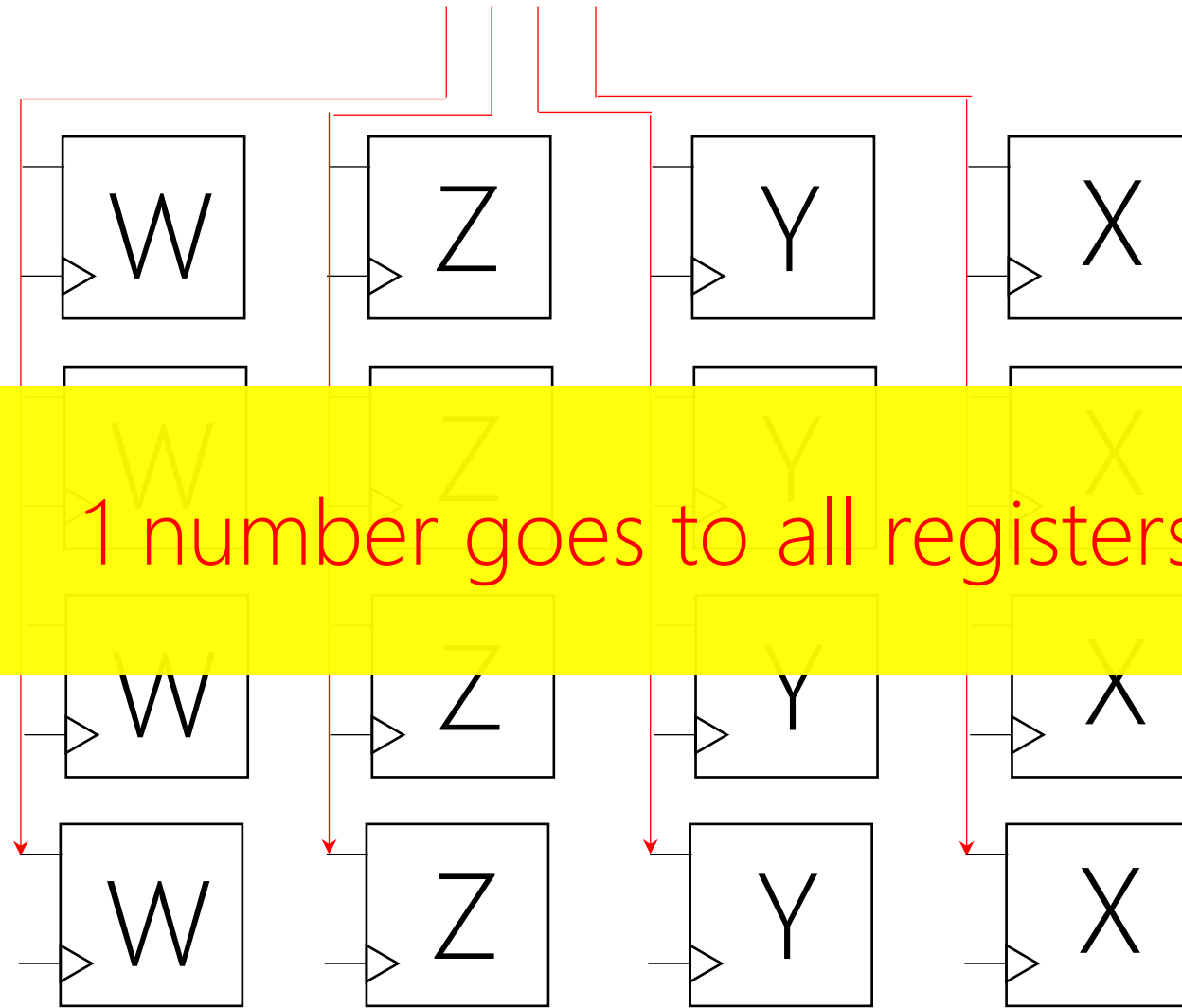
4 × Numbers → 4 × n Cells → 4 × n-bit Registers

4-bit Data Bus



4 × Numbers → 4 × n Cells → 4 × n-bit Registers

4-bit Data Bus



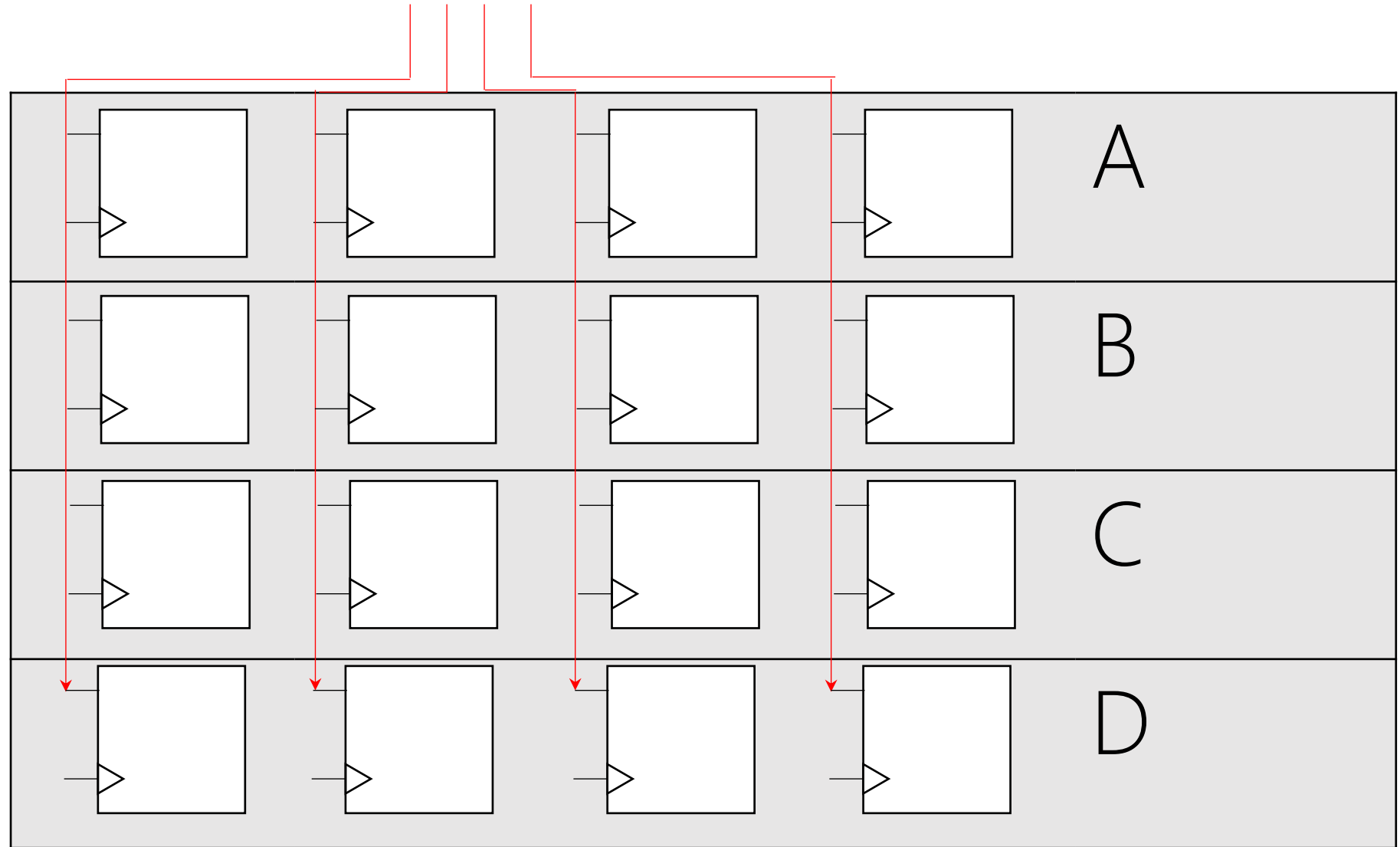
$4 \times \text{Numbers} \rightarrow 4 \times n \text{ Cells} \rightarrow 4 \times n\text{-bit Registers}$

Which Register?

Select Register by Name?

If there are few rows

4-bit Data Bus



4 × Numbers → 4 × n Cells → 4 × n-bit Registers

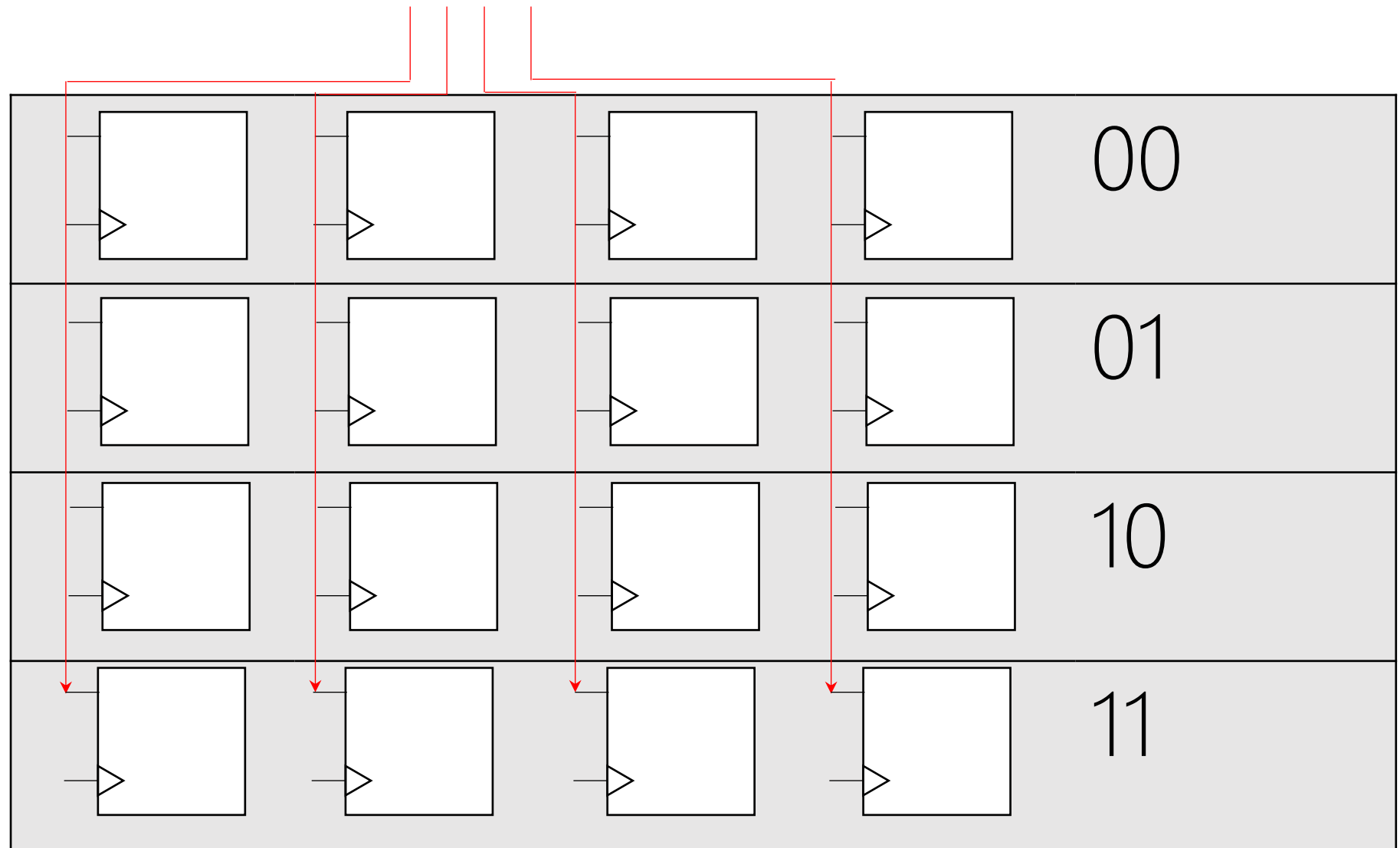
Select Register by Name?

If there are few rows

But what if we have thousands of rows?

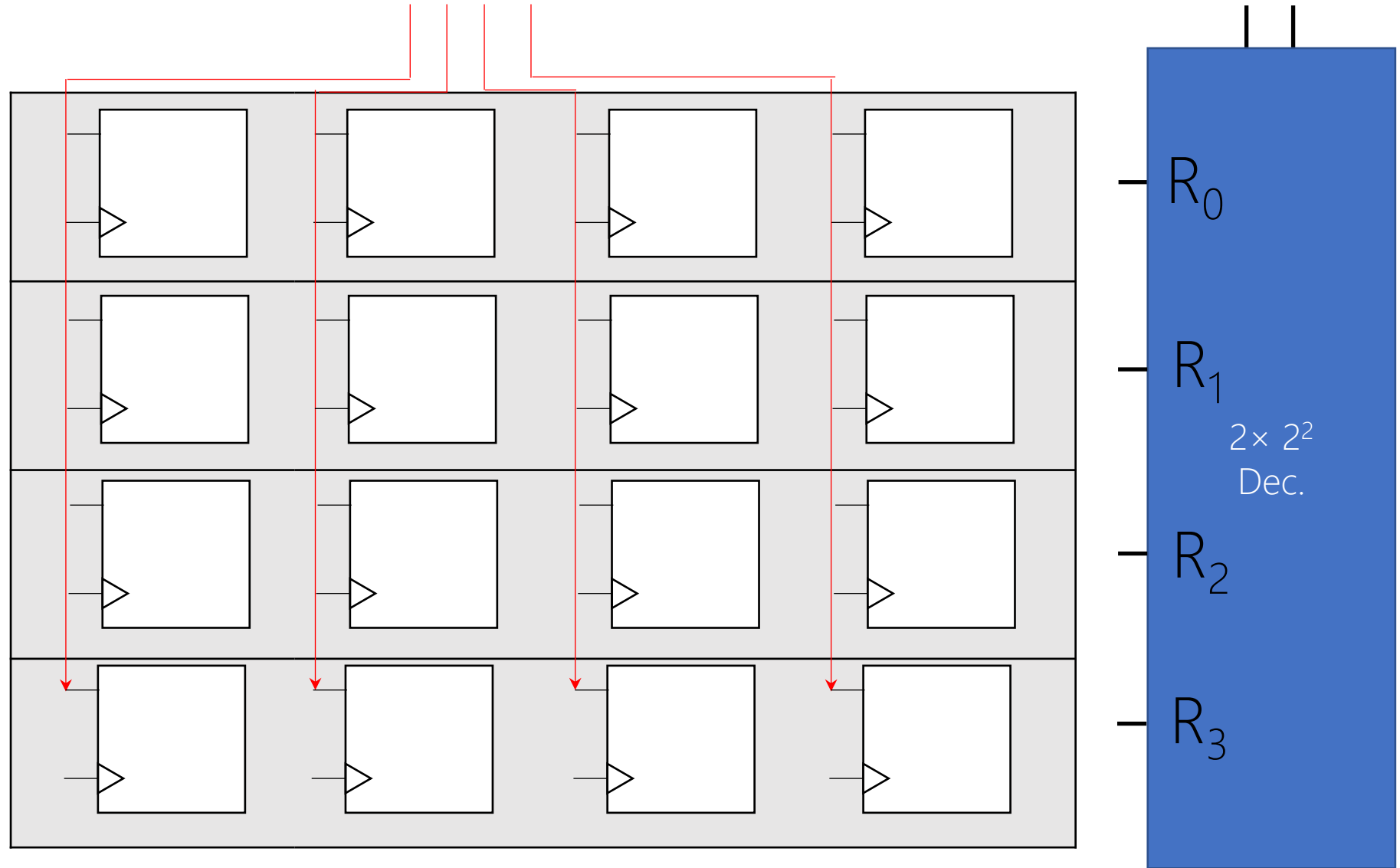
Select Register by Address?

4-bit Data Bus



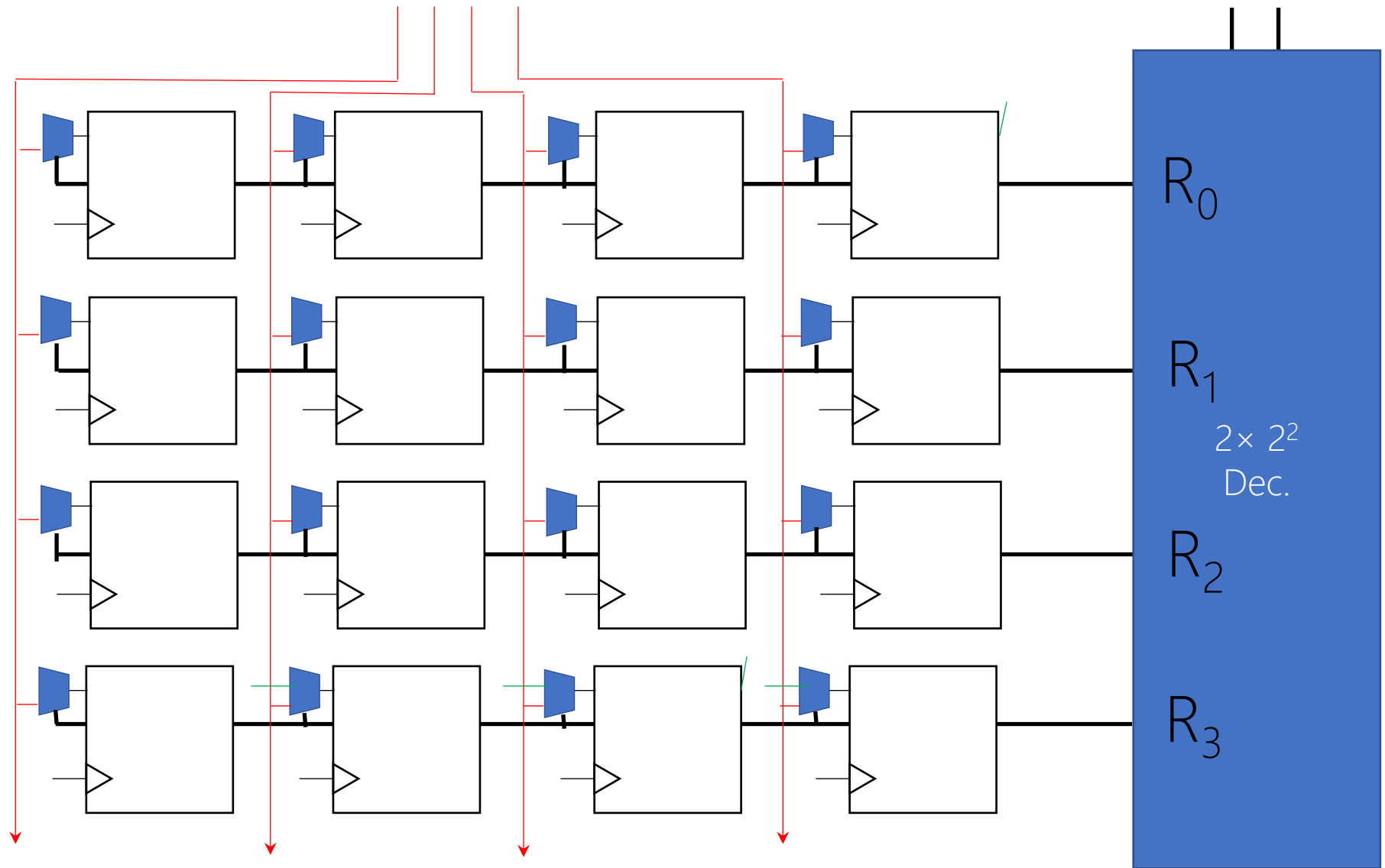
4-bit Data Bus

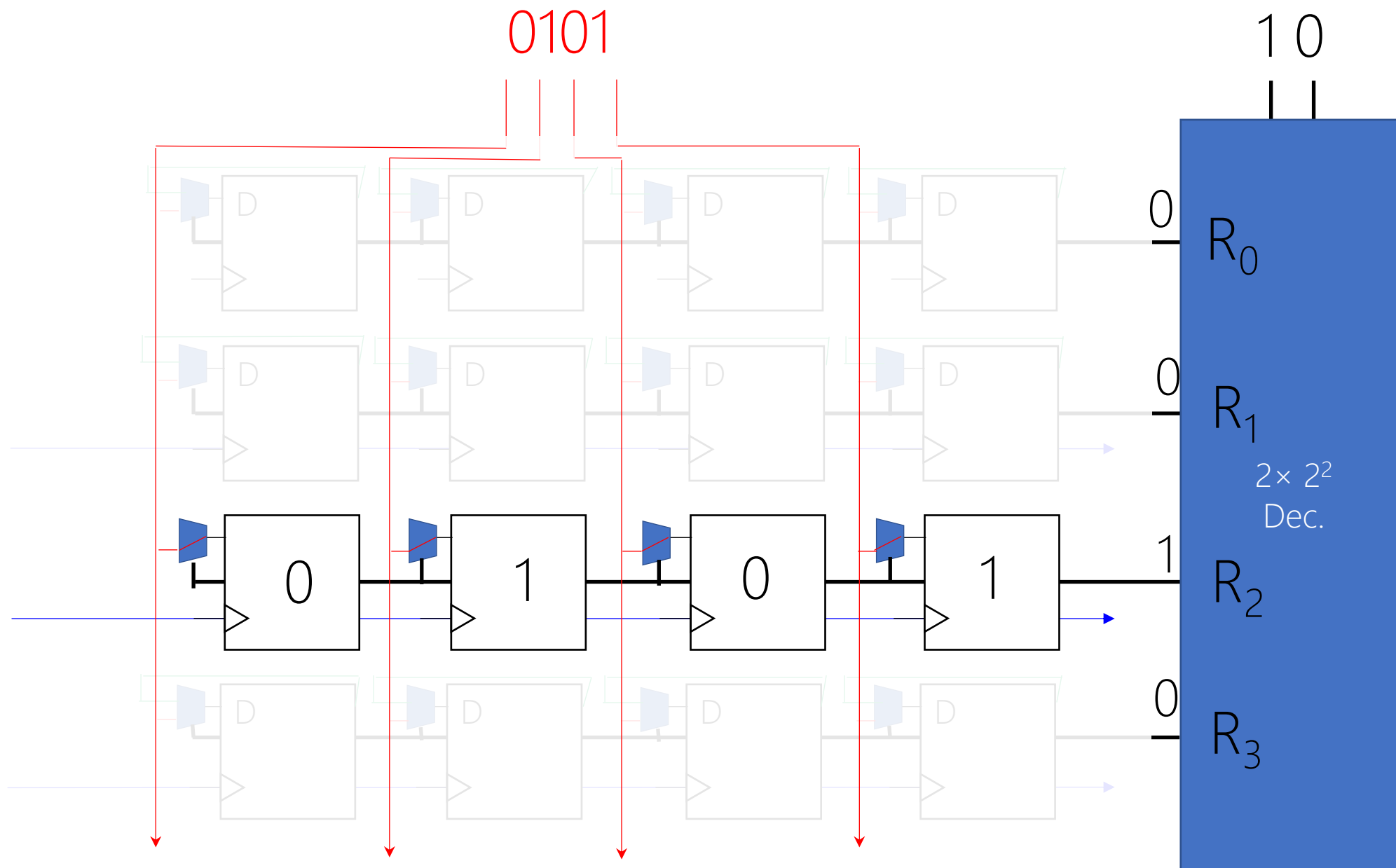
2-bit Address Bus

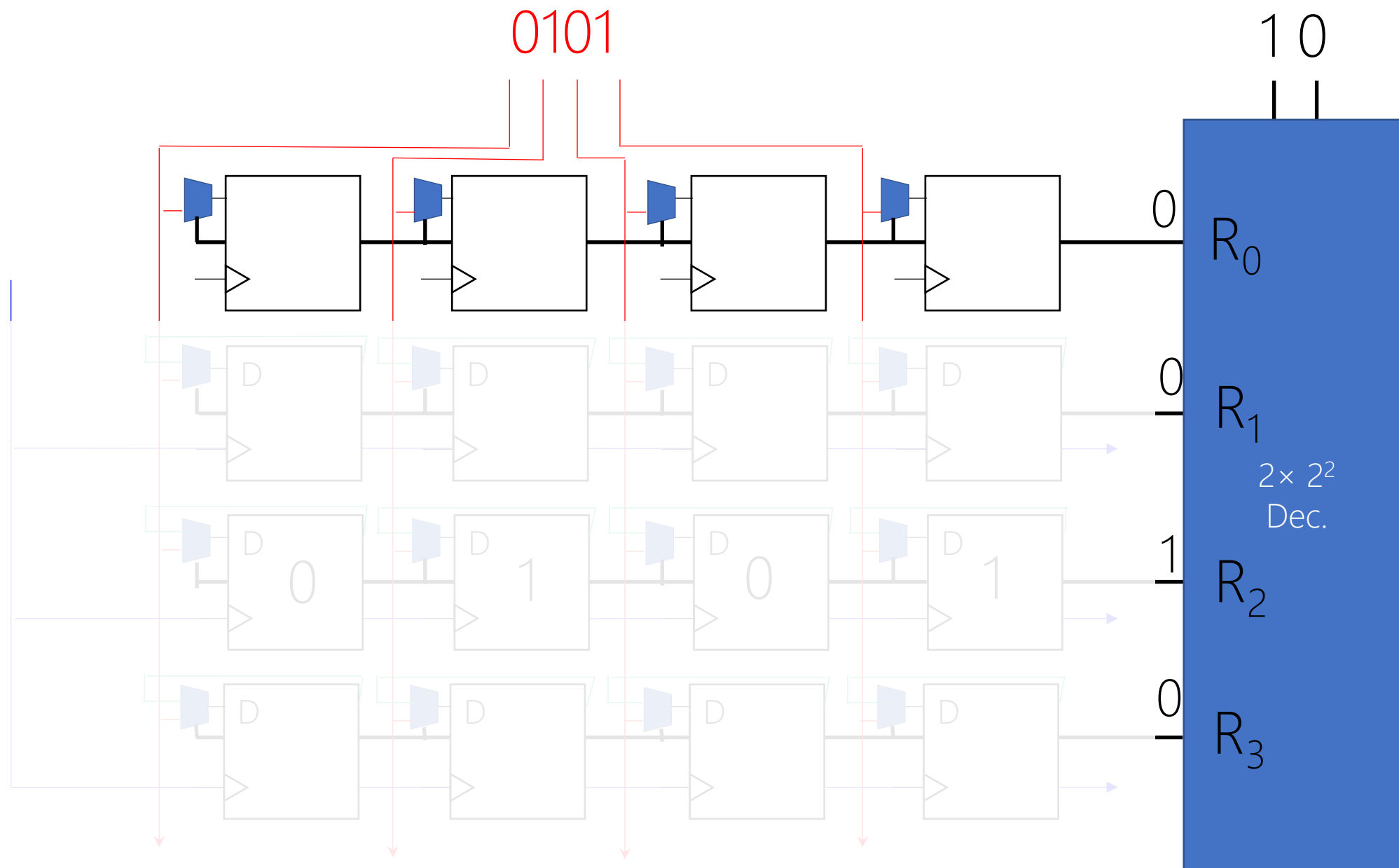


4-bit Data Bus

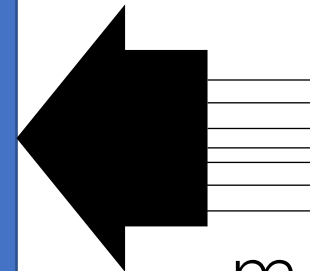
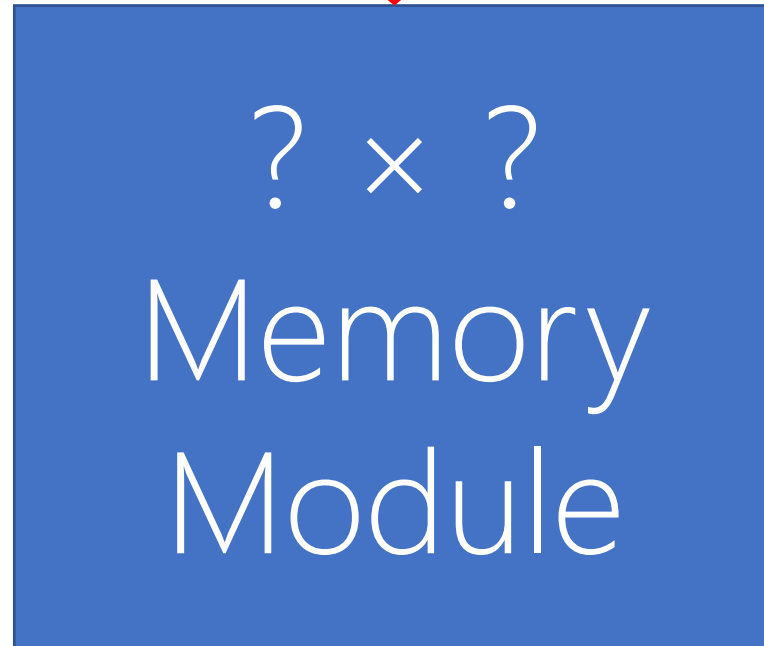
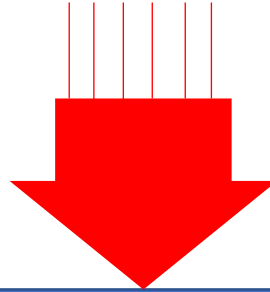
2-bit Address Bus





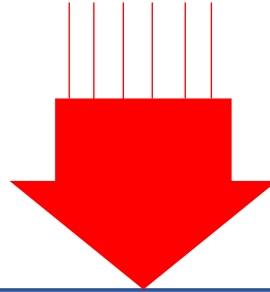


n-bit Data Bus

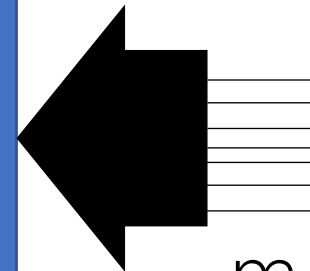


m-bit Address Bus

n-bit Data Bus



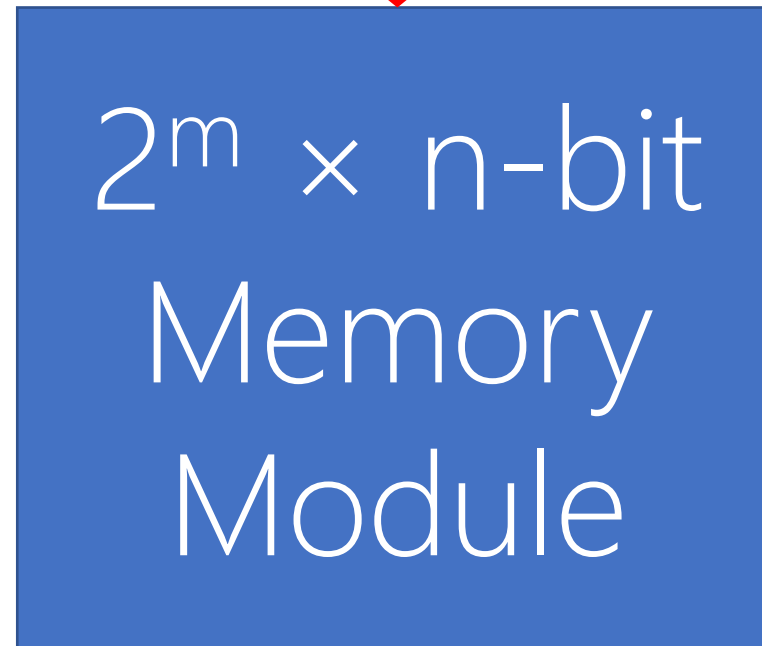
$2^m \times n\text{-bit}$
Memory
Module



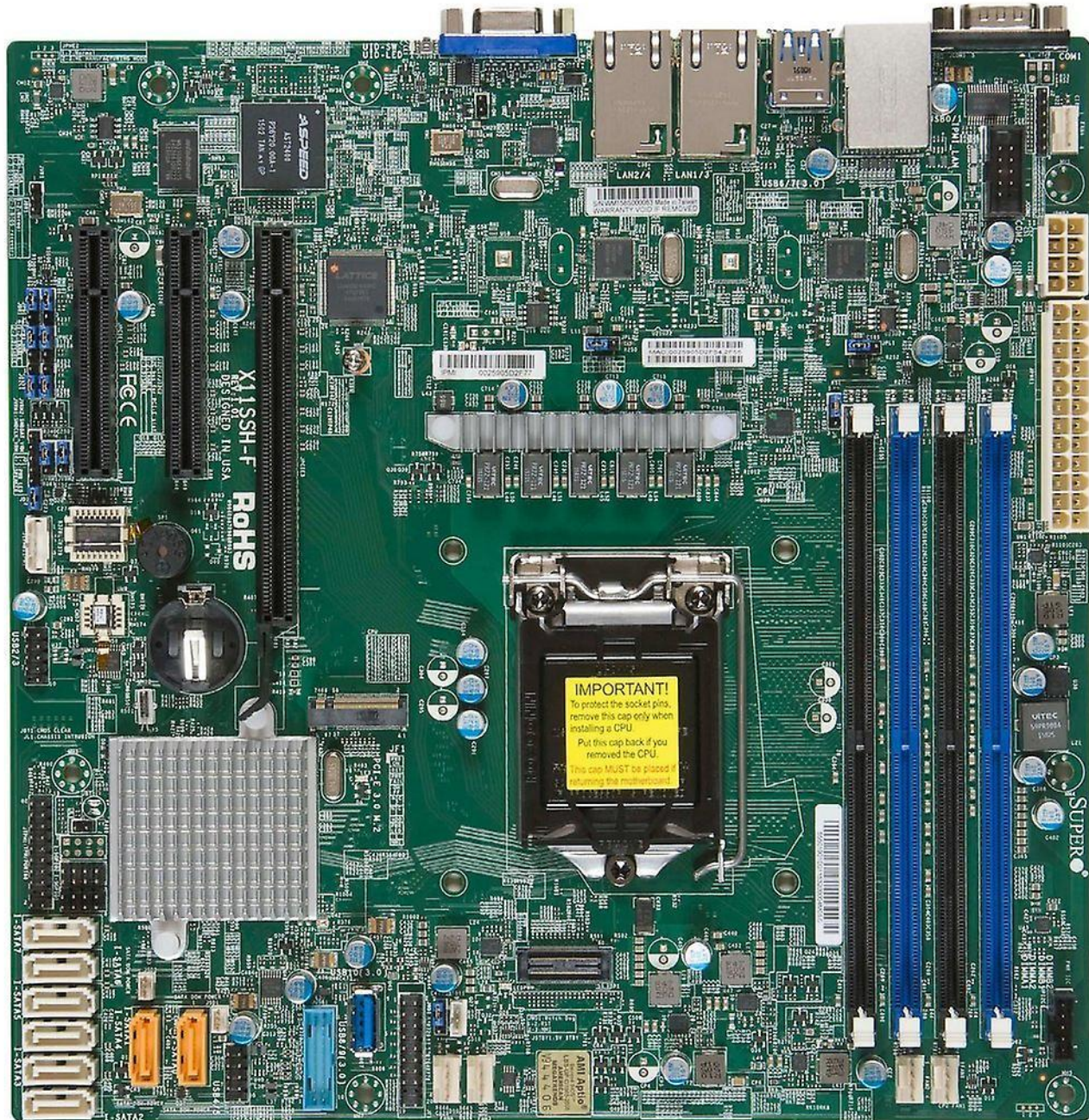
m-bit Address Bus

n-bit Data Bus

Size of bus is fixed!



m-bit Address Bus



64-bit Data Bus:

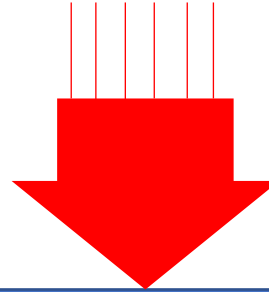
Numbers $\in [0, 2^{64})$

Long Unsigned Integer in C/C++
How about signed integers?

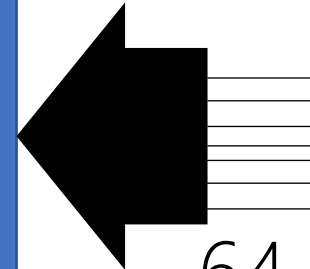
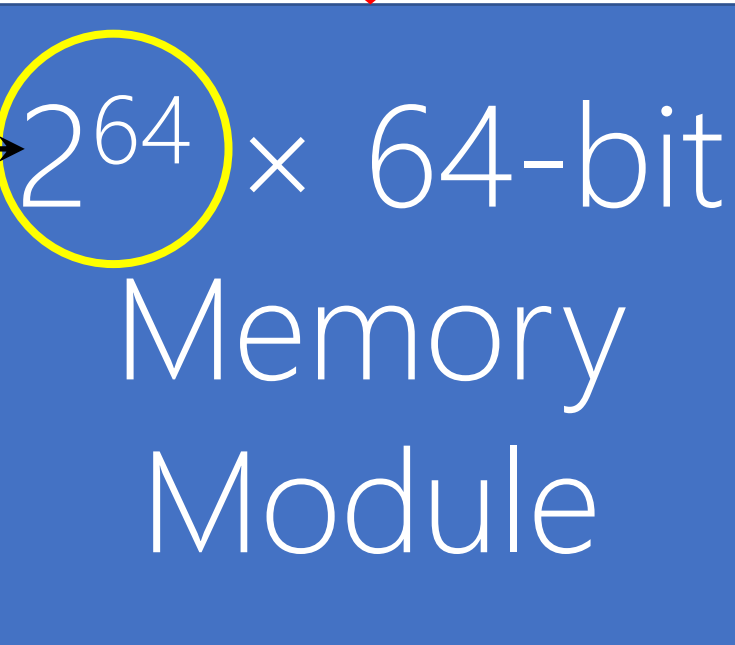
64-bit Address Bus:

2^{64} Memory Locations

64-bit Data Bus



Buying 2^{64} !
You must be rich.
You don't need it



64-bit Address Bus

$2^{64} = 2^{10} \times 2^{10} \times 2^{10} \times 2^{10} \times 2^{10} \times 2^{10} \times 2^4 = 16$ Exa Memory Locations

Kilo → Mega → Giga → Peta → Exa

Brother Genuine Supplies. Worth it every time.

Shop Now

brother

Electronics > Computers & Accessories > Computer Components > Memory



Crucial RAM 16GB DDR4 2400 MHz CL17 Desktop Memory CT16G4DFD824A

Visit the Crucial Store



9,324 ratings | 46 answered questions

Price: ~~CDN\$~~ 81.01 & FREE Shipping

Get a \$5 promotional credit on purchase of \$25 or more in Amazon.ca Gift Cards. Restrictions Apply.

$16\text{ G} = 16 \times 2^{10} \text{ (M)} = 16 \times 2^{10} \times 2^{10} \text{ (K)} = 16 \times 2^{10} \times 2^{10} \times 2^{10} \text{ Memory Locations}$



16GB Dual Rank

Style: 2400 MT/s

2666 MT/s

2400 MT/s

3200 MT/s

Brother Genuine Supplies. Worth it every time.

Shop Now

brother

Electronics > Computers & Accessories > Computer Components > Memory



Crucial RAM 16GB DDR4 2400 MHz CL17 Desktop Memory CT16G4DFD824A

Visit the Crucial Store



9,324 ratings | 46 answered questions

$16\text{ G} = 16 \times 2^{10} \text{ (M)} = 16 \times 2^{10} \times 2^{10} \text{ (K)} = 16 \times 2^{10} \times 2^{10} \times 2^{10}$ Memory Locations
How many Cells?



New (15) from **CDN\$ 81.00** + FREE Shipping

Size :

16GB Dual Rank

Style: 2400 MT/s

2666 MT/s

2400 MT/s

3200 MT/s

Brother Genuine Supplies. Worth it every time.

Shop Now

brother

Electronics > Computers & Accessories > Computer Components > Memory



Crucial RAM 16GB DDR4 2400 MHz CL17 Desktop Memory

$16\text{ G} = 16 \times 2^{10} \text{ (M)} = 16 \times 2^{10} \times 2^{10} \text{ (K)} = 16 \times 2^{10} \times 2^{10} \times 2^{10} \text{ Memory Locations}$
How many Cells?

$$(2^4=16) \times 2^{10} \times 2^{10} \times 2^{10} \times (2^6=64\text{-bit Data Bus}) = 2^{40}$$



New (15) from **CDN\$ 81.00** + FREE Shipping

Size :

16GB Dual Rank

Style: 2400 MT/s

2666 MT/s

2400 MT/s

3200 MT/s

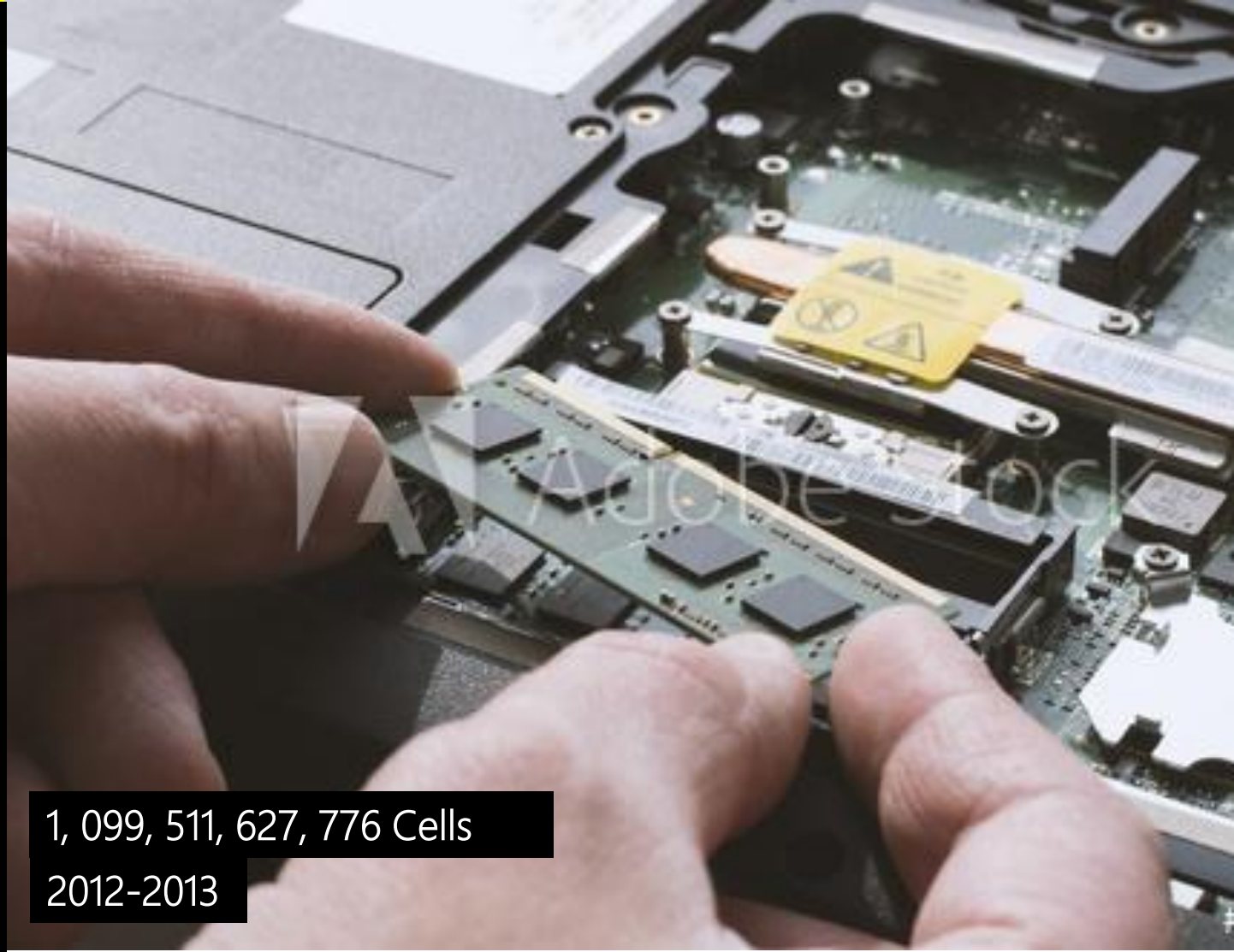
1, 099, 511, 627, 776 Cells

$16\text{ G} = 16 \times 2^{10} \text{ (M)} = 16 \times 2^{10} \times 2^{10} \text{ (K)} = 16 \times 2^{10} \times 2^{10} \times 2^{10} \text{ Memory Locations}$
How many Cells?

$$(2^4=16) \times 2^{10} \times 2^{10} \times 2^{10} \times (2^6=64\text{-bit Data Bus}) = 2^{40}$$

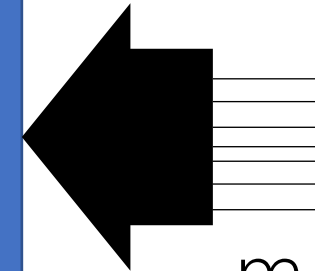
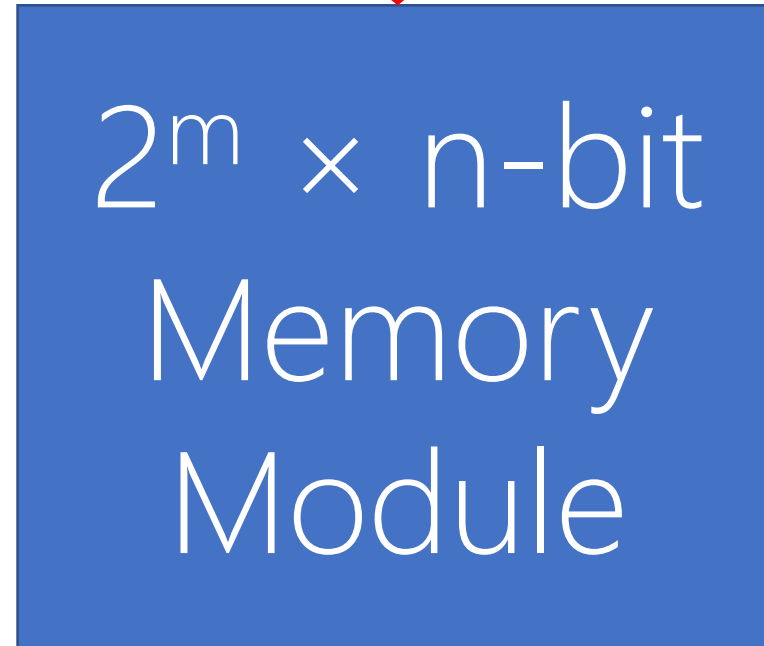
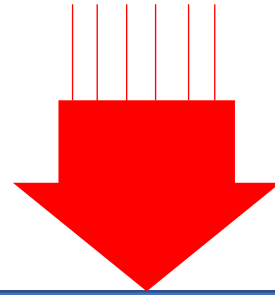


1 Cell
1958–1959

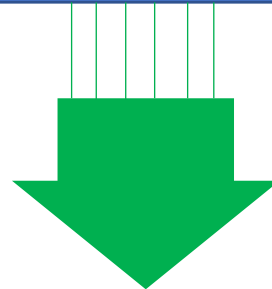


1, 099, 511, 627, 776 Cells
2012–2013

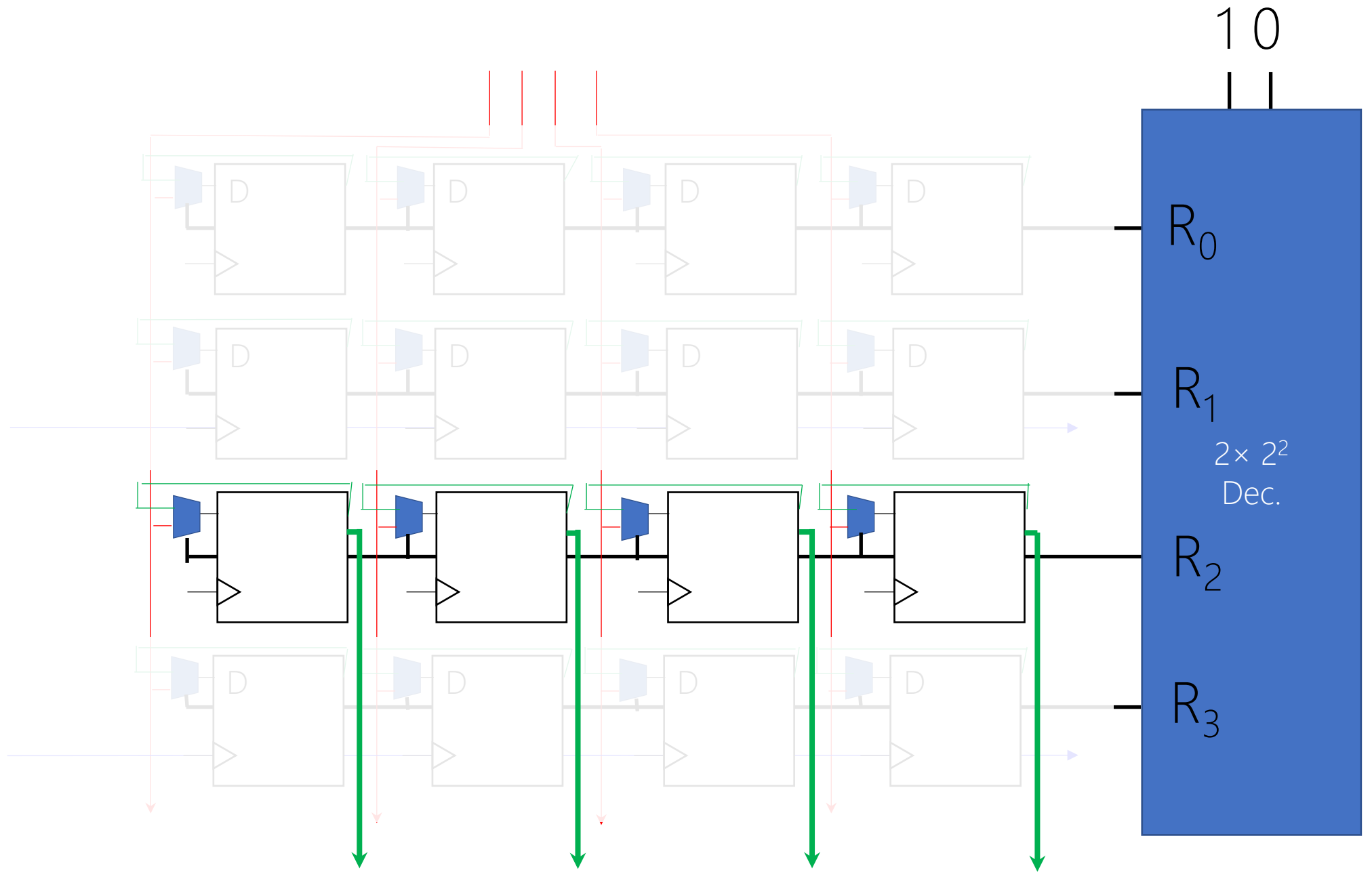
n-bit Data Bus (Write)

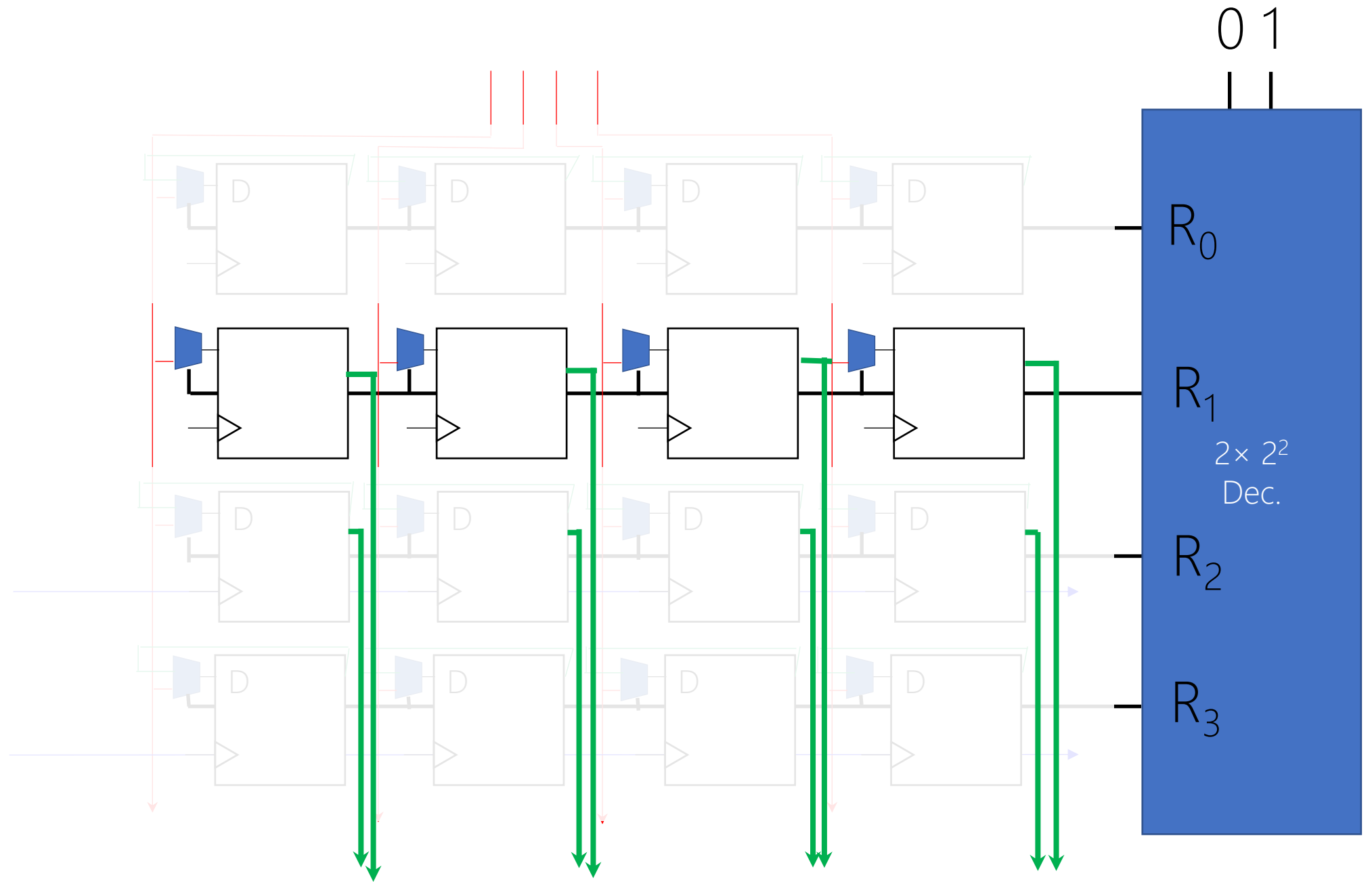


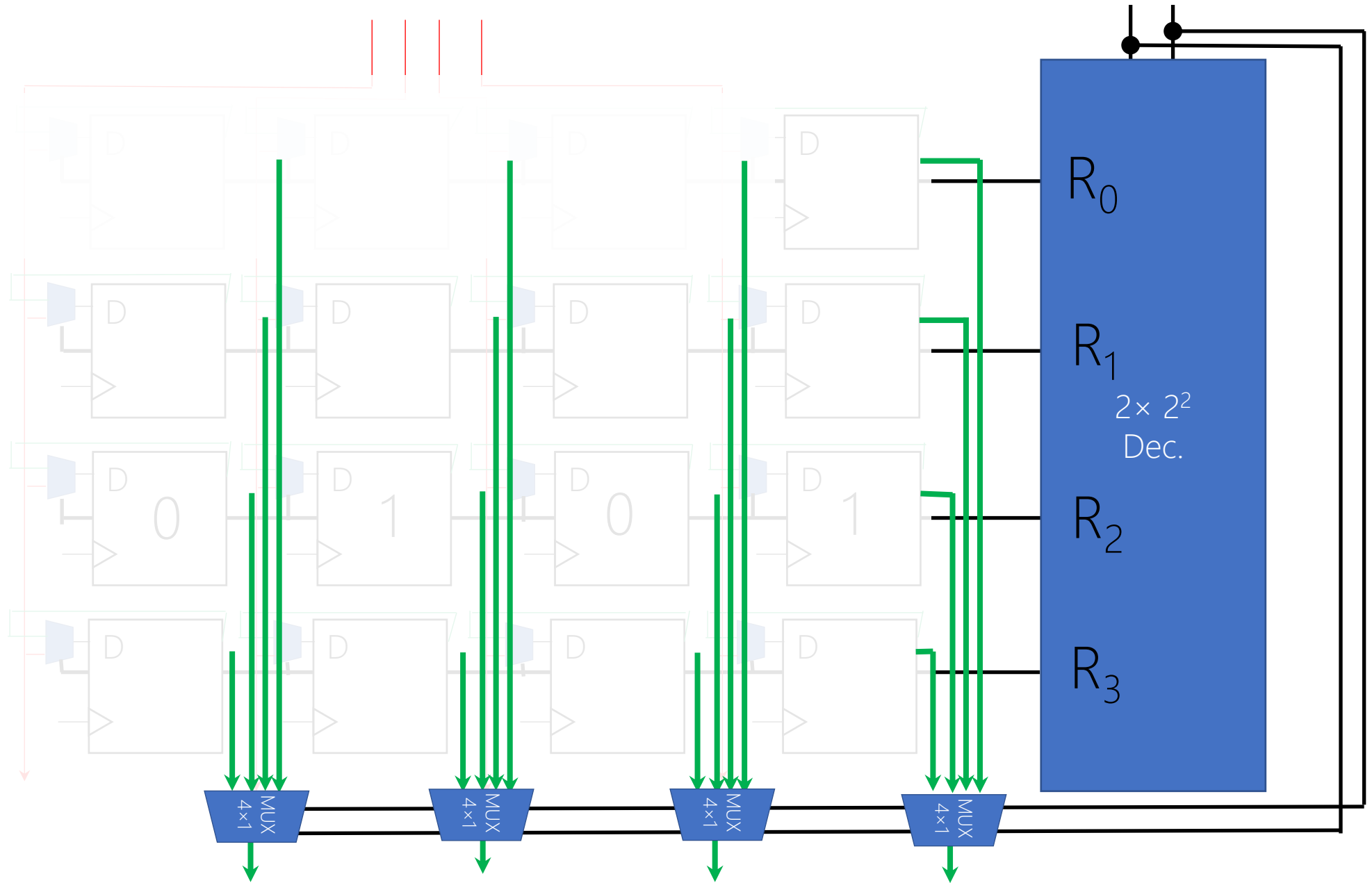
m-bit Address Bus

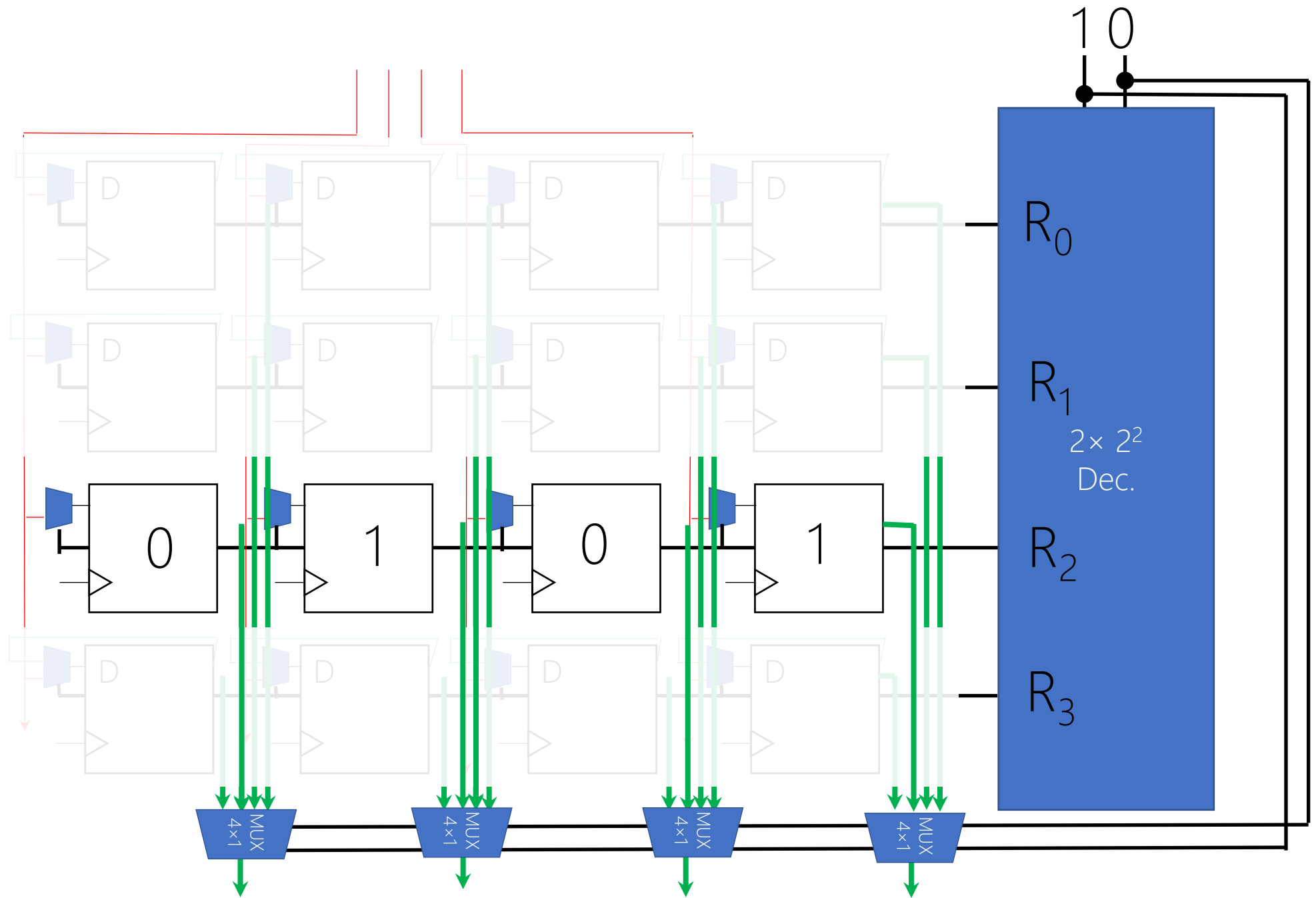


n-bit Data Bus (Read)

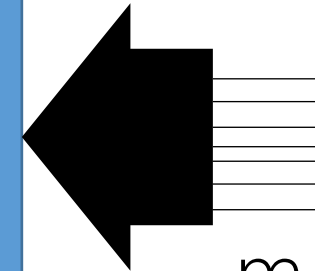
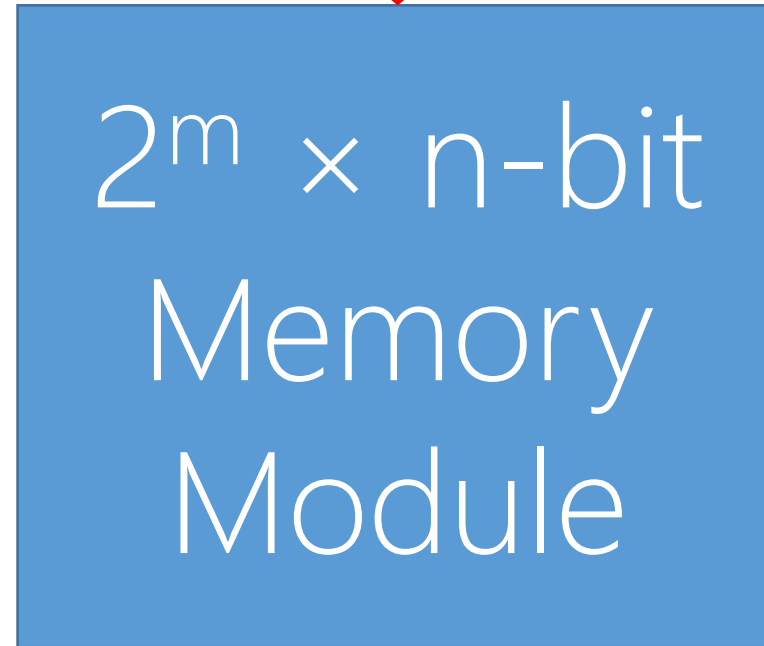
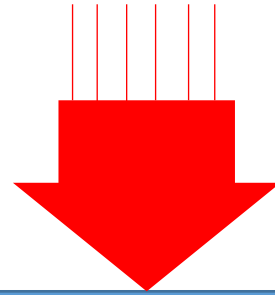




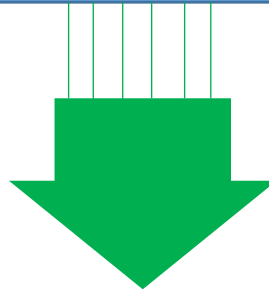




n-bit Data Bus (Write)



m-bit Address Bus

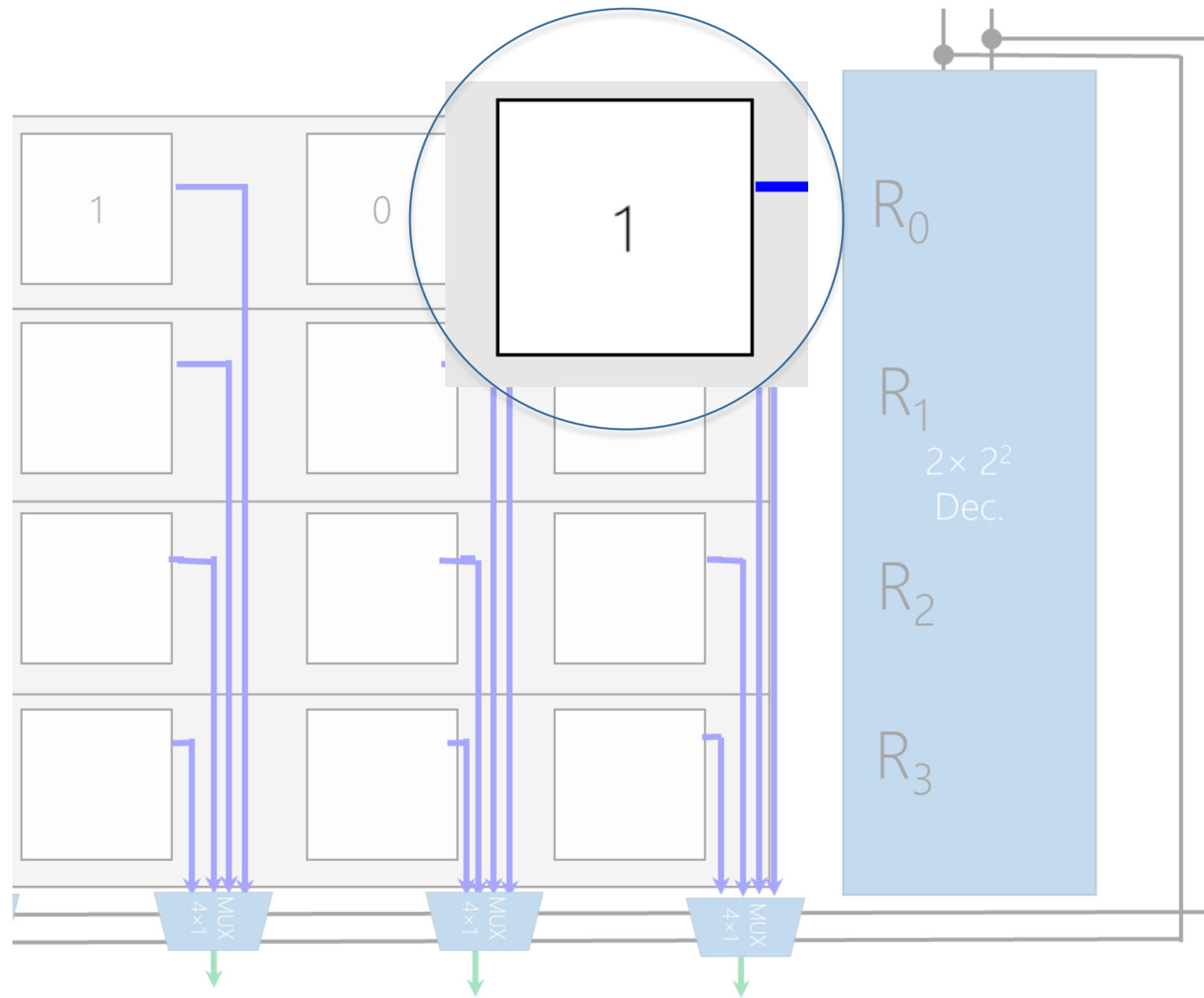


n-bit Data Bus (Read)

What is that cell?

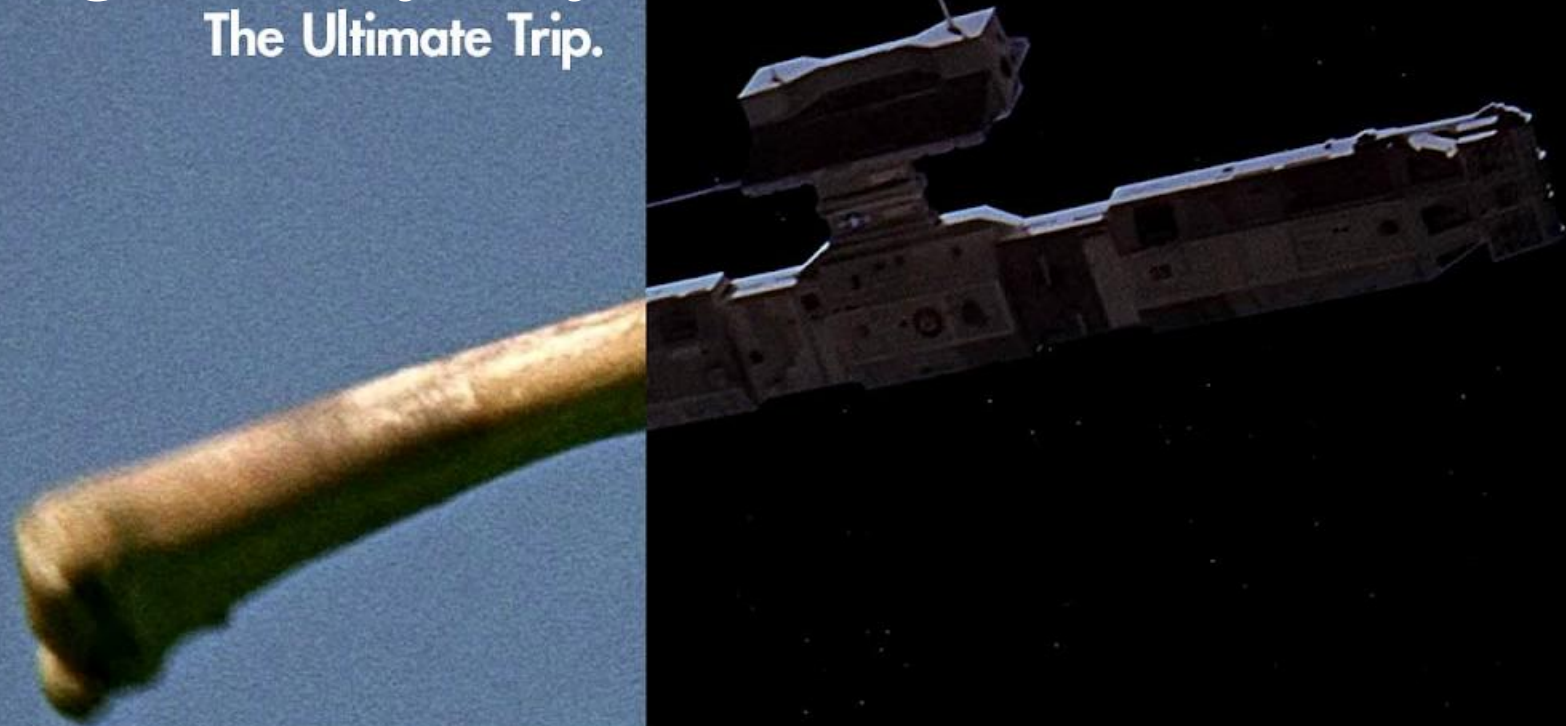


1 Cell
1958–1959

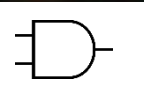


W2025: A Digital Odyssey

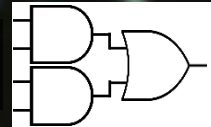
The Ultimate Trip.



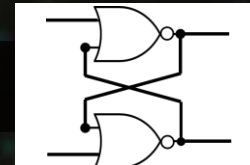
Number Systems | $(12)_{10} \rightarrow (1100)_2$
Logic Gates |



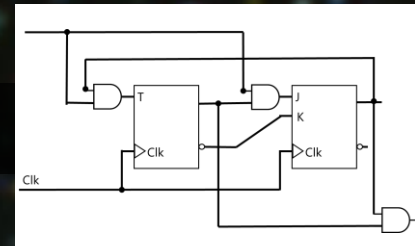
Combinational Logic |

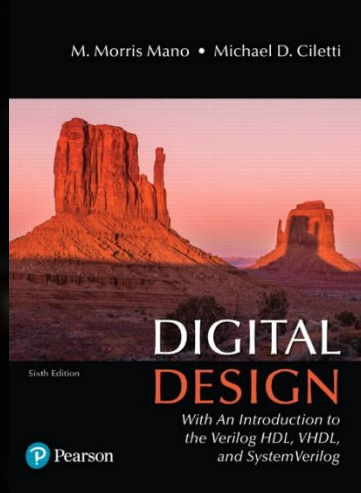


Flip-Flop |



Sequential Logic |





Chapter 5

Synchronous Sequential Logic

Memento, Christopher Nolan
Guy Pearce, Carrie-Anne Moss, Joe Pantoliano
September 2000
Budget \$4.5 m
Box office \$40 m



Sequential Logic



ACADEMY AWARD WINNER
ANTHONY HOPKINS

ACADEMY AWARD WINNER
OLIVIA COLMAN

★★★★★
"STUNNING"

"ONE OF HOPKINS'
GREAT SCREEN
PERFORMANCES"

"HOPKINS DELIVERS A
TOUR-DE-FORCE
PERFORMANCE"

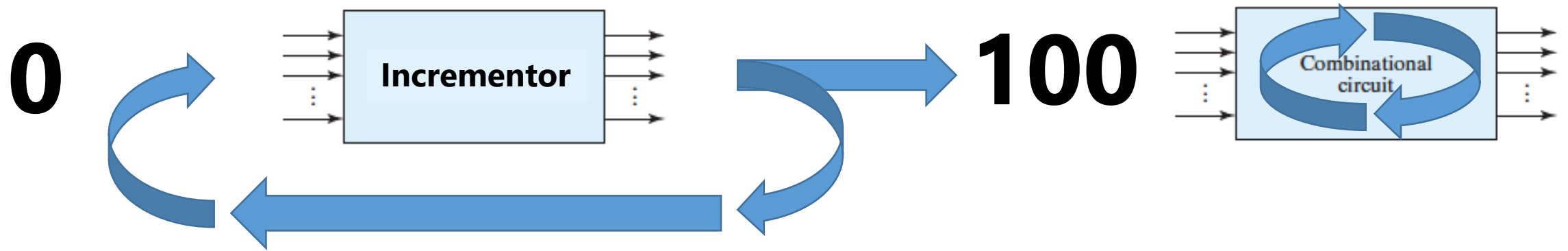
"COLMAN IS
DYNAMITE"

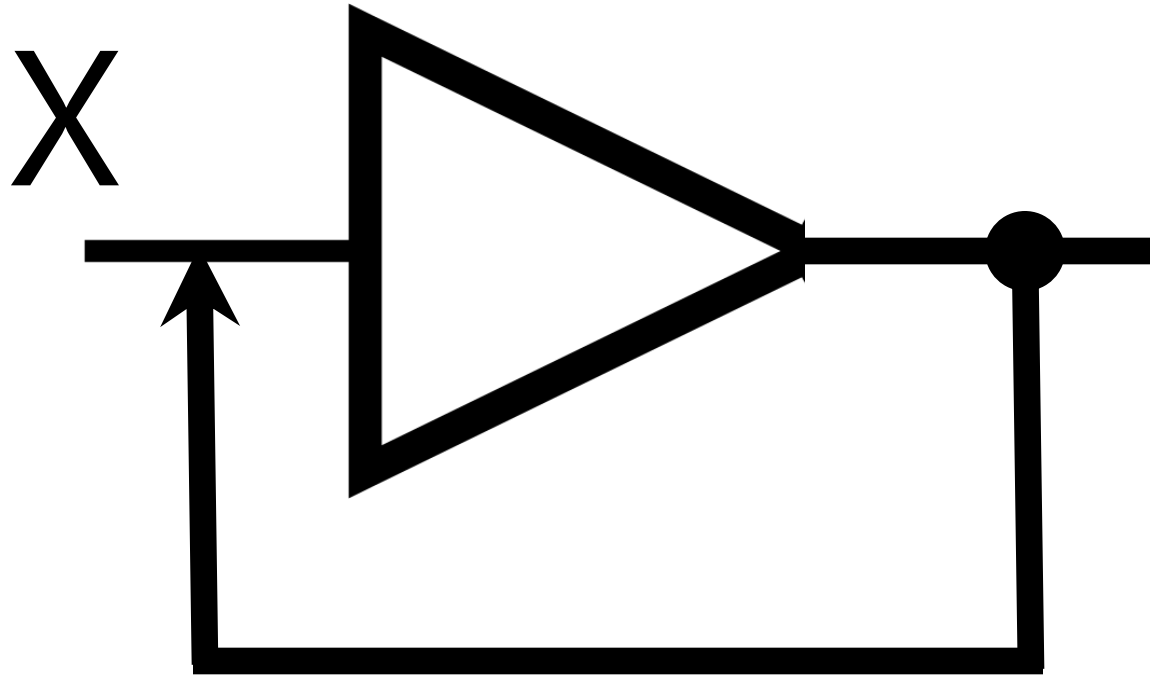
★★★★★
"BREATHTAKING"

MARK GATISS INDOOR FOOTBALL THOMAS SEWELL OLIVIA WILLIAMS

THE

Counter: $0 \rightarrow 1 \rightarrow 2 \rightarrow \dots$

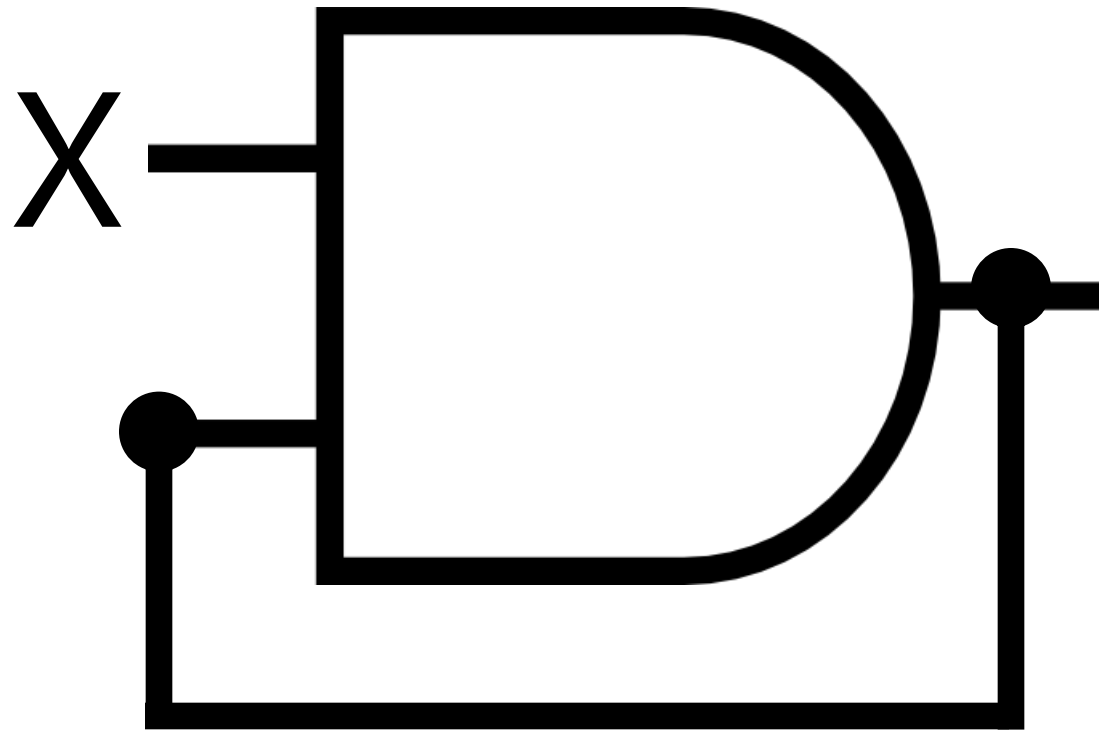




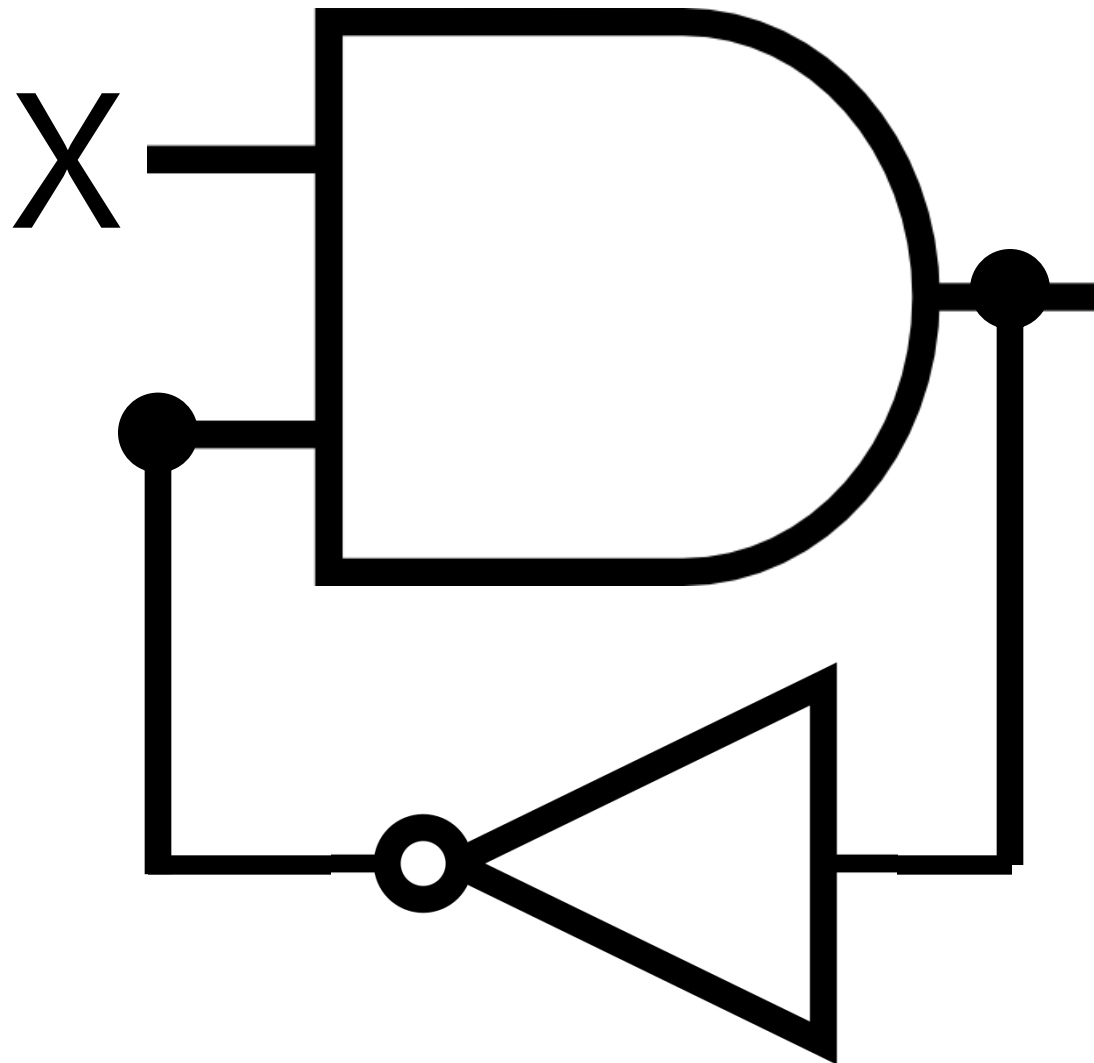
This design is logically **incorrect**! Why?

Buffer gate accepts one input.

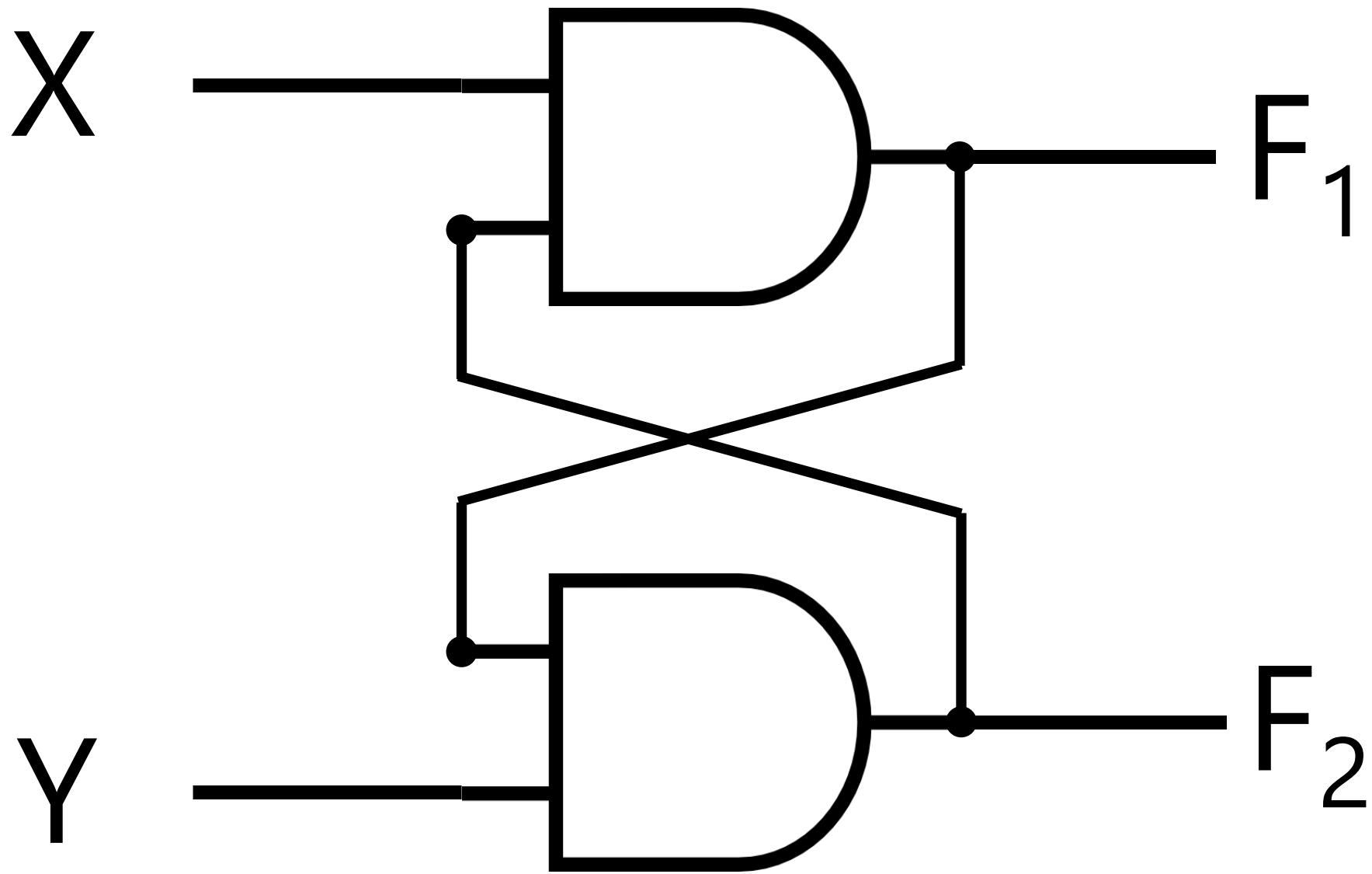
We have two inputs: X and X_{T-1}

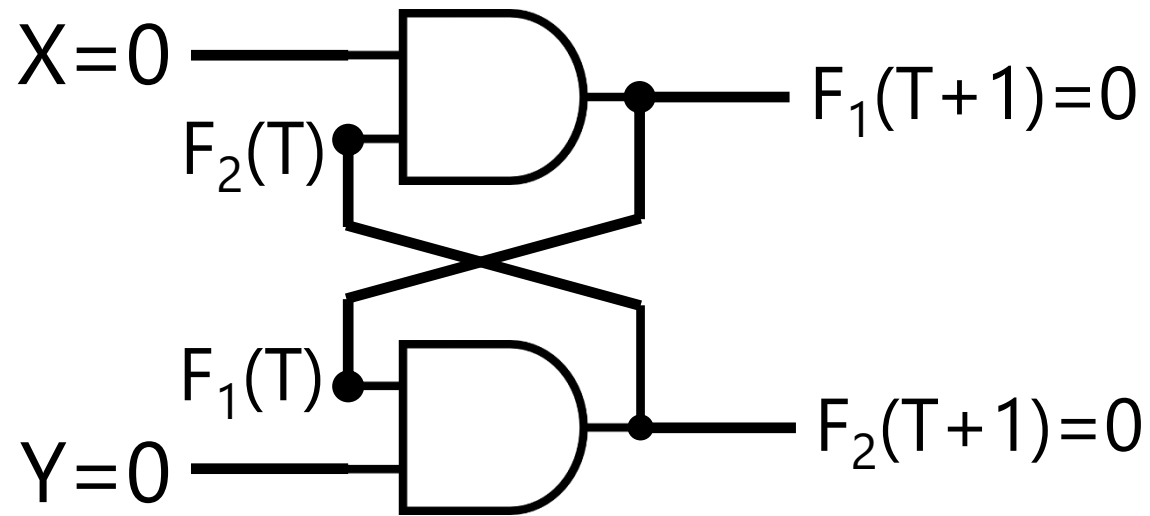


This design is logically correct!
But what's the problem?

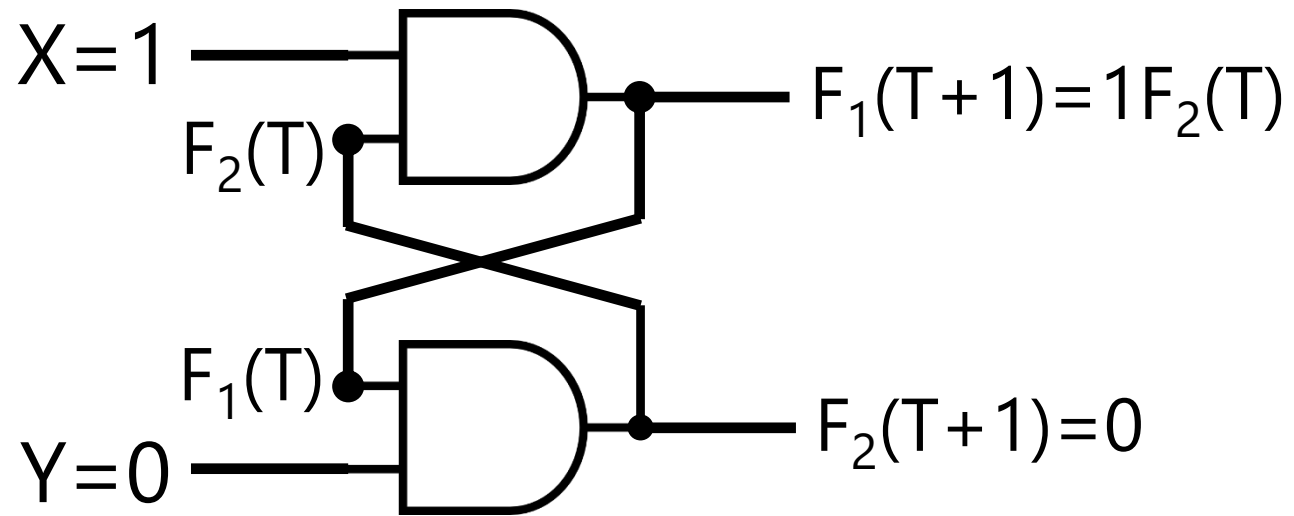


This design is also logically correct.
But what's the problem?

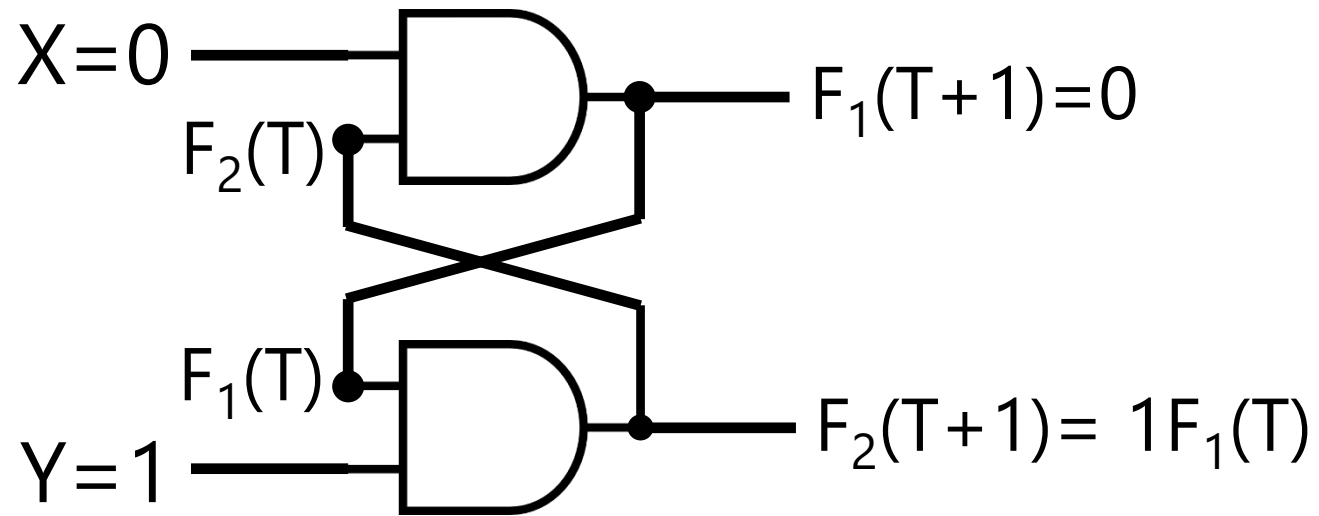




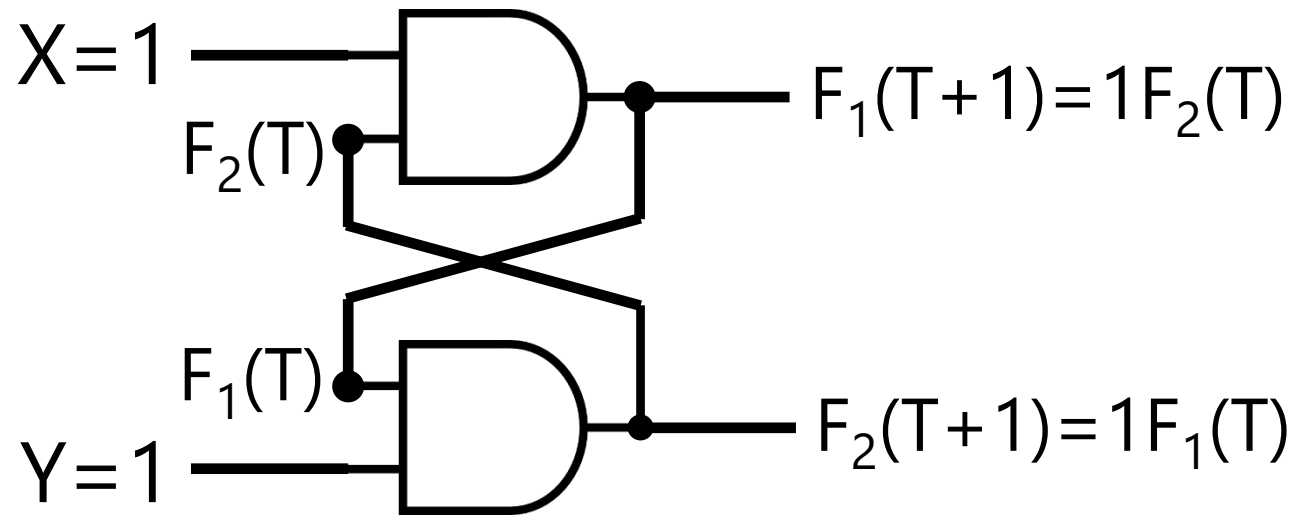
Y	X	$F_1(T+1)$	$F_2(T+1)$
0	0	0	0
0	1		
1	0		
1	1		



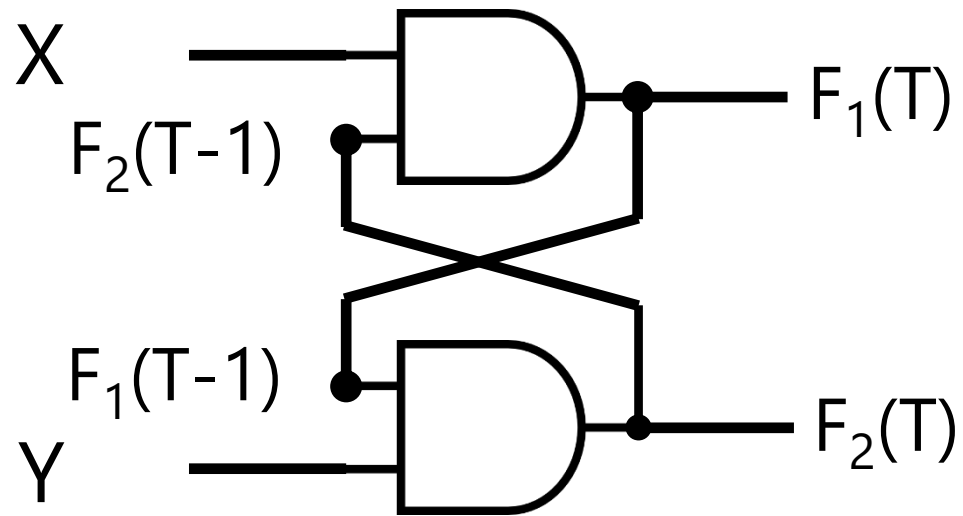
Y	X	$F_1(T+1)$	$F_2(T+1)$
0	0	0	0
0	1	$F_2(T)$	0
1	0		
1	1		



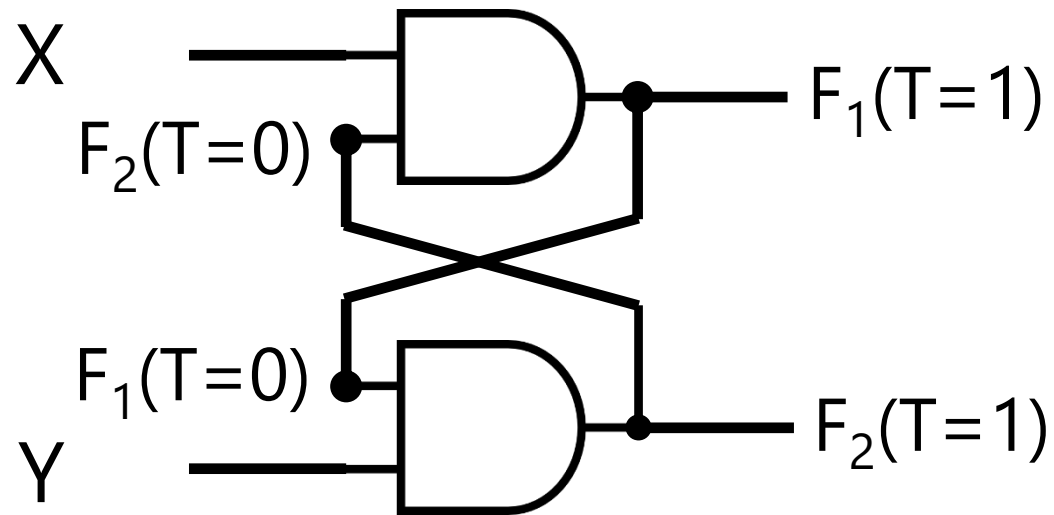
Y	X	$F_1(T+1)$	$F_2(T+1)$
0	0	0	0
0	1	$F_2(T)$	0
1	0	0	$F_1(T)$
1	1		



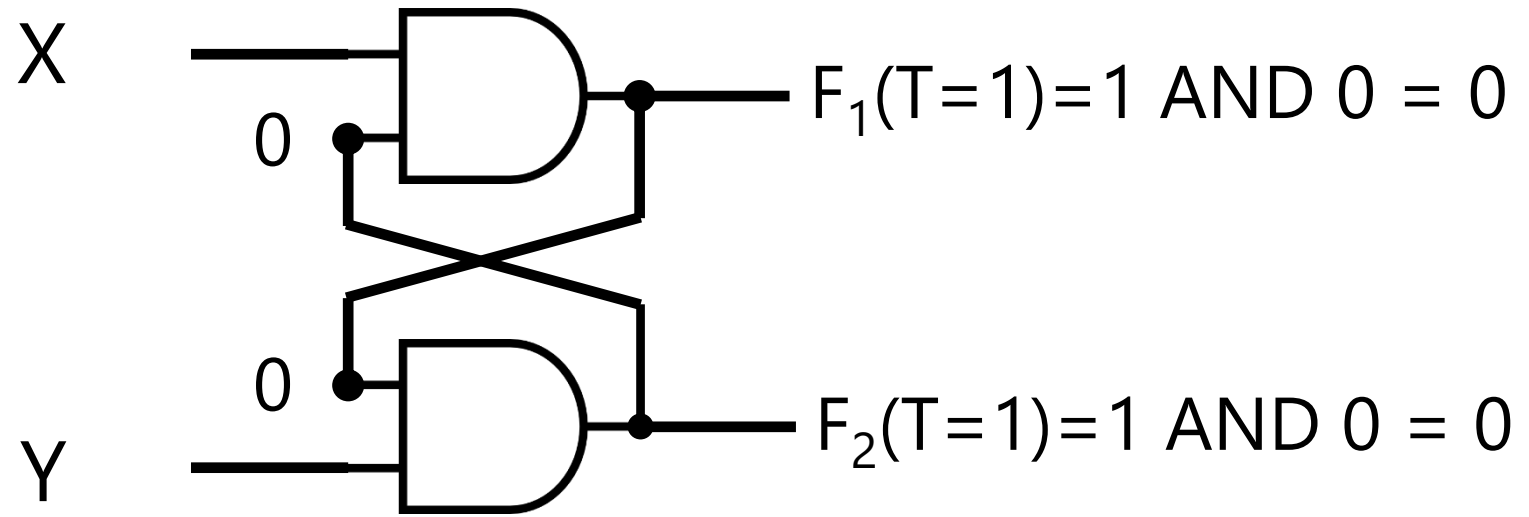
Y	X	$F_1(T+1)$	$F_2(T+1)$
0	0	0	0
0	1	$F_2(T)$	0
1	0	0	$F_1(T)$
1	1	$F_2(T)$	$F_1(T)$



Y	X	$F_1(T)$	$F_2(T)$
0	0	0	0
0	1	$F_2(T-1)$	0
1	0	0	$F_1(T-1)$
1	1	$F_2(T-1)$	$F_1(T-1)$

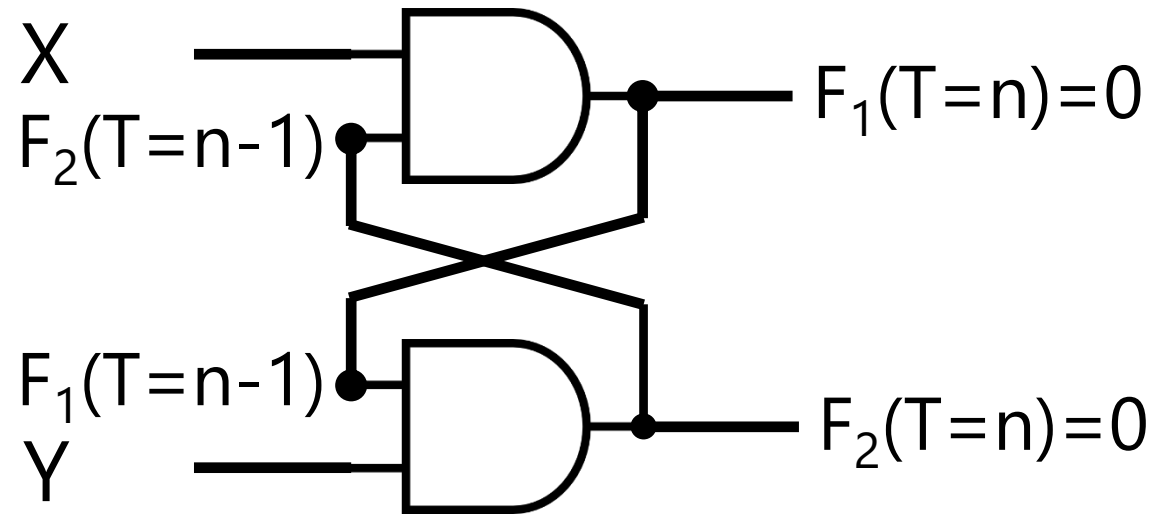


Y	X	$F_1(T=1)$	$F_2(T=1)$
0	0	0	0
0	1	$F_2(T=0)$	0
1	0	0	$F_1(T=0)$
1	1	$F_2(T=0)$	$F_1(T=0)$



by default, in positive logic $F(T=0) = 0$

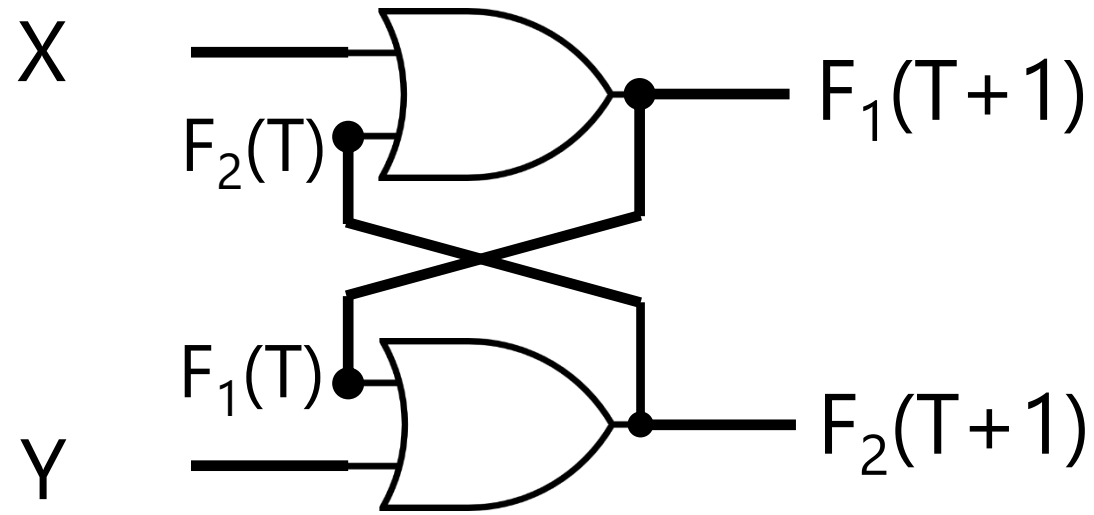
Y	X	$F_1(T=1)$	$F_2(T=1)$
0	0	0	0
0	1	0	0
1	0	0	0
1	1	0	0



When it goes to 00 state, never recover to 1!

Y	X	$F_1(T=n)$	$F_2(T=n)$
0	0	0	0
0	1	0	0
1	0	0	0
1	1	0	0

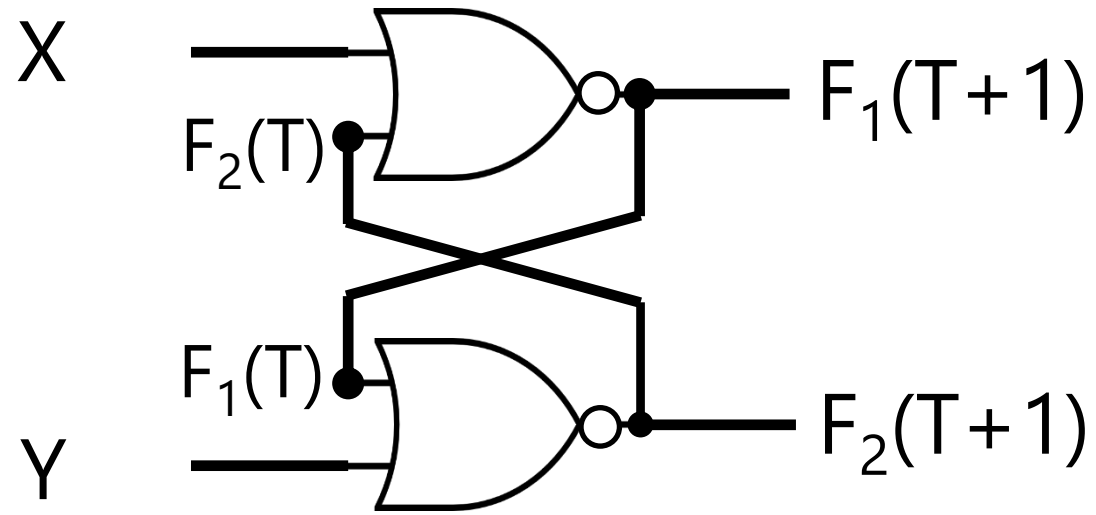
Try OR gates



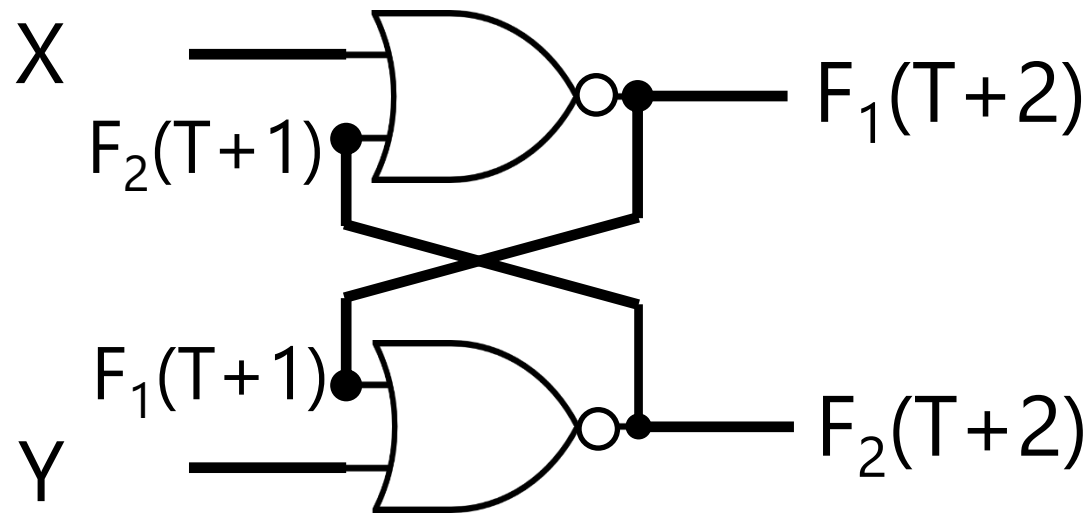
How about OR gates? Do they solve the problem? **No! Why?**

Y	X	$F_1(T+1)$	$F_2(T+1)$
0	0		
0	1		
1	0		
1	1		

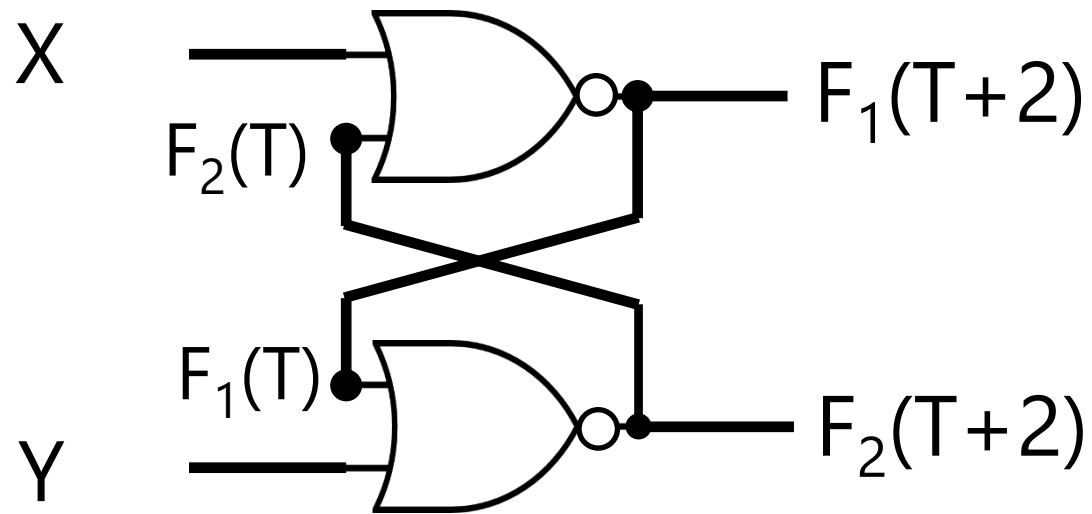
Try NOR gates



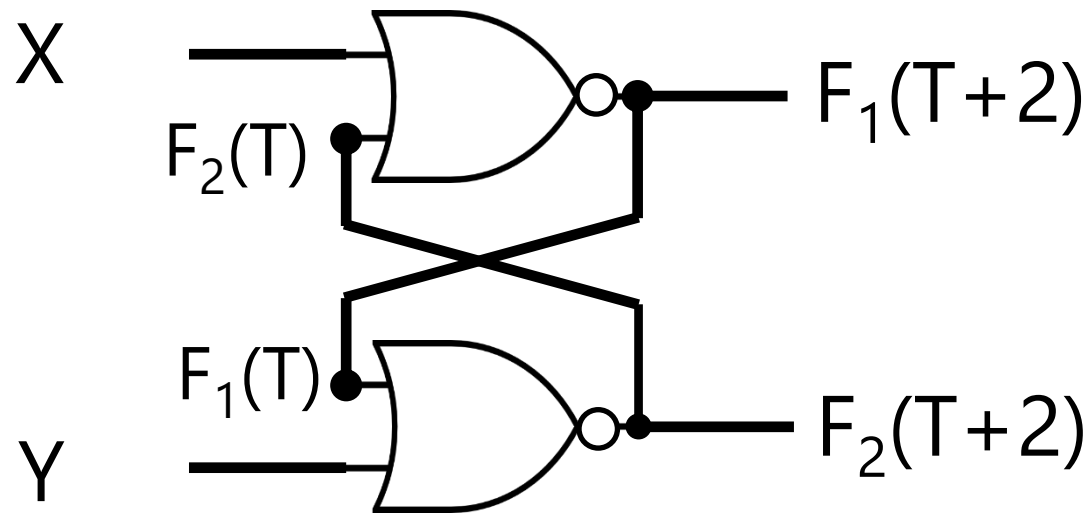
Y	X	$F_1(T+1)$	$F_2(T+1)$
0	0	$F'_2(T)$	$F'_1(T)$
0	1	0	$F'_1(T)$
1	0	$F'_2(T)$	0
1	1	0	0



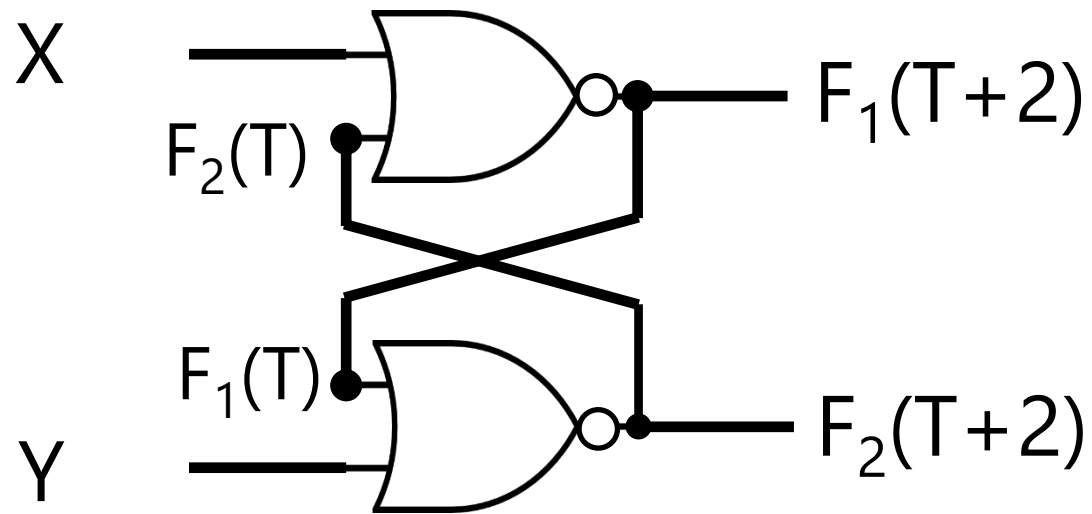
Y	X	$F_1(T+1)$	$F_2(T+1)$	$F_1(T+2)$	$F_2(T+2)$
0	0	$F'_2(T)$	$F'_1(T)$	$F'_2(T+1)$	$F'_1(T+1)$
0	1	0	$F'_1(T)$	0	$F'_1(T+1)$
1	0	$F'_2(T)$	0	$F'_2(T+1)$	0
1	1	0	0	0	0



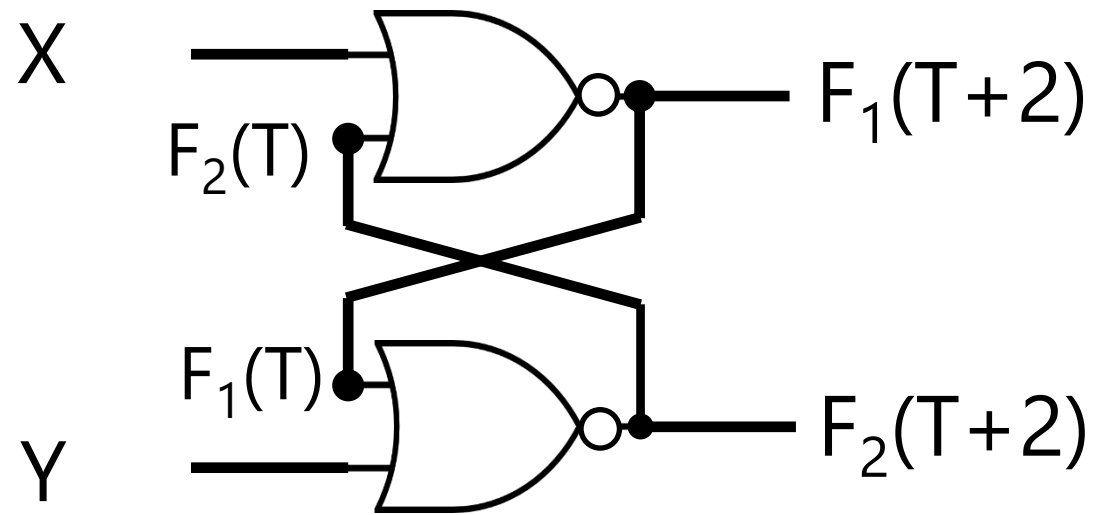
Y	X	$F_1(T+1)$	$F_2(T+1)$	$F_1(T+2)$	$F_2(T+2)$
0	0	$F'_2(T)$	$F'_1(T)$	$F'_2(T+1)$	$F'_1(T+1)$
0	1	0	$F'_1(T)$	0	$F'_1(T+1)$
1	0	$F'_2(T)$	0	$F'_2(T+1)$	0
1	1	0	0	0	0



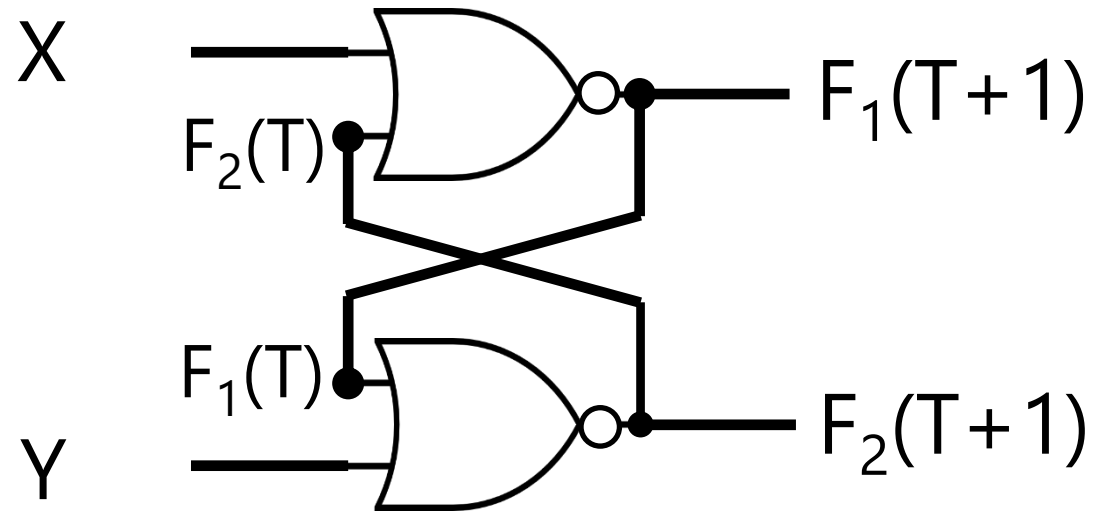
Y	X	$F_1(T+1)$	$F_2(T+1)$	$F_1(T+2)$	$F_2(T+2)$
0	0	$F'_2(T)$	$F'_1(T)$	$F_1(T)$	$F'_1(T+1)$
0	1	0	$F'_1(T)$	0	$F'_1(T+1)$
1	0	$F'_2(T)$	0	$F'_2(T+1)$	0
1	1	0	0	0	0



Y	X	$F_1(T+1)$	$F_2(T+1)$	$F_1(T+2)$	$F_2(T+2)$
0	0	$F'_2(T)$	$F'_1(T)$	$F_1(T)$	$F_2(T)$
0	1	0	$F'_1(T)$	0	$F'_1(T+1)$
1	0	$F'_2(T)$	0	$F'_2(T+1)$	0
1	1	0	0	0	0

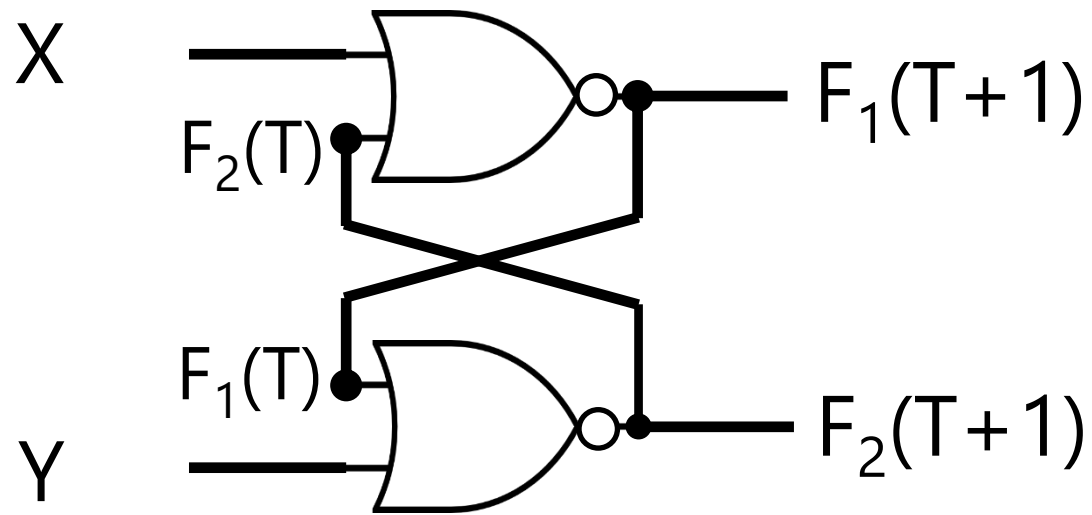


Y	X	$F_1(T+1)$	$F_2(T+1)$	$F_1(T+2)$	$F_2(T+2)$
0	0	$F'_2(T)$	$F'_1(T)$	$F_1(T)$	$F_2(T)$
0	1	0	$F'_1(T)$	0	1
1	0	$F'_2(T)$	0	$F'_2(T+1)$	0
1	1	0	0	0	0

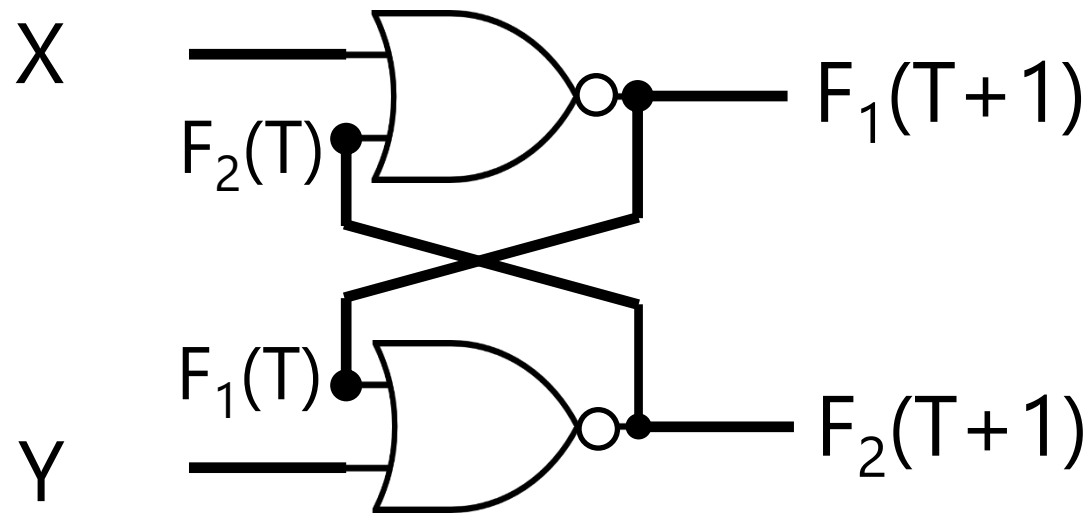


WOW! If wait longer, it stores the current state!

Y	X	$F_1(T+1)$	$F_2(T+1)$	$F_1(T+2)$	$F_2(T+2)$
0	0	$F'_2(T)$	$F'_1(T)$	$F_1(T)$	$F_2(T)$
0	1	0	$F'_1(T)$	0	1
1	0	$F'_2(T)$	0	1	0
1	1	0	0	0	0

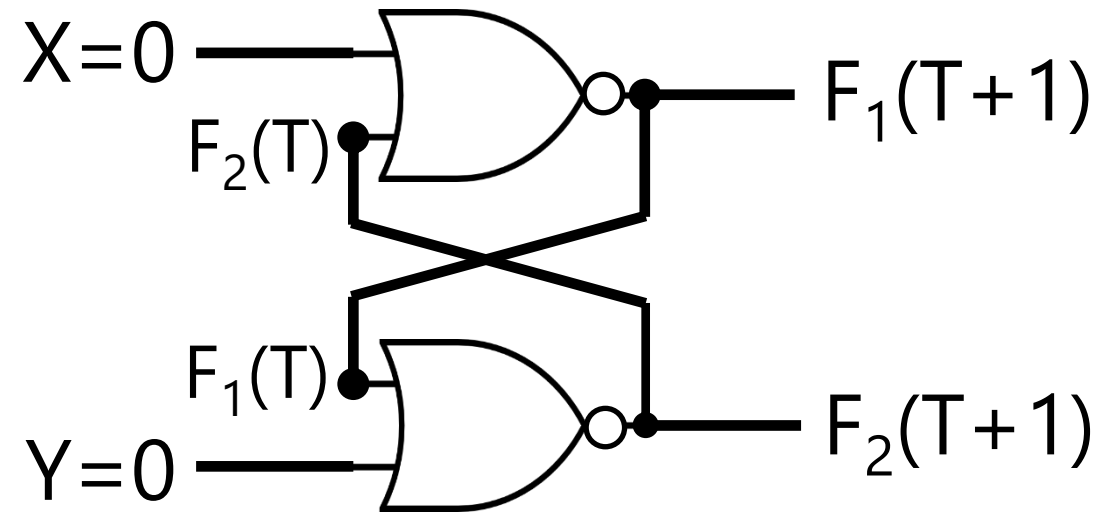


Y	X	$F_1(T+1)$	$F_2(T+1)$	$F_1(T+2)$	$F_2(T+2)$	$F_1(T+3)$	$F_2(T+3)$	$F_1(T+4)$	$F_2(T+4)$
0	0	$F'_2(T)$	$F'_1(T)$	$F_1(T)$	$F_2(T)$	$F'_2(T+2)$	$F'_1(T+2)$	$F'_2(T+3)$	$F'_1(T+3)$
0	1	0	$F'_1(T)$	0	1	0	$F'_1(T+2)$	0	$F'_1(T+3)$
1	0	$F'_2(T)$	0	1	0	$F'_2(T+2)$	0	$F'_2(T+3)$	0
1	1	0	0	0	0	0	0	0	0



Y	X	$F_1(T+1)$	$F_2(T+1)$	$F_1(T+2)$	$F_2(T+2)$	$F_1(T+3)$	$F_2(T+3)$	$F_1(T+4)$	$F_2(T+4)$
0	0	$F'_2(T)$	$F'_1(T)$	$F_1(T)$	$F_2(T)$	$F'_2(T)$	$F'_1(T)$	$F_1(T)$	$F_2(T)$
0	1	0	$F'_1(T)$	0	1	0	1	0	1
1	0	$F'_2(T)$	0	1	0	0	0	1	0
1	1	0	0	0	0	0	0	0	0

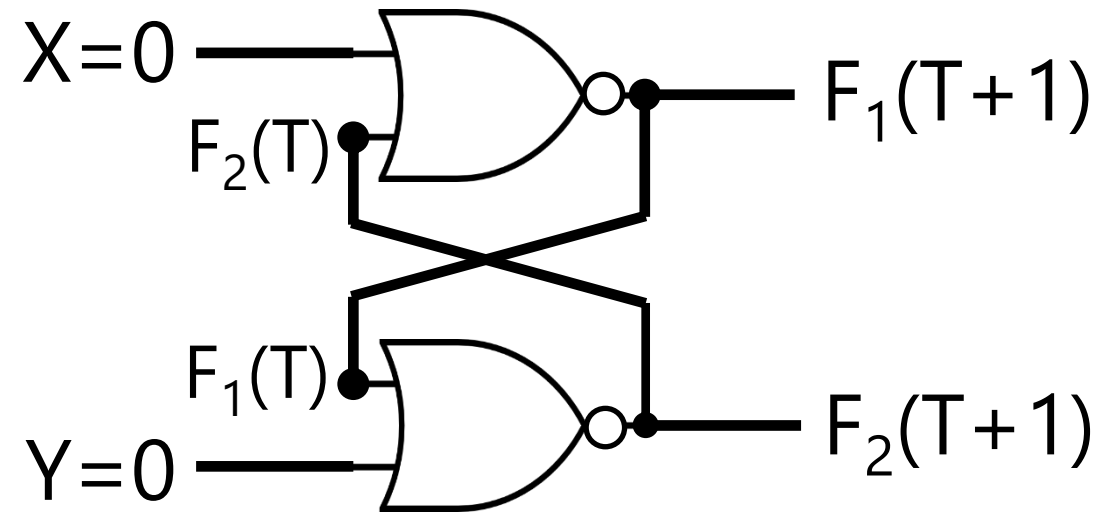
Store/Hold Action



Y	X	$F_1(T+1)$	$F_2(T+1)$	$F_1(T+2)$	$F_2(T+2)$	$F_1(T+3)$	$F_2(T+3)$	$F_1(T+4)$	$F_2(T+4)$
0	0	$F'_2(T)$	$F'_1(T)$	$F_1(T)$	$F_2(T)$	$F'_2(T)$	$F'_1(T)$	$F_1(T)$	$F_2(T)$
0	1	0	$F'_1(T)$	0	1	0	1	0	1
1	0	$F'_2(T)$	0	1	0	0	0	1	0
1	1	0	0	0	0	0	0	0	0

Store/Hold Action

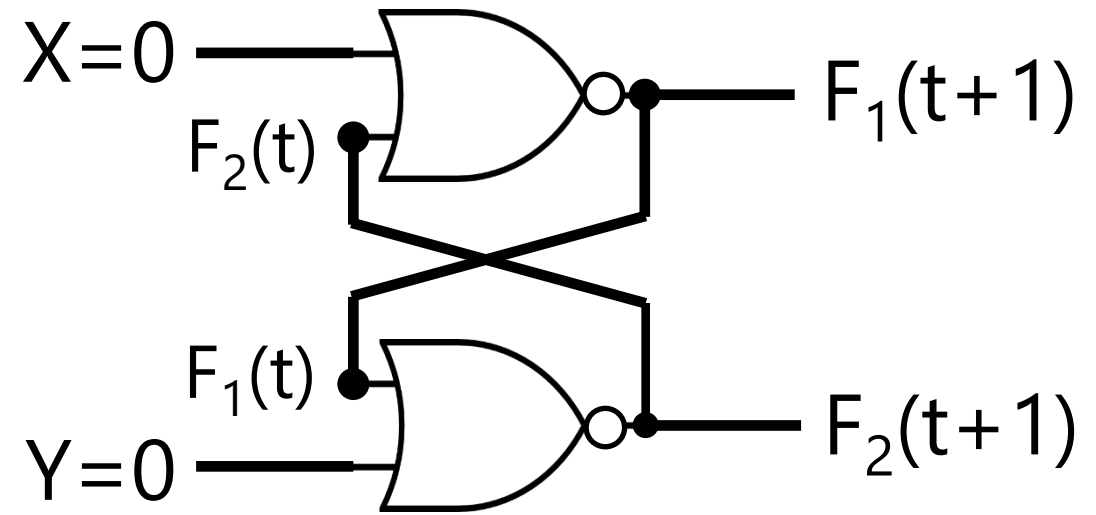
$2T \rightarrow t$



Y	X	$F_1(T+2)$	$F_2(T+2)$
0	0	$F_1(T)$	$F_2(T)$
0	1	0	1
1	0	1	0
1	1	0	0

Store/Hold Action

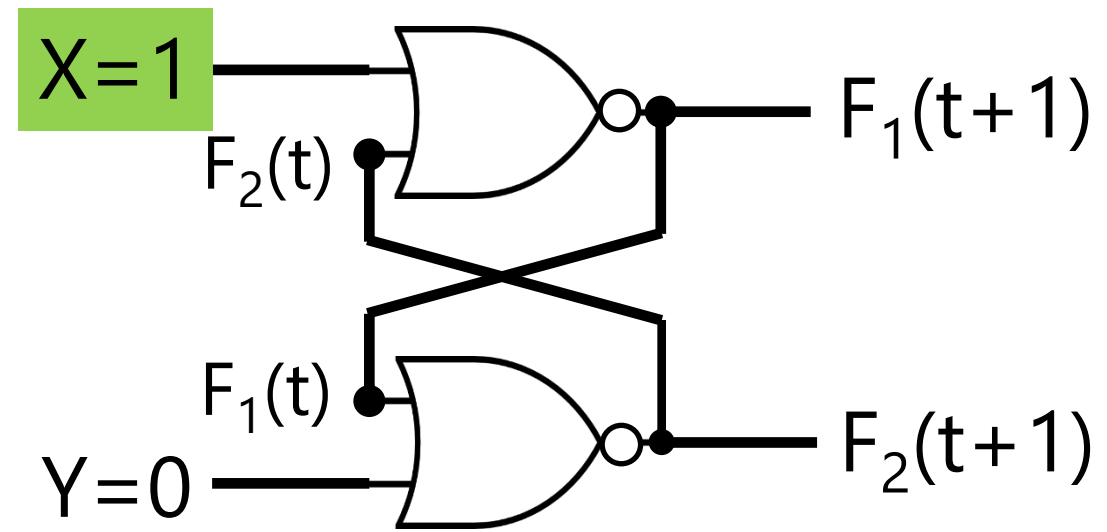
$2T \rightarrow t$



Y	X	$F_1(t+1)$	$F_2(t+1)$
0	0	$F_1(t)$	$F_2(t)$
0	1	0	1
1	0	1	0
1	1	0	0

Reset Action

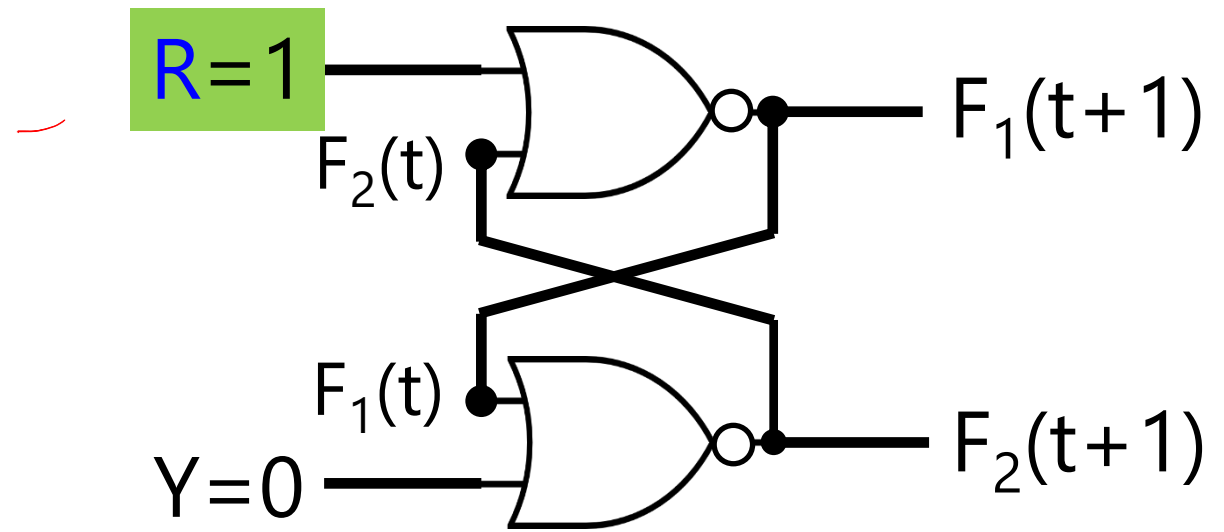
$2T \rightarrow t$



Y	X	$F_1(t+1)$	$F_2(t+1)$
0	0	$F_1(t)$	$F_2(t)$
0	1	0	1
1	0	1	0
1	1	0	0

Reset Action

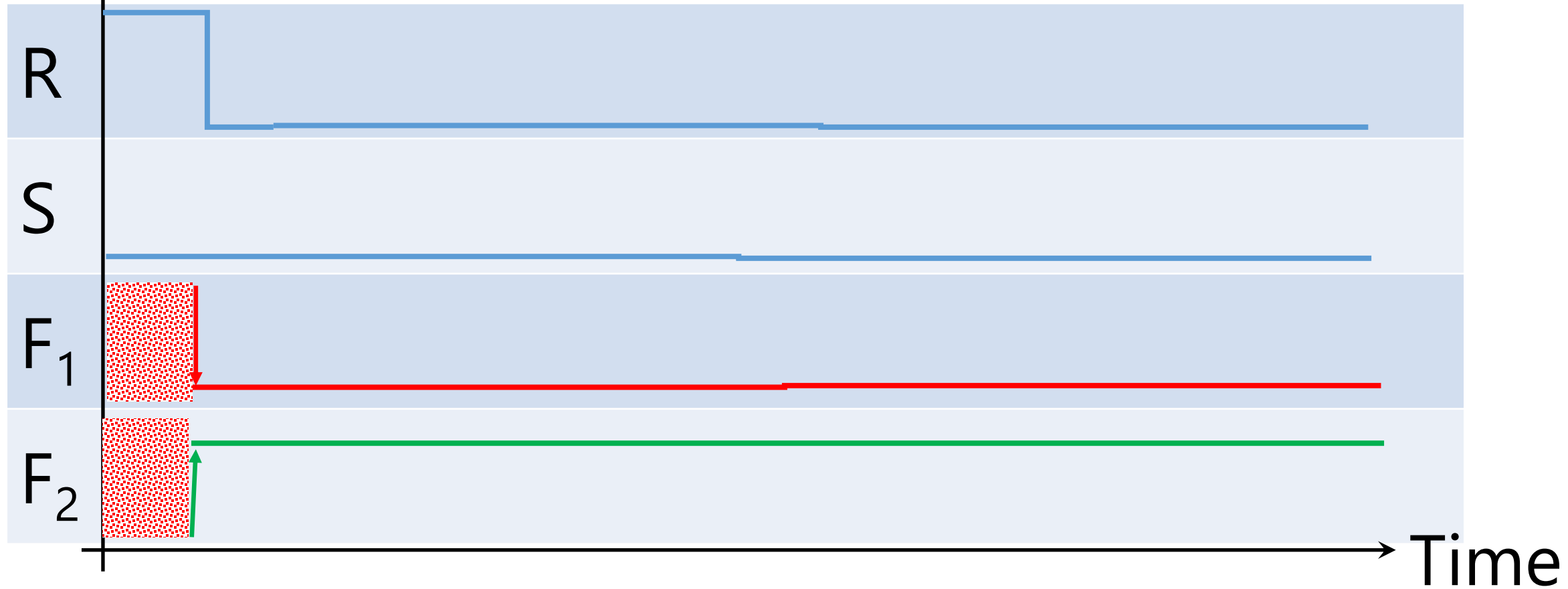
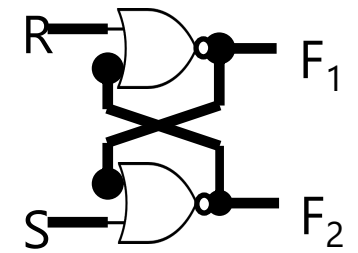
$2T \rightarrow t$



Y	R	$F_1(t+1)$	$F_2(t+1)$
0	0	$F_1(t)$	$F_2(t)$
0	1	0	1
1	0	1	0
1	1	0	0

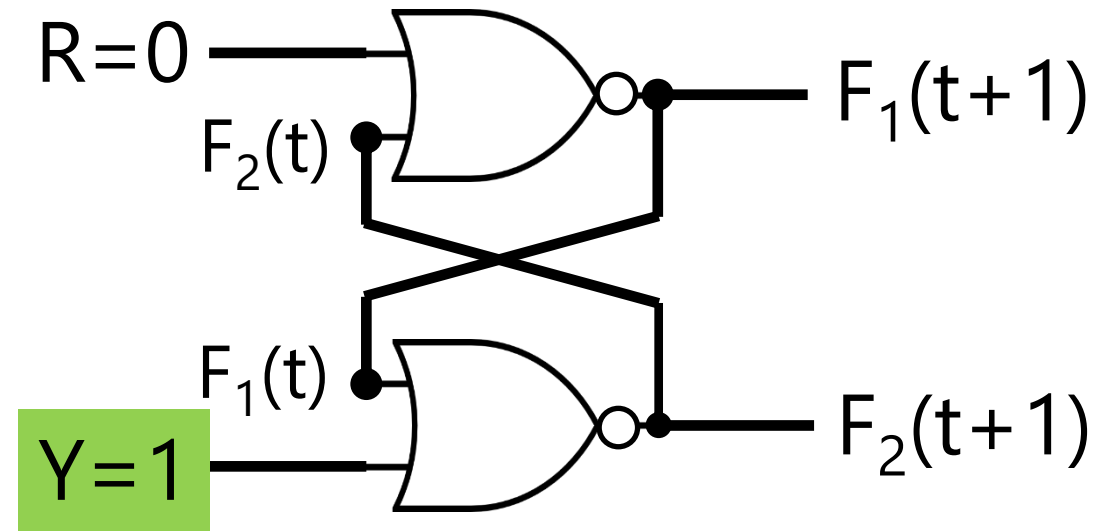
Voltage

Store "0": Reset $\rightarrow$ Hold



Set Action

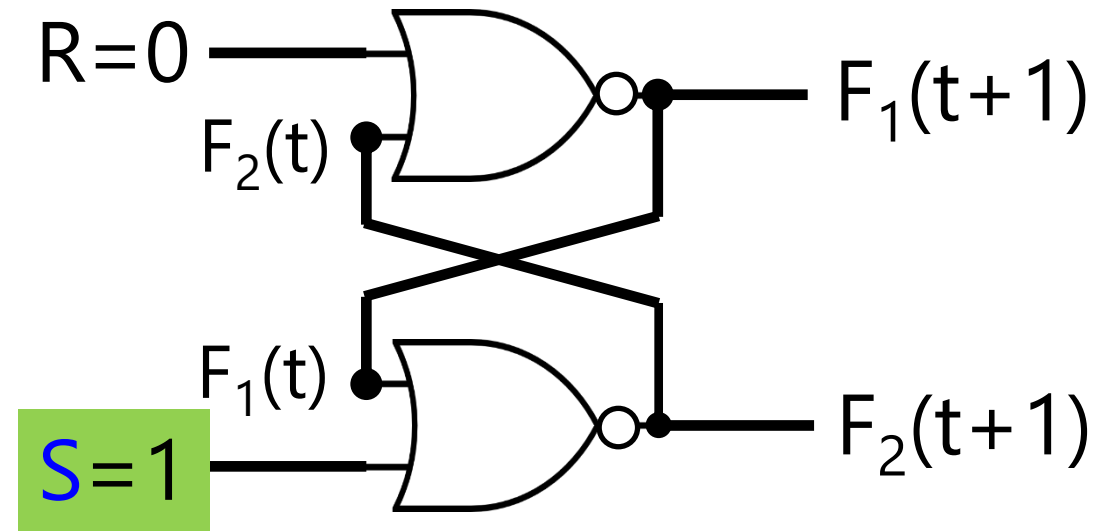
$2T \rightarrow t$



Y	R	$F_1(t+1)$	$F_2(t+1)$
0	0	$F_1(t)$	$F_2(t)$
0	1	0	1
1	0	1	0
1	1	0	0

Set Action

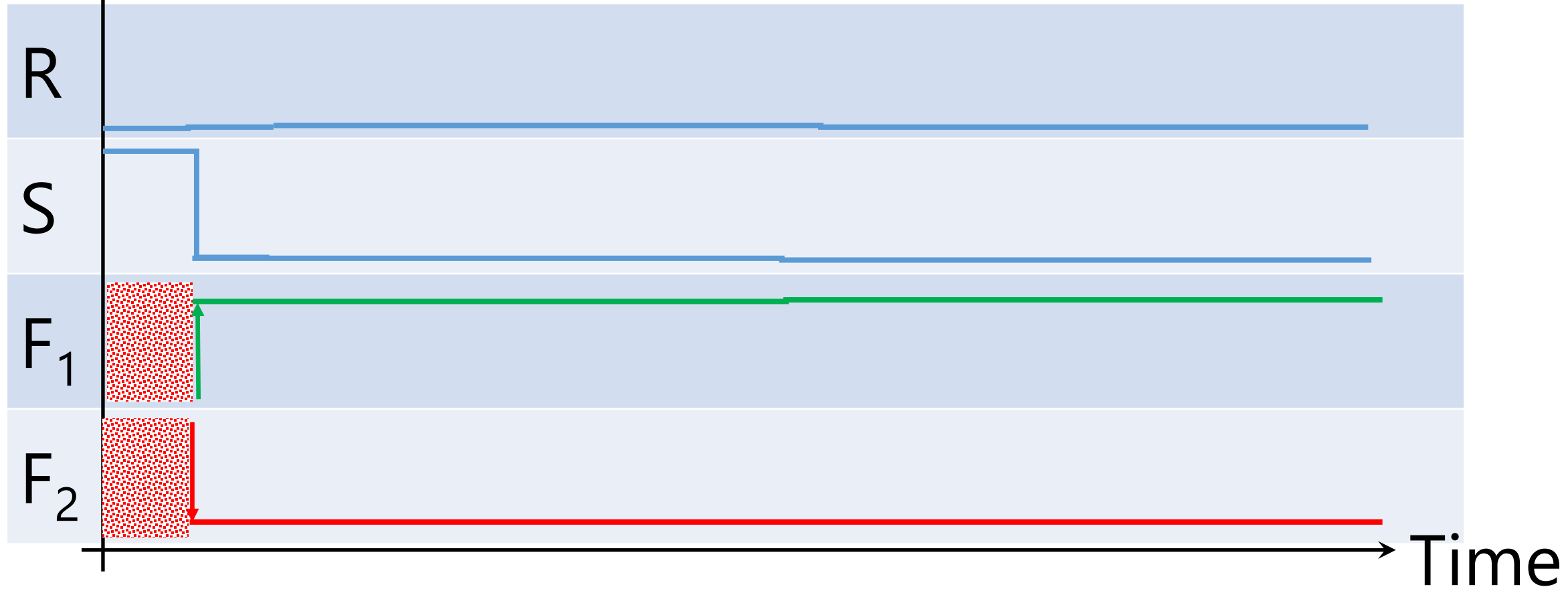
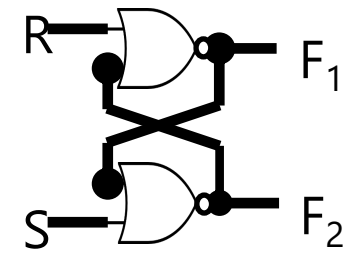
$2T \rightarrow t$



S	R	$F_1(t+1)$	$F_2(t+1)$
0	0	$F_1(T)$	$F_2(T)$
0	1	0	1
1	0	1	0
1	1	0	0

Voltage

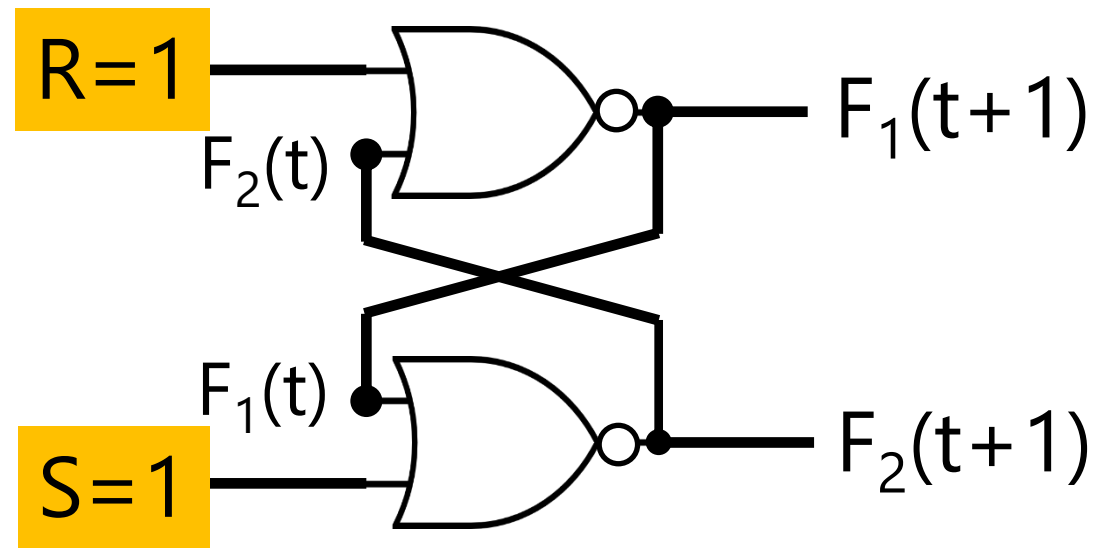
Store "1": Set $\rightarrow$ Hold



Forbidden Action

$2T \rightarrow t$

~~Set & Reset~~

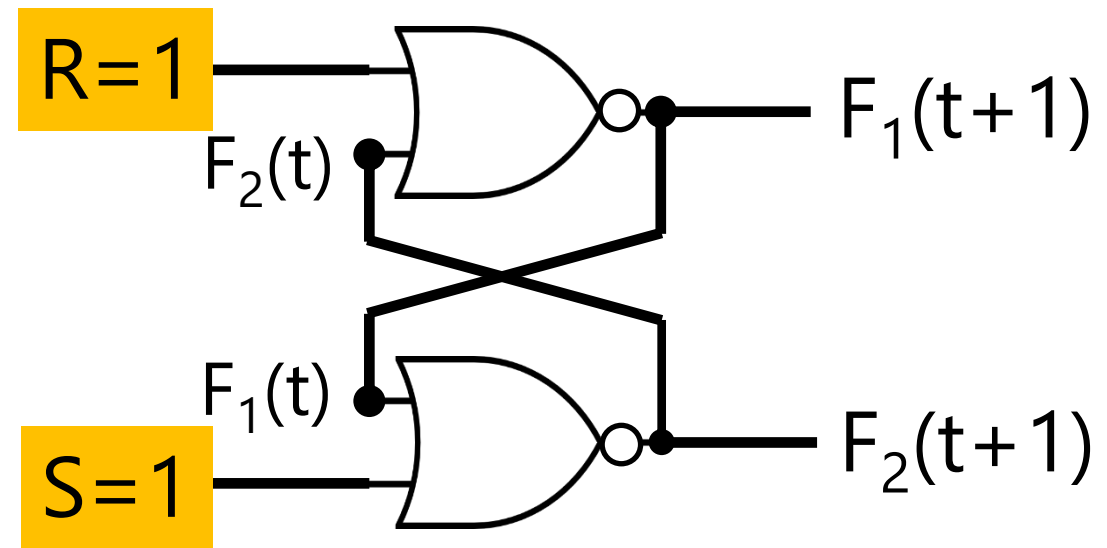


S	R	$F_1(t+1)$	$F_2(t+1)$
0	0	$F_1(t)$	$F_2(t)$
0	1	0	1
1	0	1	0
1	1	0	0

Forbidden Action

$2T \rightarrow t$

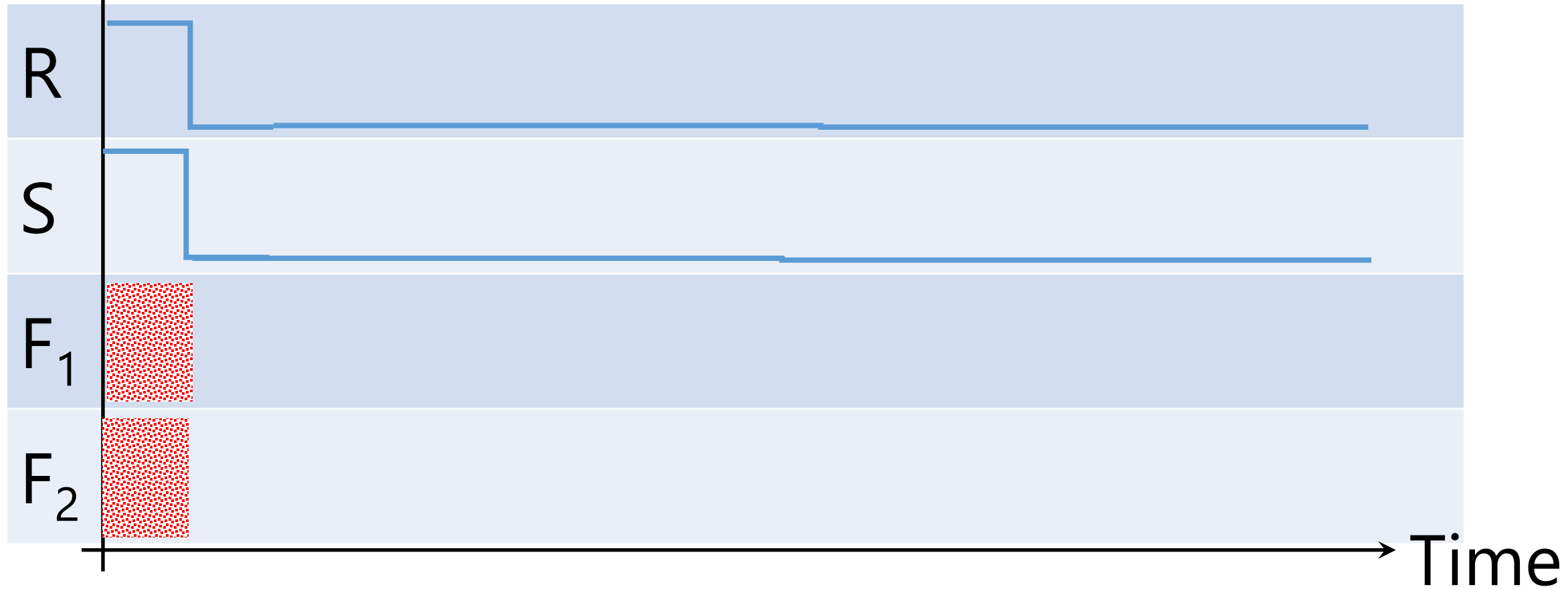
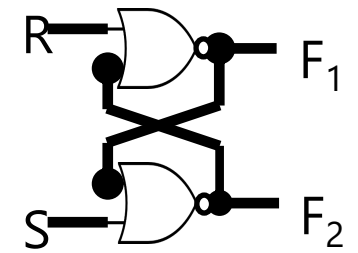
~~Set & Reset~~



S	R	$F_1(t+1)$	$F_2(t+1)$
0	0	$F_1(t)$	$F_2(t)$
0	1	0	1
1	0	1	0
1	1	X	X

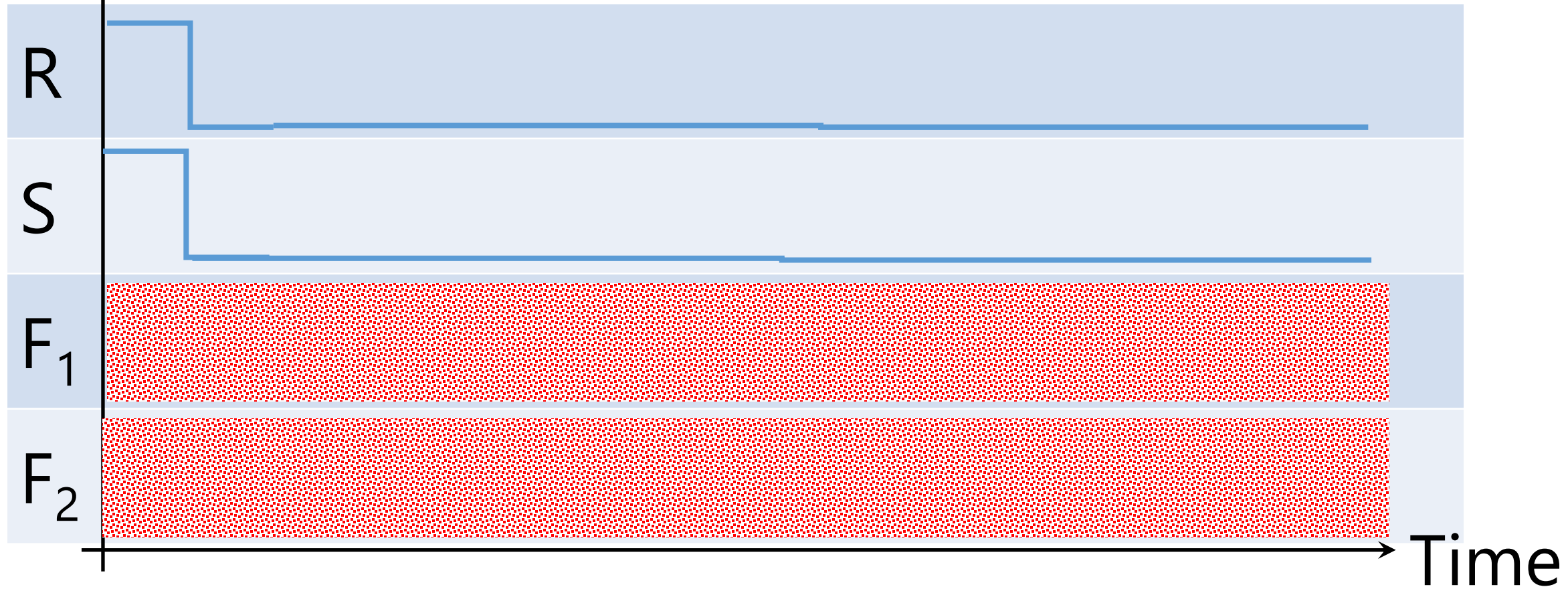
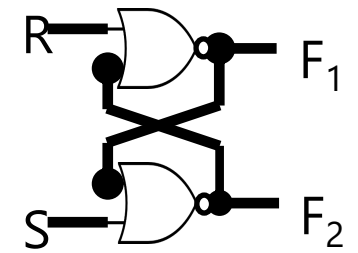
Voltage

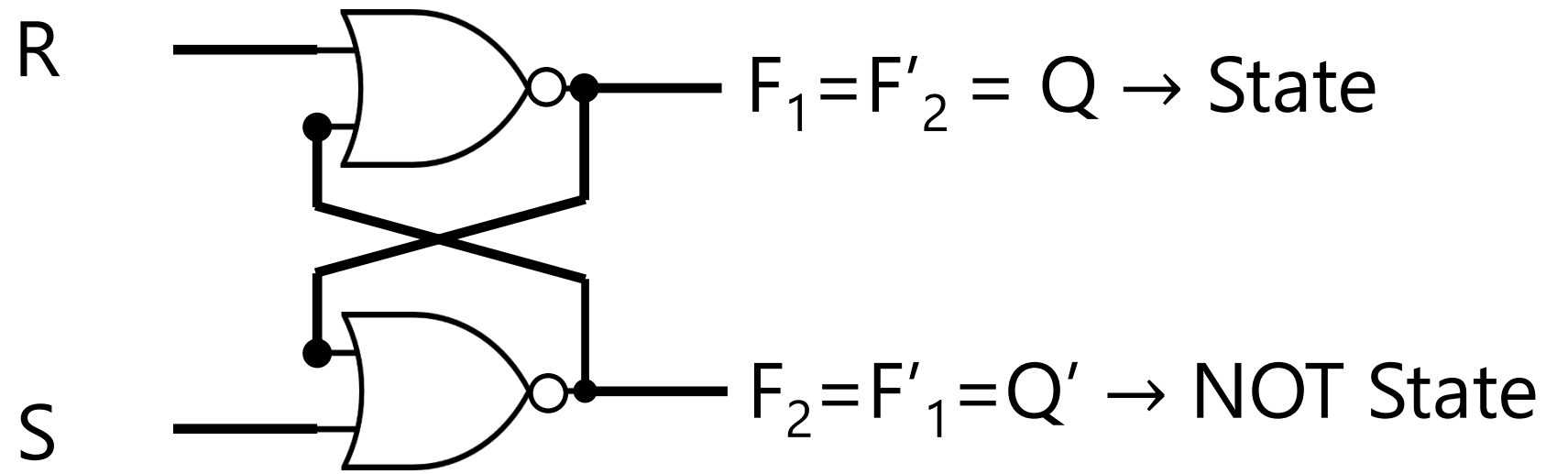
Don't Care State



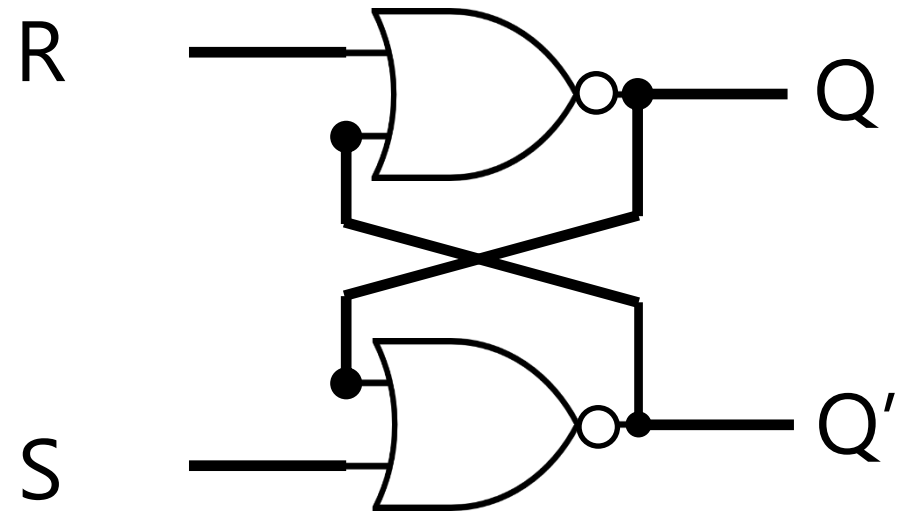
Voltage

Don't Care State

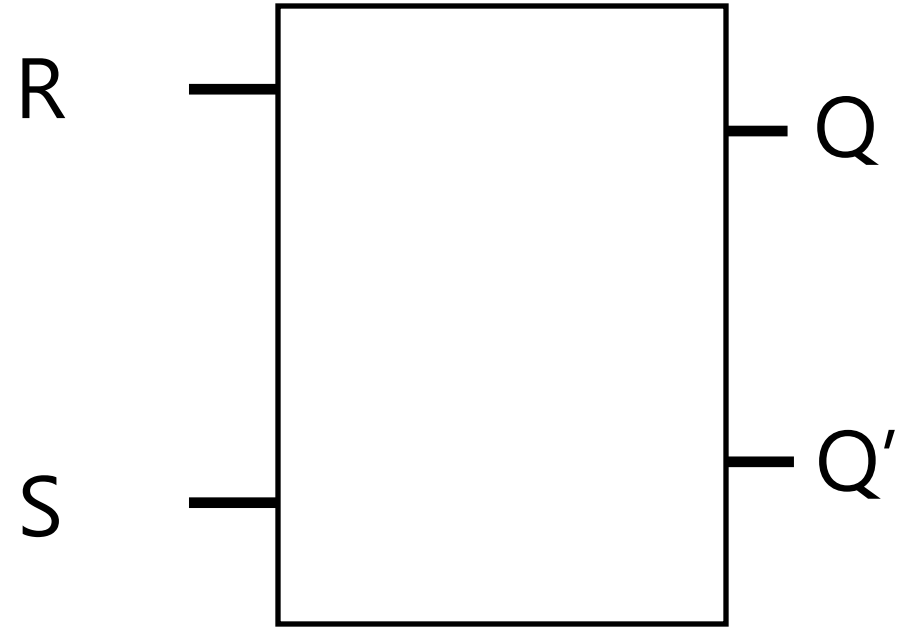




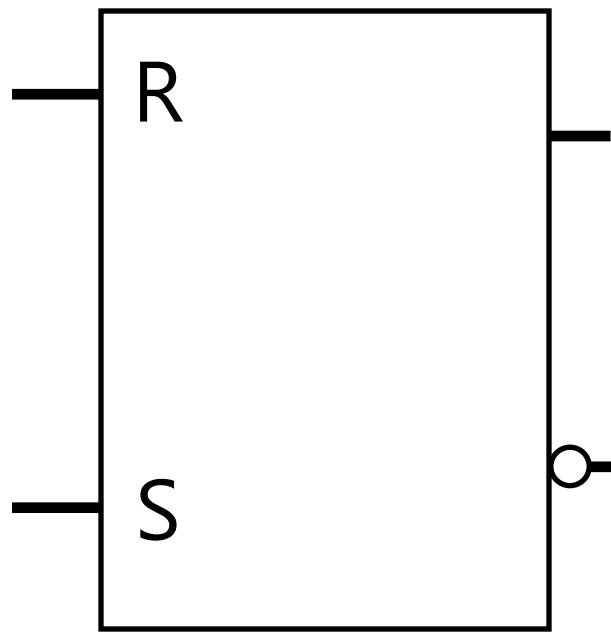
S	R	Q	Q'
0	0	Q_t	Q'_t
0	1	0	1
1	0	1	0
1	1	$\times$	$\times$



	S	R	Q	Q'
Hold	0	0	Q_t	Q'_t
Reset	0	1	0	1
Set	1	0	1	0
	1	1	x	x

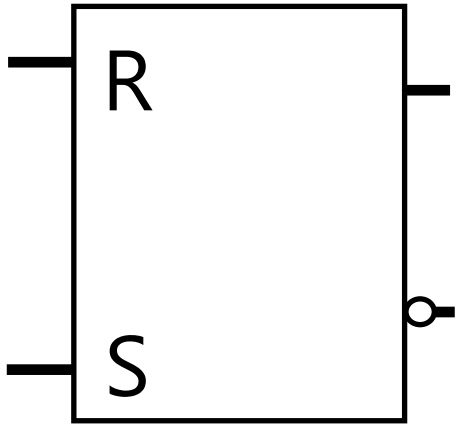


	S	R	Q	Q'
Hold	0	0	Q_t	Q'_t
Reset	0	1	0	1
Set	1	0	1	0
	1	1	$\times$	$\times$



	S	R	Q	Q'
Hold	0	0	Q_t	Q'_t
Reset	0	1	0	1
Set	1	0	1	0
	1	1	x	x

Block Diagram

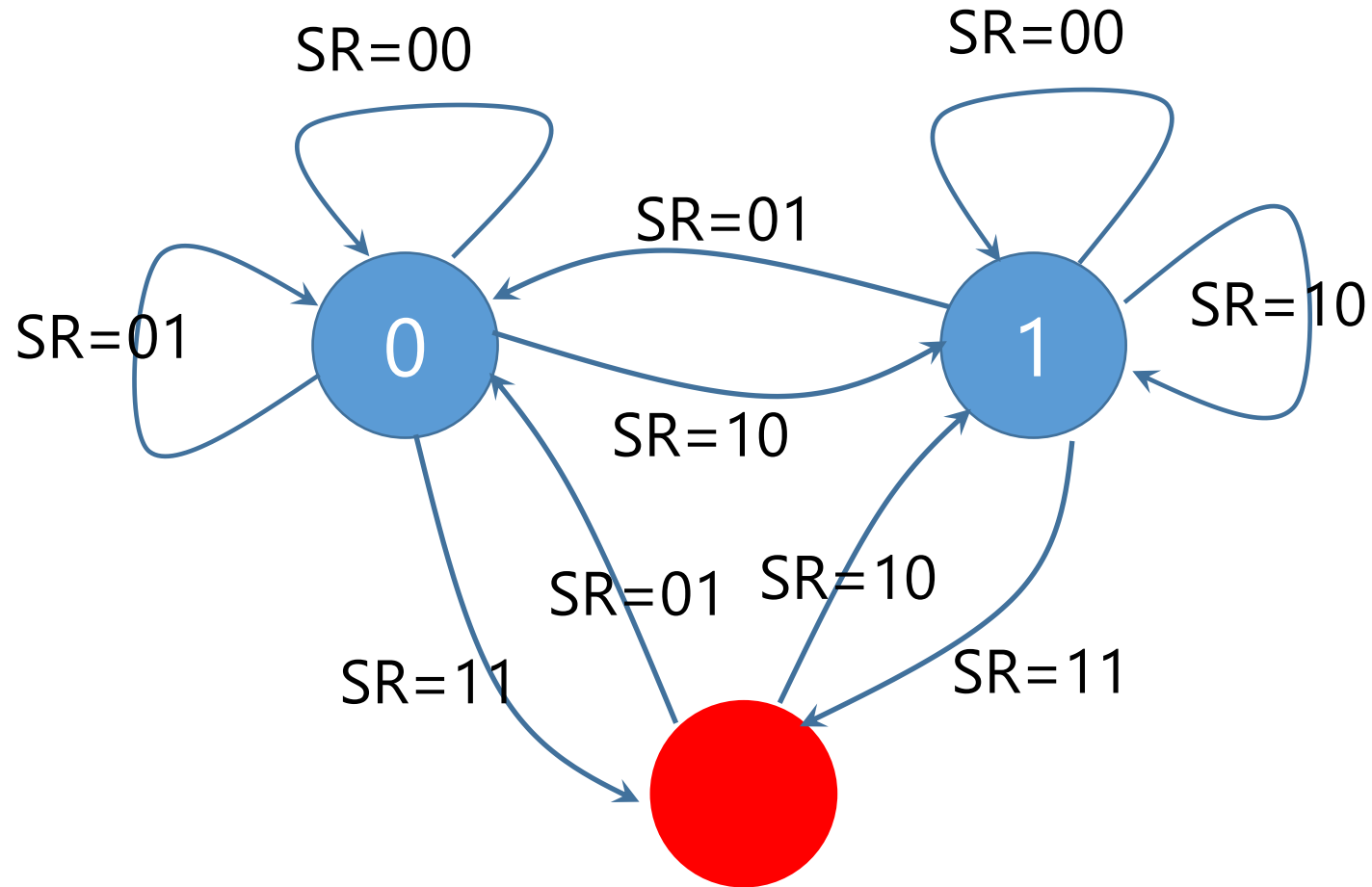


Characteristic Table

S	R	Q
0	0	Q_t
0	1	0
1	0	1
1	1	$\times$

State Transition Diagram

Life Cycle



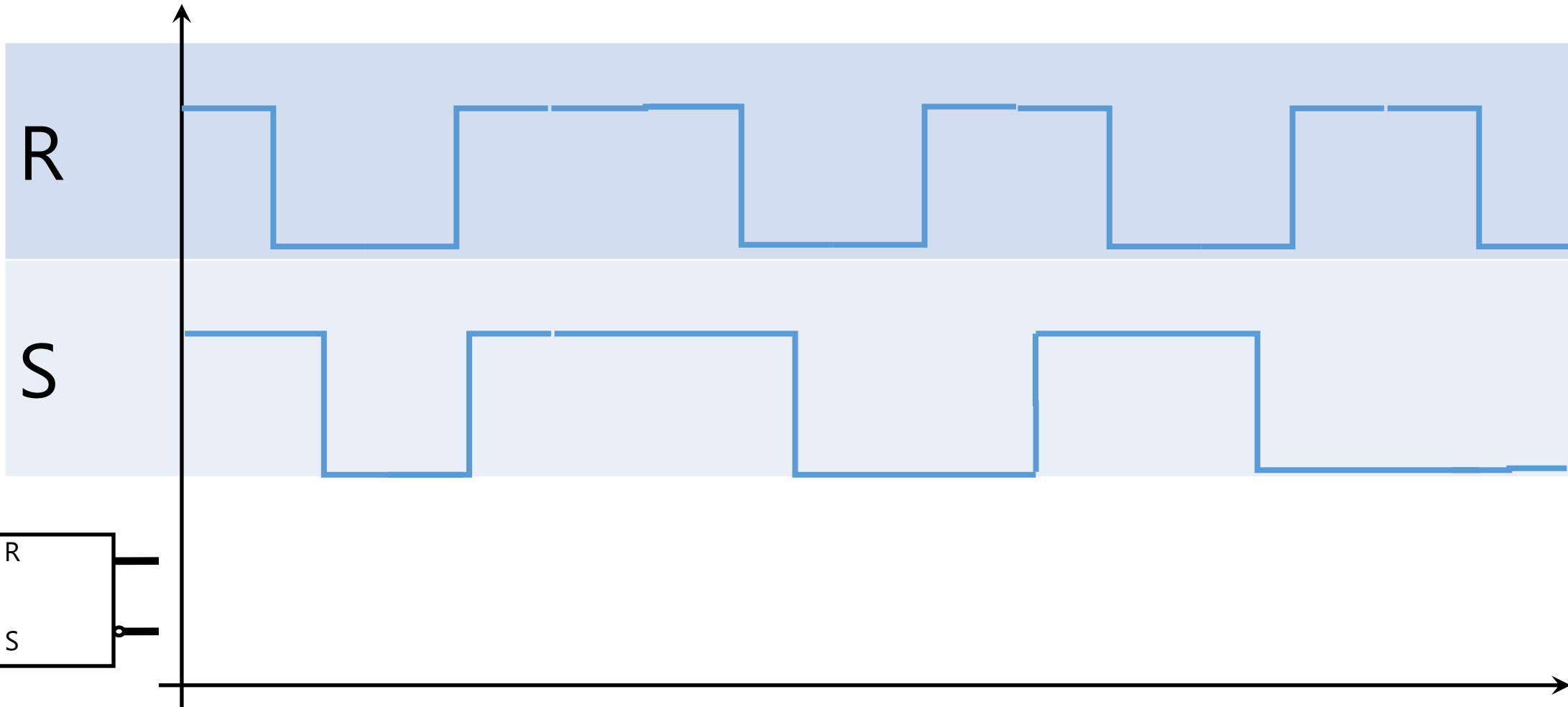
Voltage

R

S



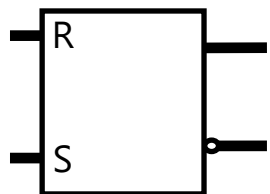
Time



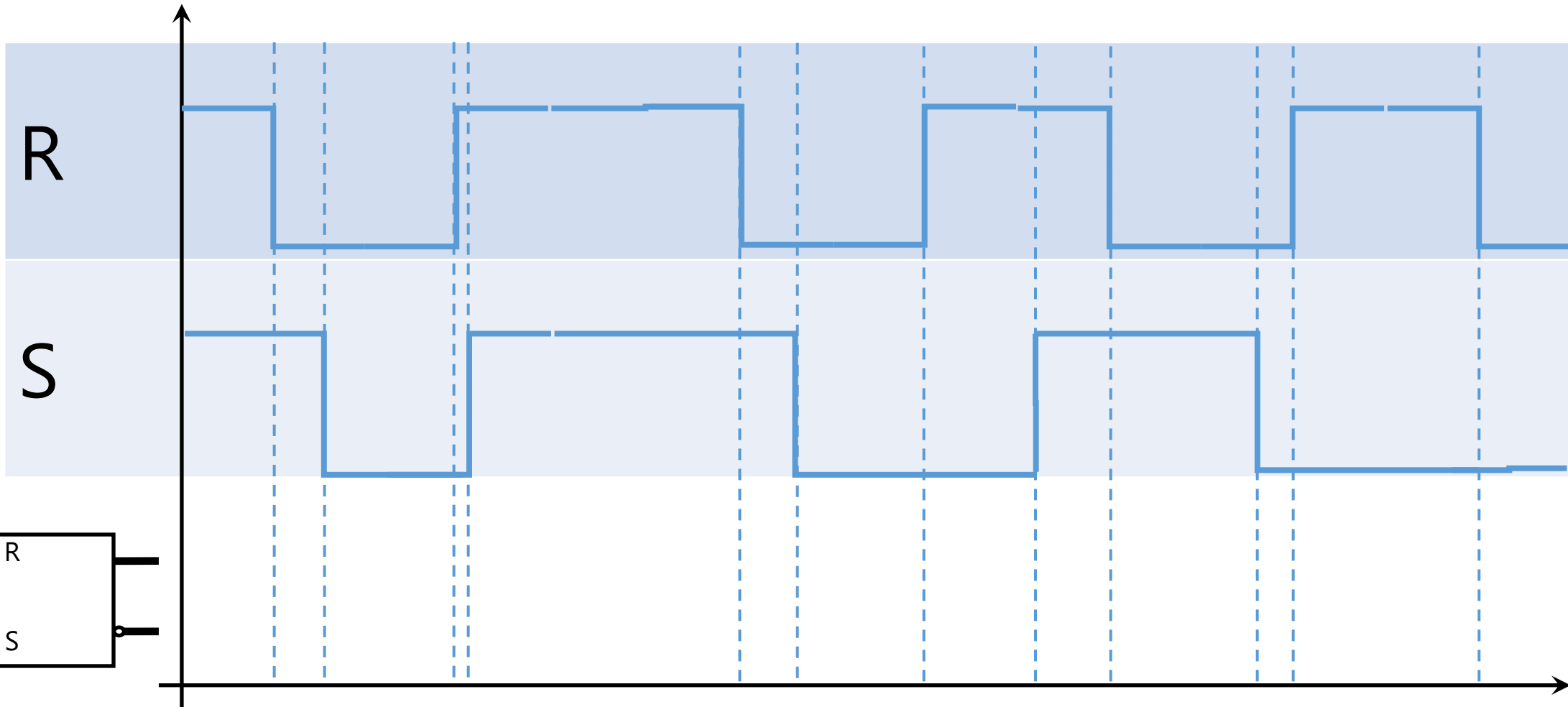
Voltage

R

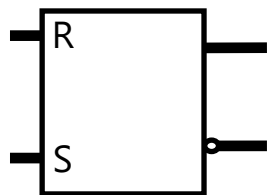
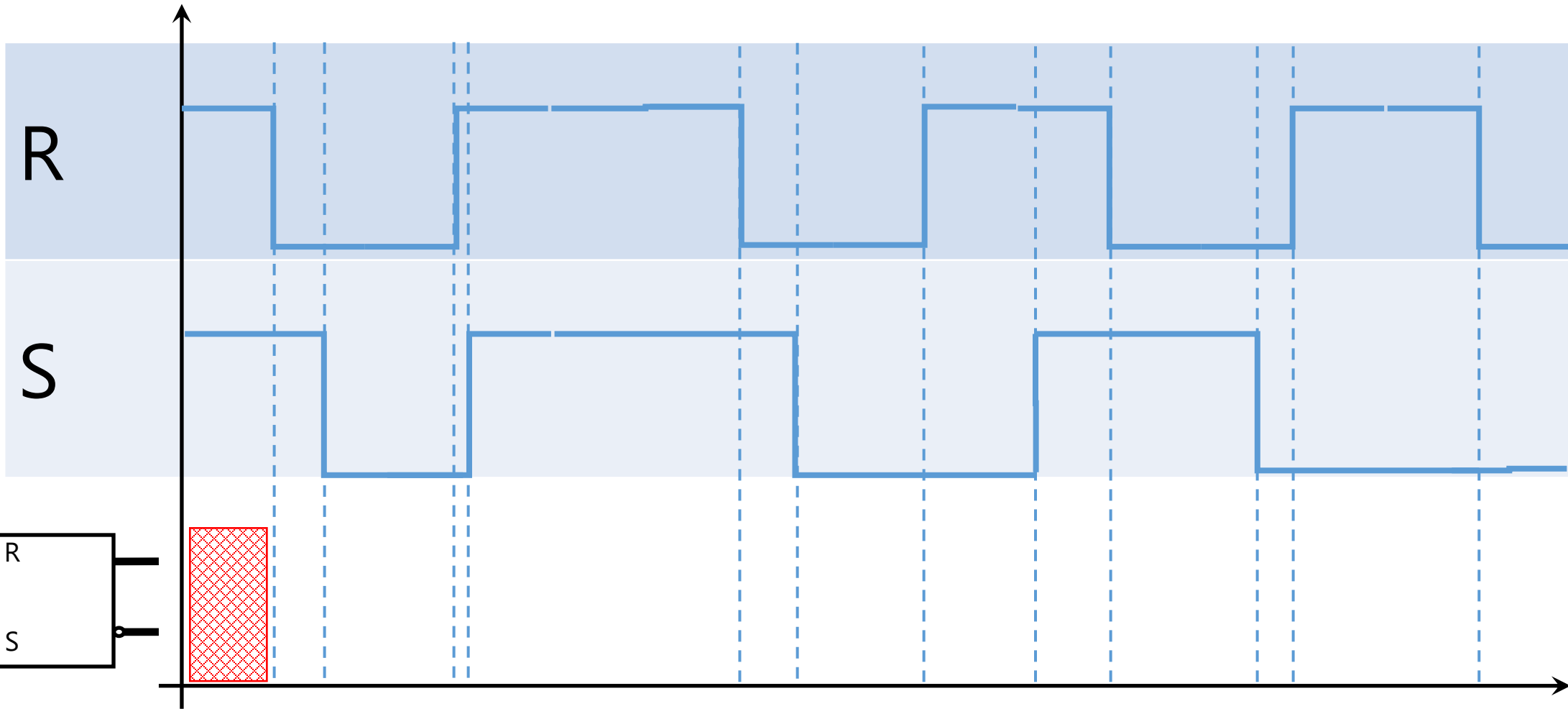
S



Time

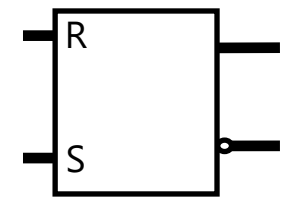
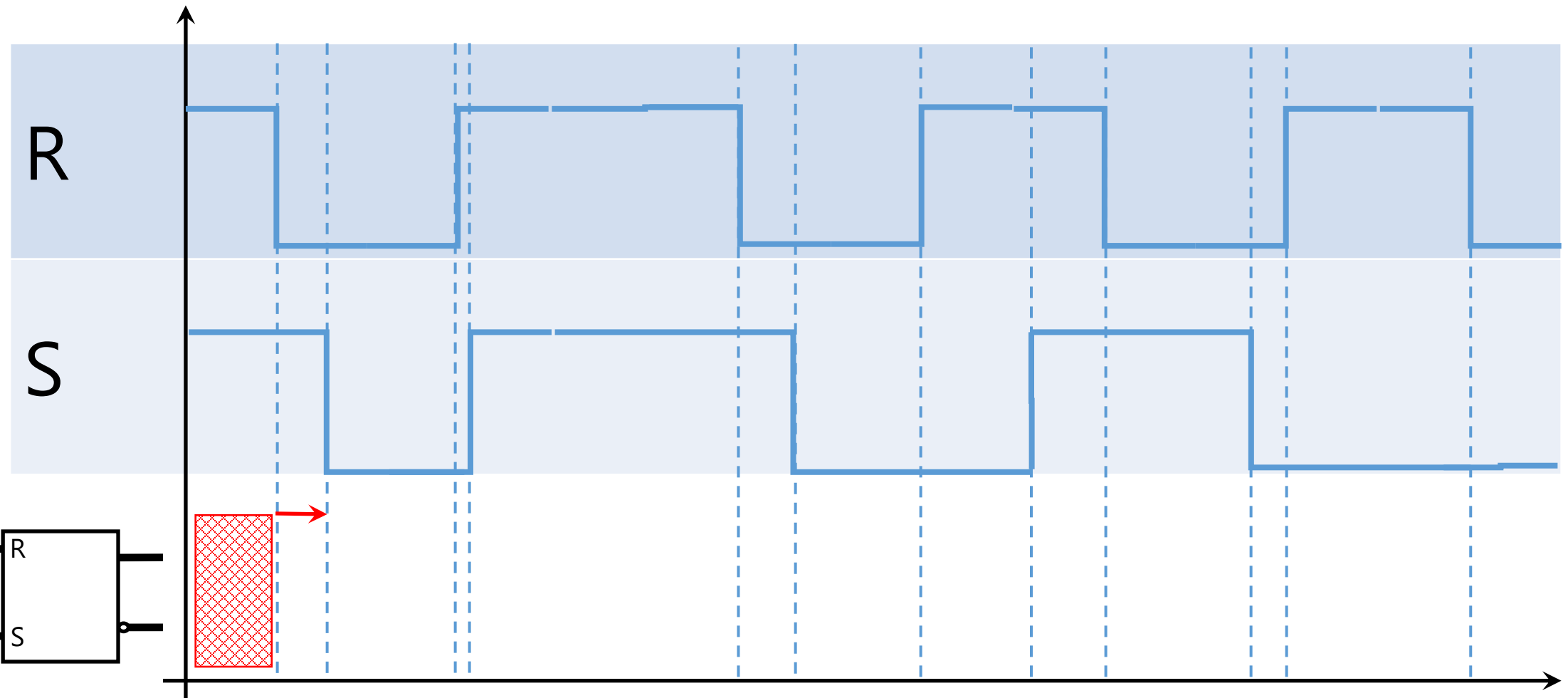


Voltage



Time

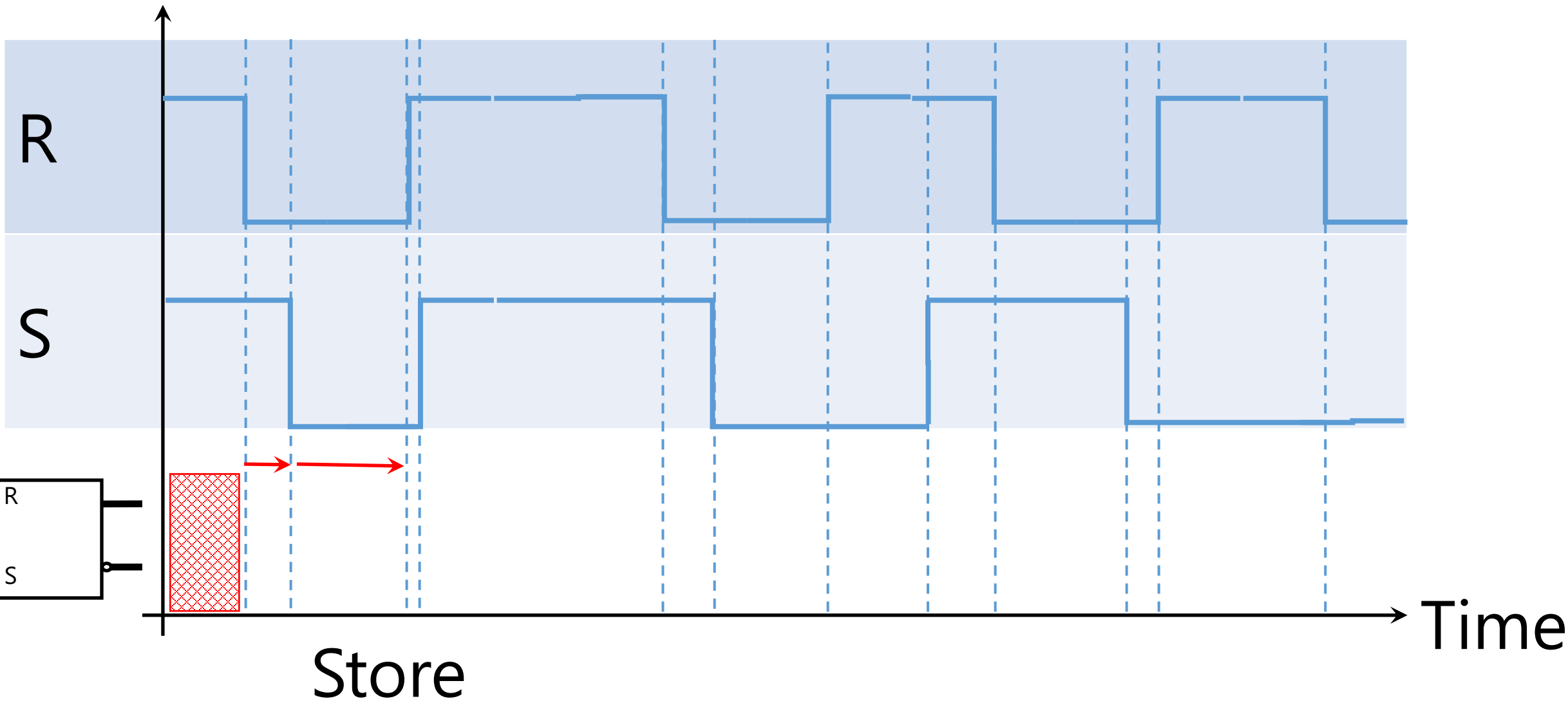
Voltage



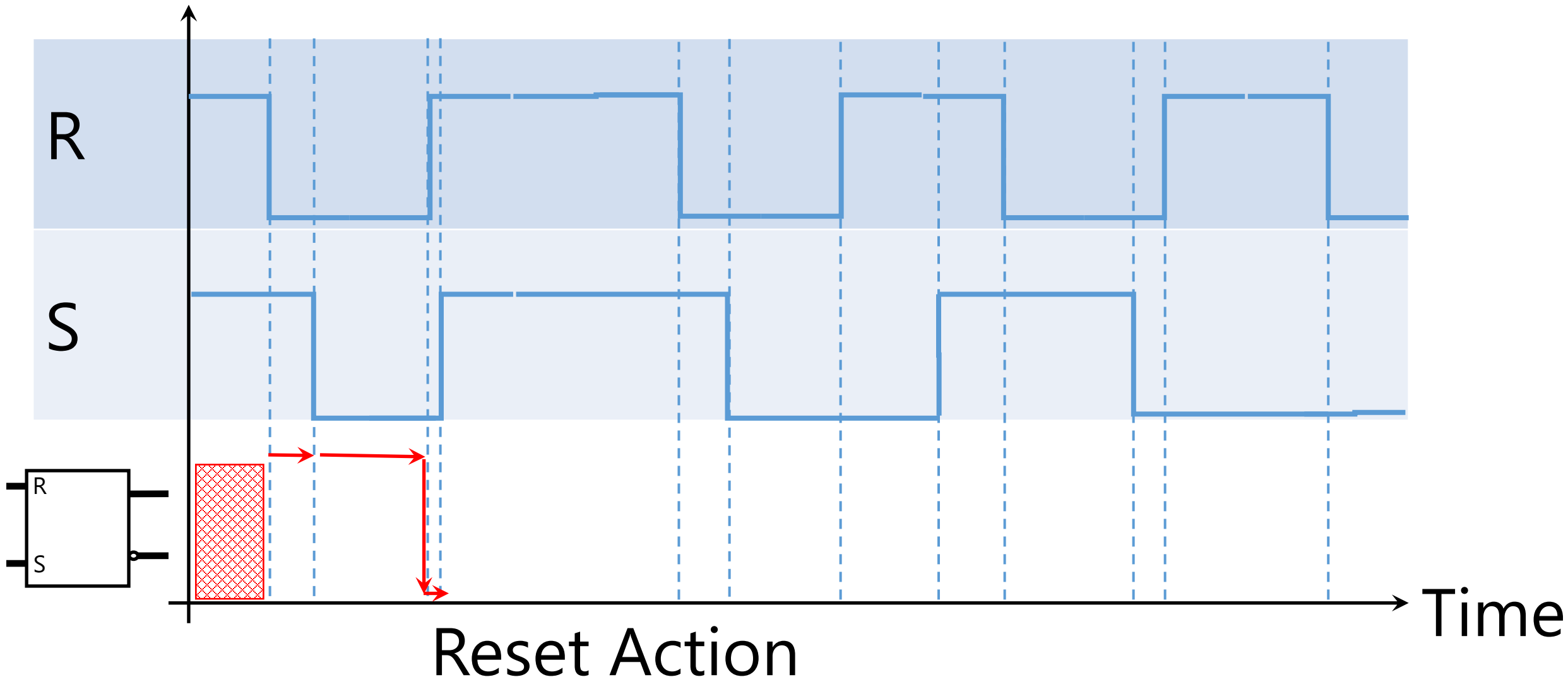
Set Action

Time

Voltage



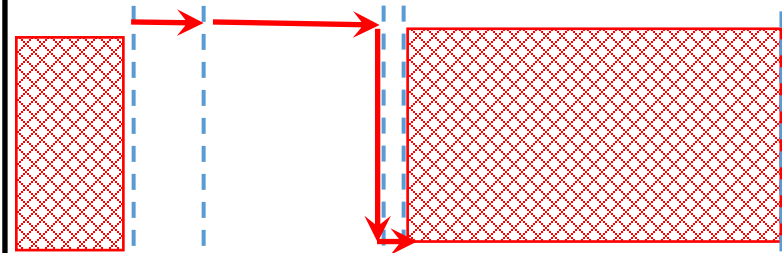
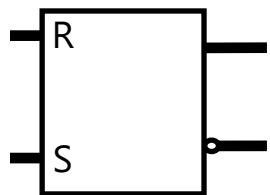
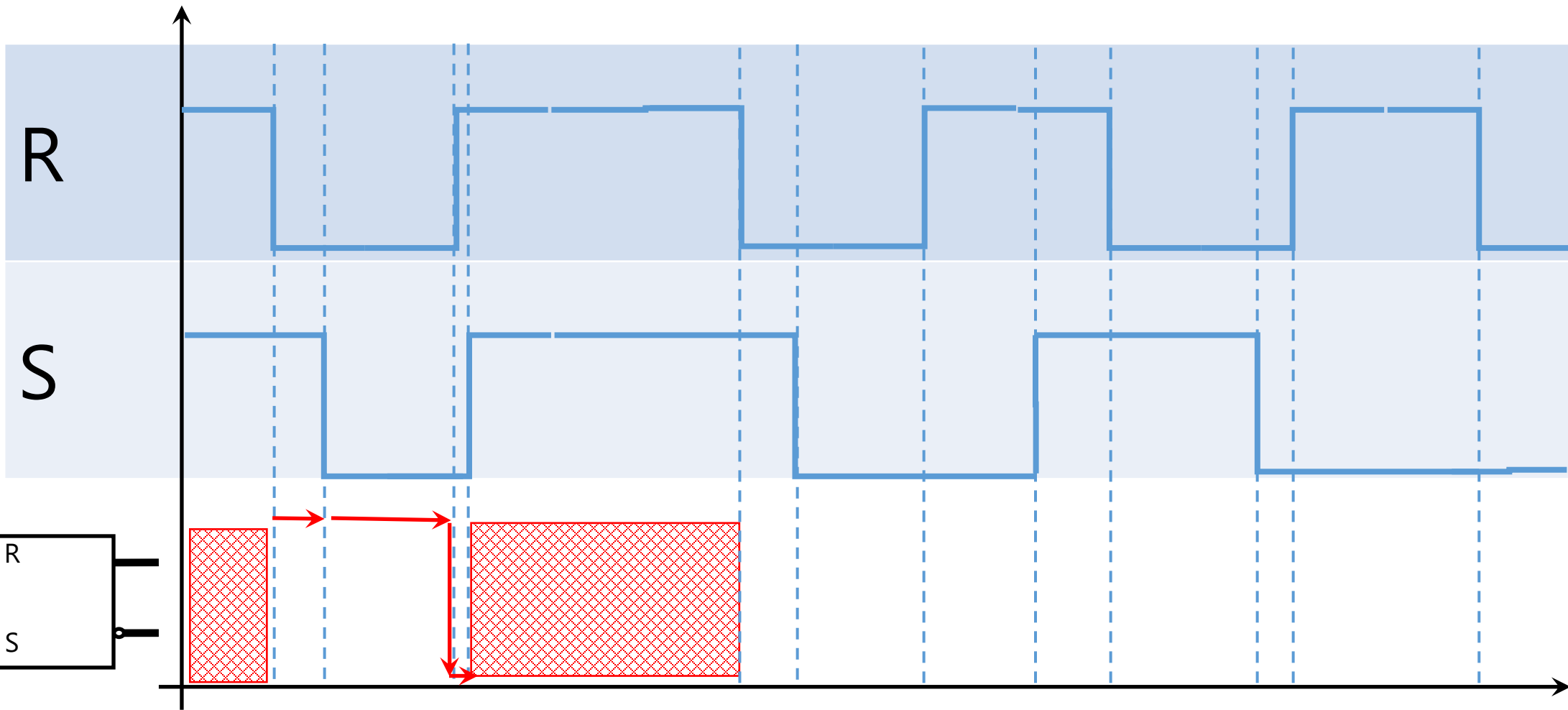
Voltage



Reset Action

Time

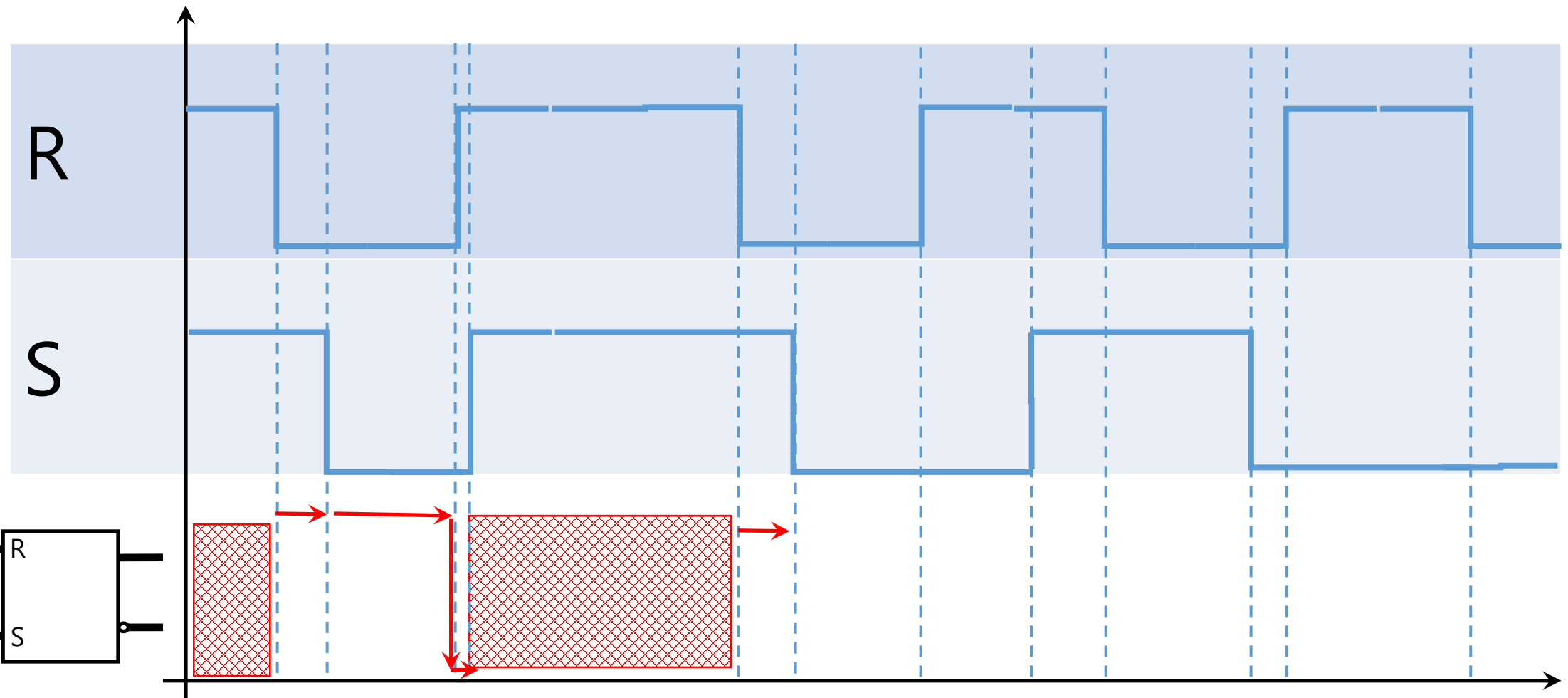
Voltage



Forbidden

Time

Voltage



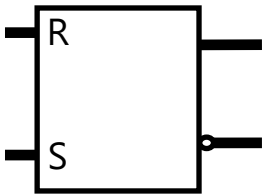
Set Action

Time

Voltage

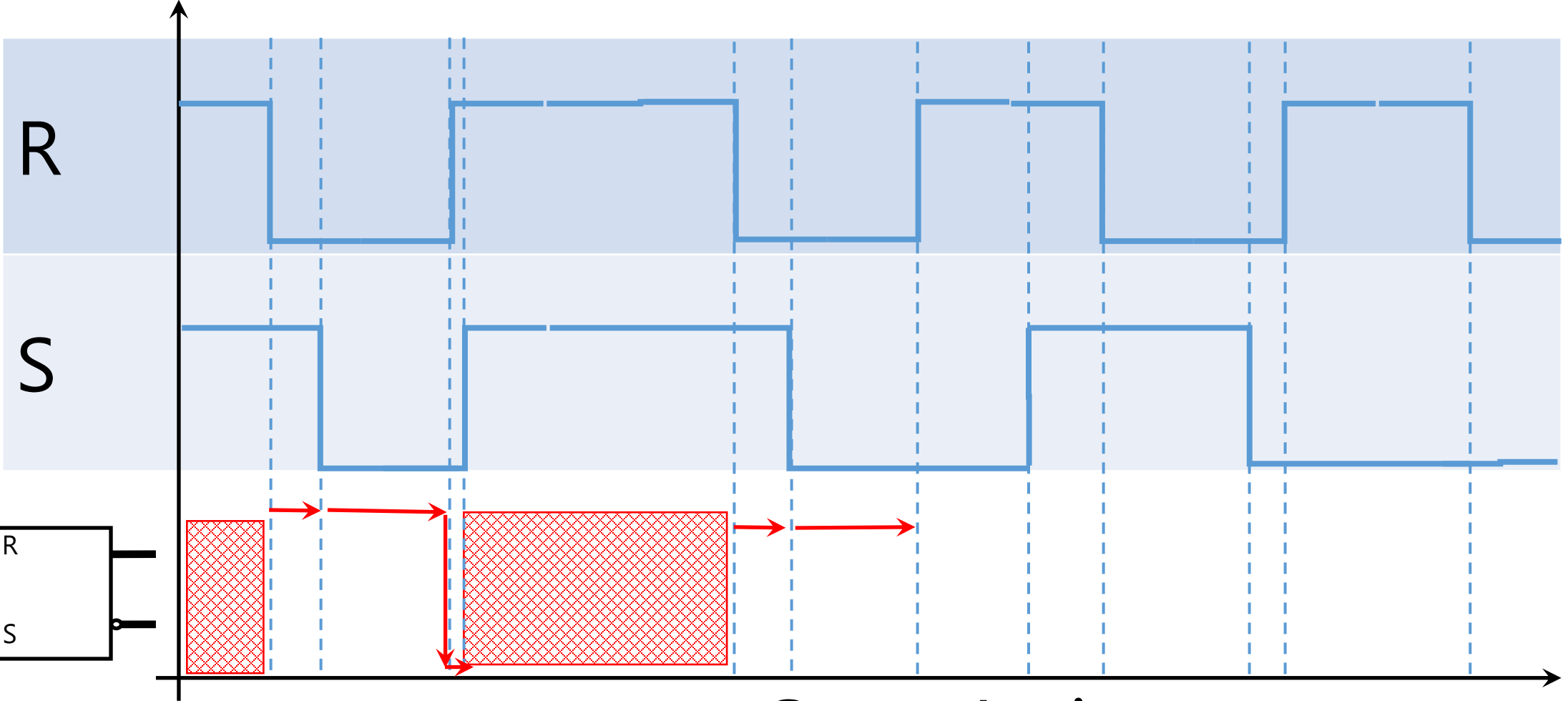
R

S

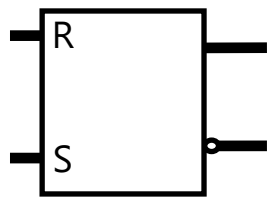
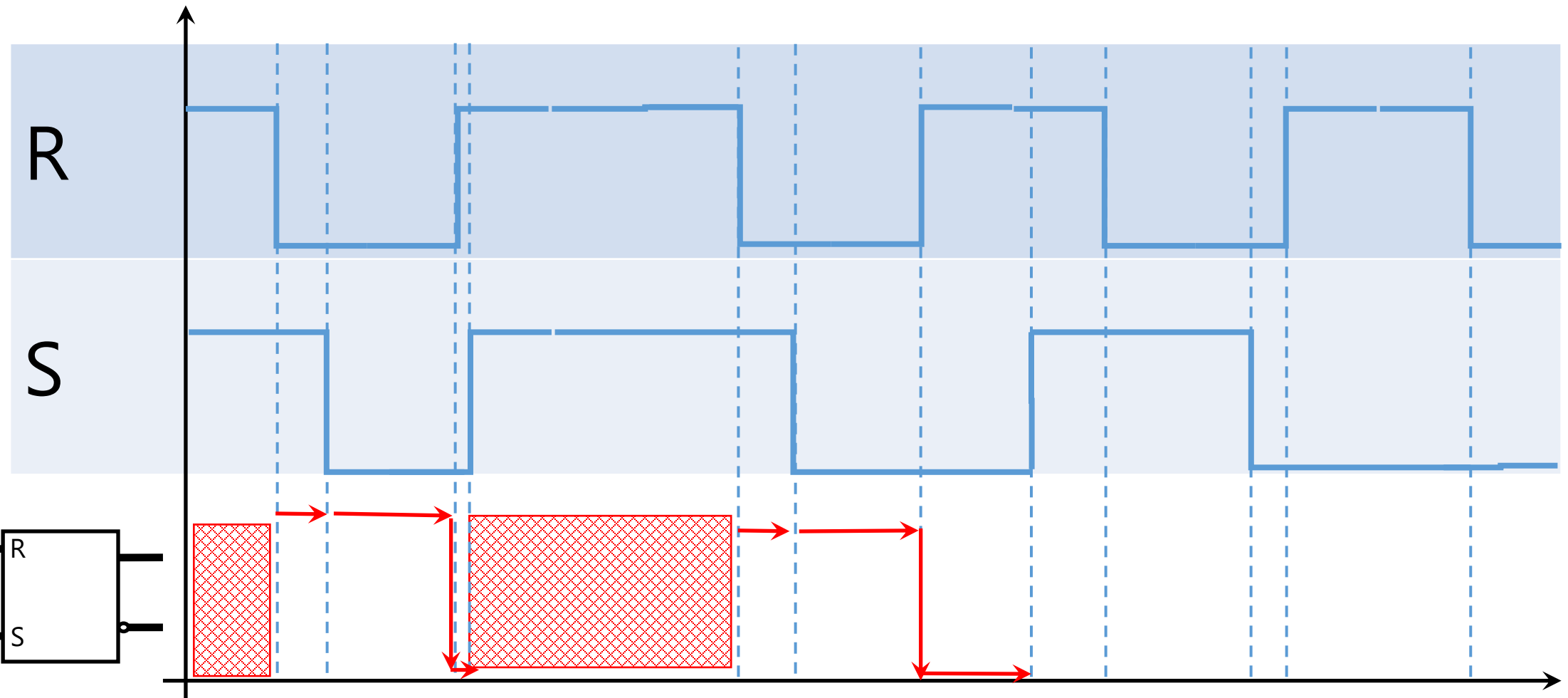


Store Action

Time



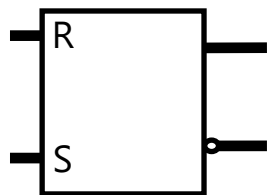
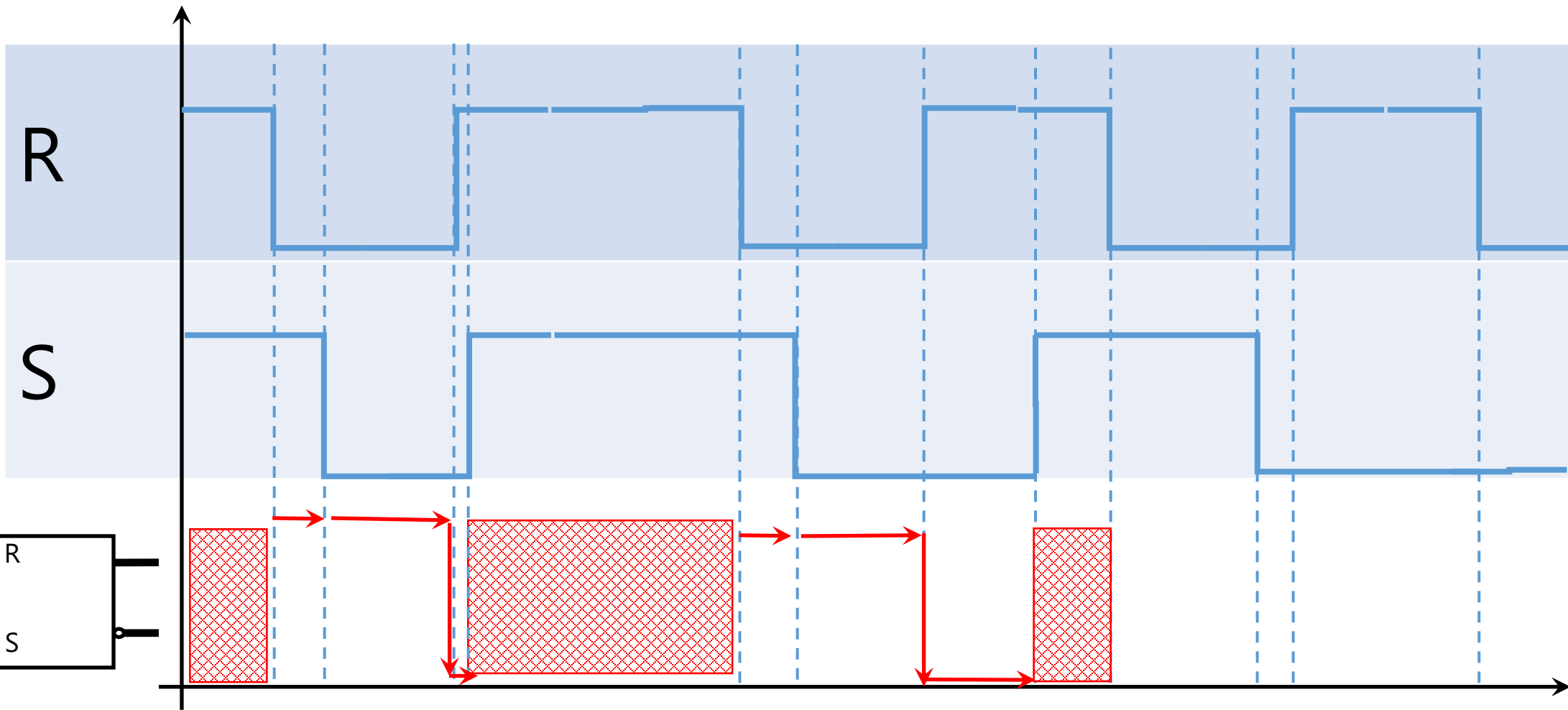
Voltage



Reset Action

Time

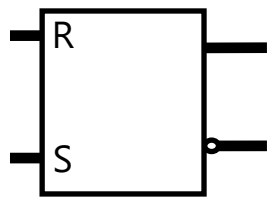
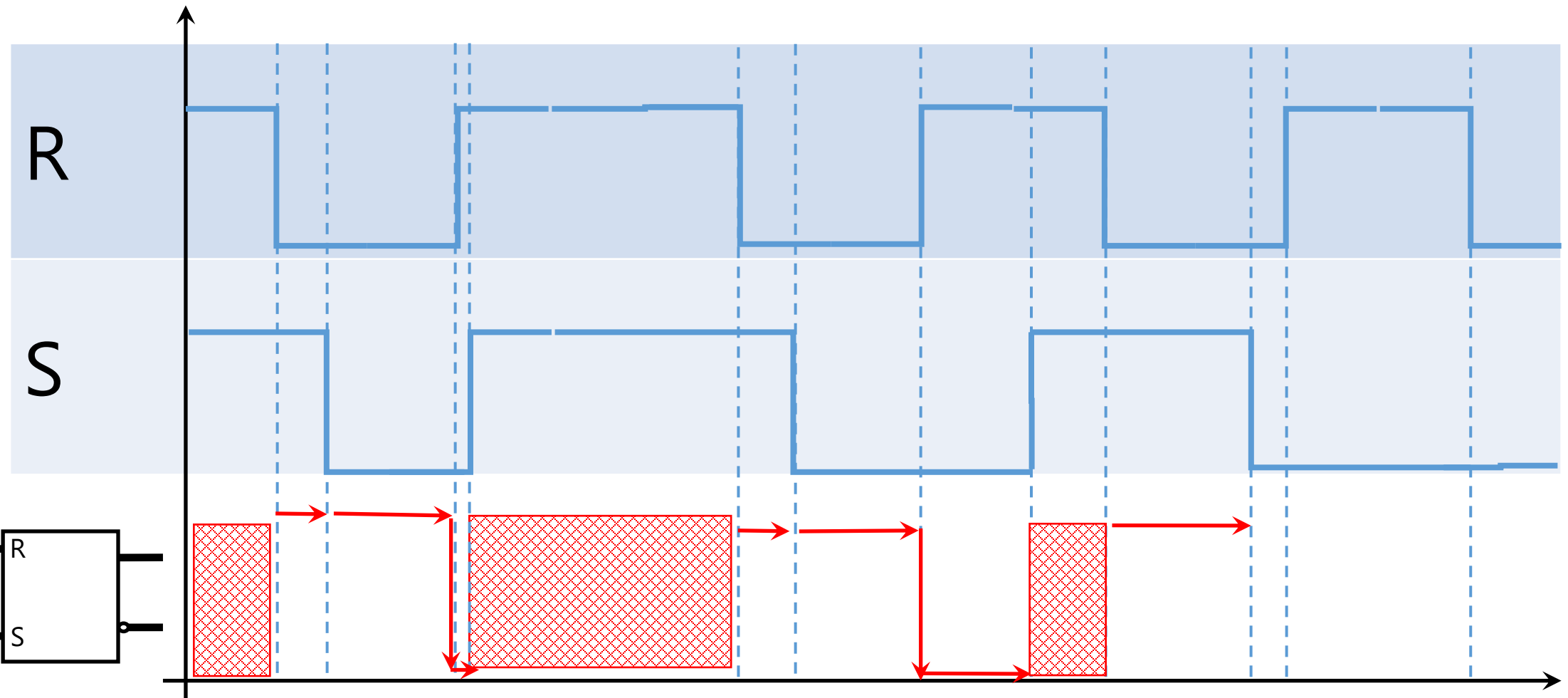
Voltage



Forbidden

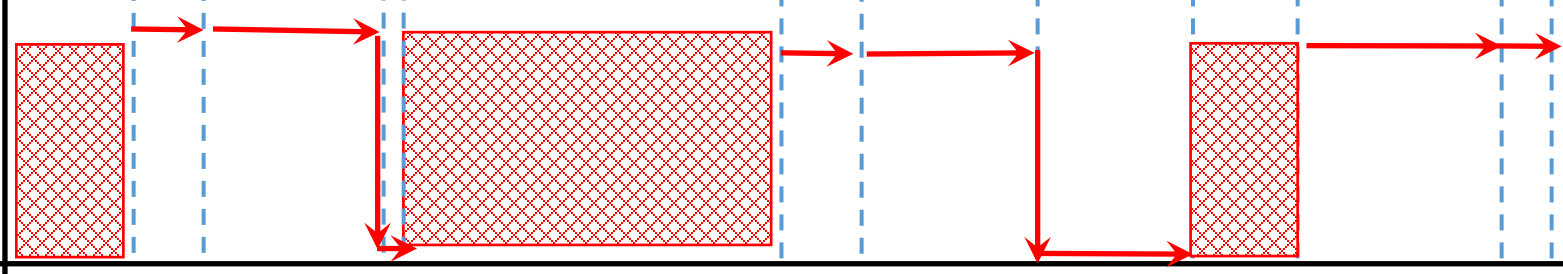
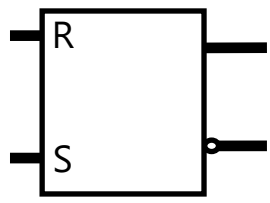
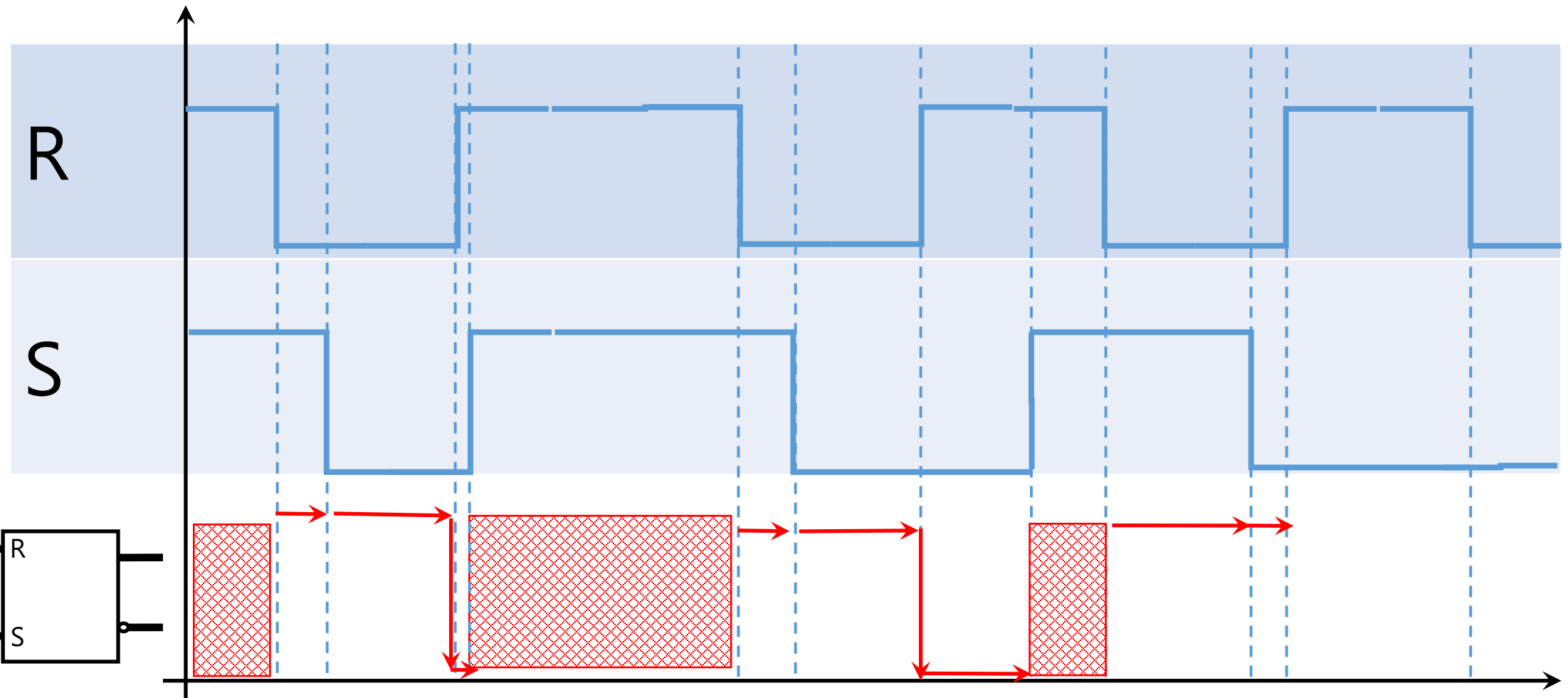
Time

Voltage



Set Action Time

Voltage

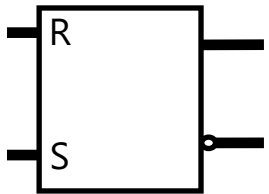


Time
Store Action

Voltage

R

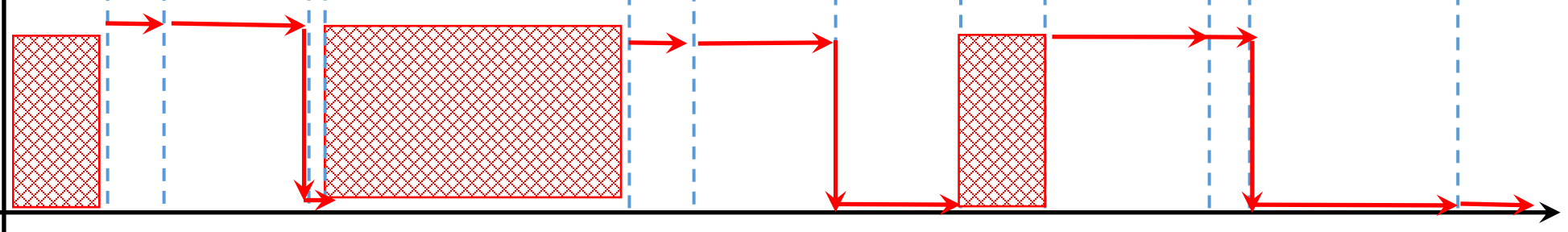
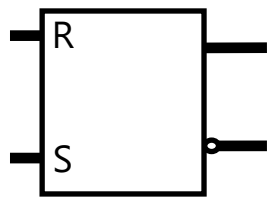
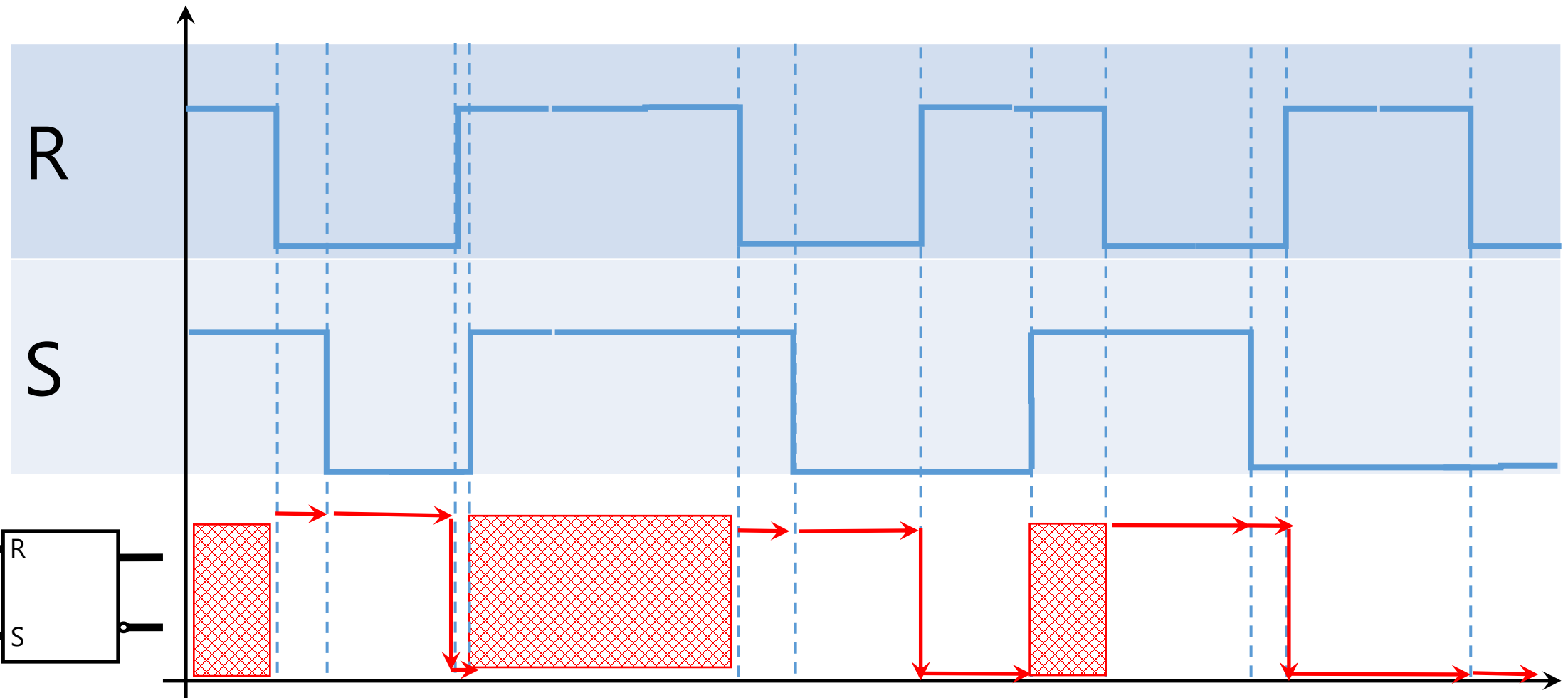
S



Time

Reset Action

Voltage



Time
Store Action

S and R control **how** the
state changes.



Put a gate/latch on **when** change applies
SR w/ enable input

SR *Latch*

Gated SR
